
LMGC90_GUI

Documentation Officielle

Version 0.4.5

© 2026 LMG90_GUI

Interface graphique · Matériaux · Modèles
Avatars · Boucles · Granulométrie
Conditions aux limites · Lois de contact
Maçonnerie · Corps déformables · Calcul
Post-traitement · Variables dynamiques · Architecture

TABLE DES MATIÈRES

1. Introduction à l'interface
2. Assistant de configuration
3. Matériaux
4. Modèles
5. Avatars — Corps rigides
6. Avatar Vide
7. Bibliothèque
8. Boucles paramétriques
9. Granulométrie
10. Conditions aux limites (DOF)
11. Lois de contact
12. Tables de visibilité
13. Assistant de maçonnerie
14. Corps déformables (EF)
15. Calcul (chipy)
16. Post-traitement
17. Variables dynamiques
18. Visualisation
19. Architecture & Guide développeur

Chapitre 1

Introduction à l'interface

Introduction to the LMGC90_GUI Graphical Interface

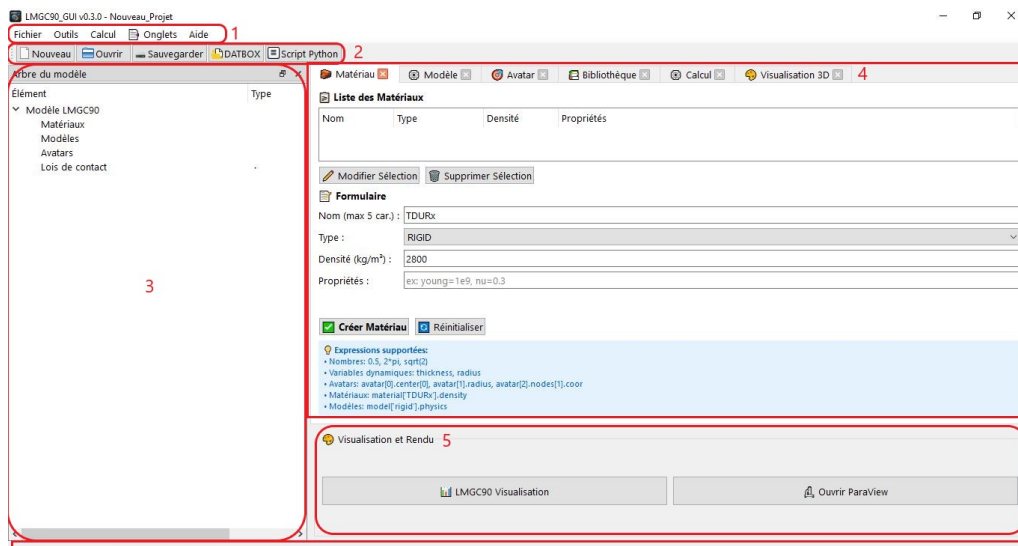
LMGC90_GUI is a modern graphical interface designed to facilitate the creation of numerical models with the **pre** (pre-processor) module of **LMGC90**.

It is also possible to launch your computations directly from LMGC90_GUI via the **chipy** module.

The interface is organized clearly and ergonomically to guide the user from start to finish of the modeling process: creating elements, boundary conditions, post-processing, file generation, and finally launching computations.

The following video presents an overview of the different parts of the interface.

Main Window



Global view of the interface

The interface is divided into **six main areas**:

1. Menu
2. Toolbar (top)
3. Model tree (left)
4. Creation tabs (top center)
5. Rendering area (bottom center)
6. Status bar (bottom)

1. Menu and Toolbar

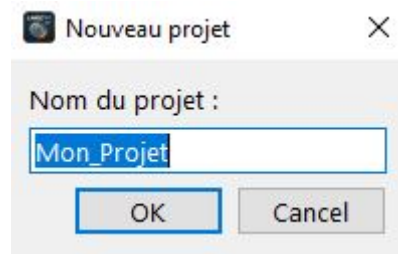
Menu

File

Action	Shortcut	Description
New	Ctrl+N	Creates a new project. A dialog box opens to enter the project name.
Open	Ctrl+O	Opens an existing project. A dialog box allows you to navigate to the project's <code>.json</code> file.
Save	Ctrl+S	Saves the current project to its current location.
Save as...	Ctrl+Shift+S	Saves the project under a new name or in a new location.
Quit	Ctrl+Q	Closes the application.

New project

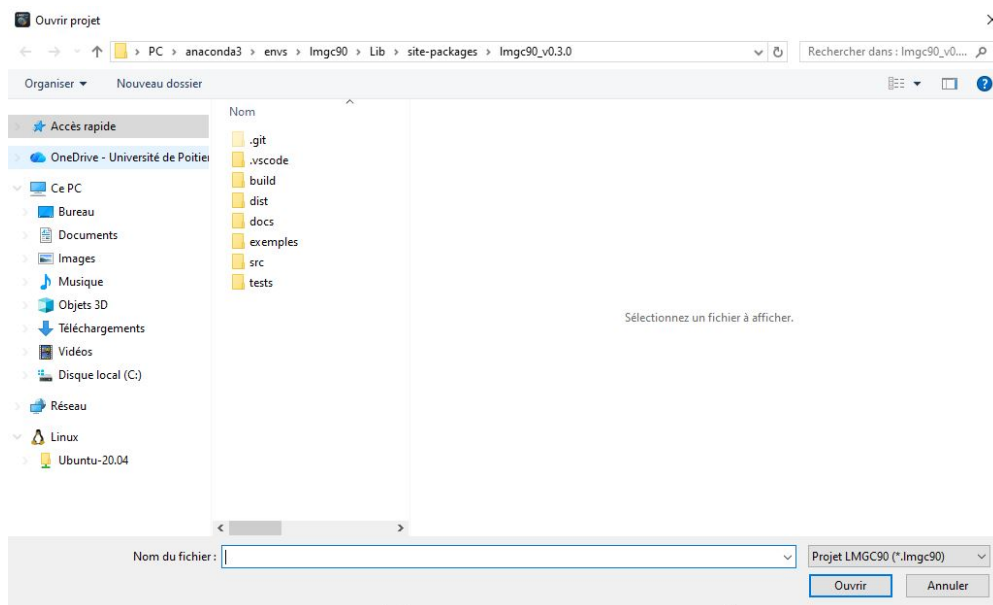
Click the **New** button in the toolbar, use the **File** → **New** menu, or press **Ctrl+N**. A dialog box opens to enter the project name.



captures/nouveau_projet.JPG

Open a project

Click the **Open** button in the toolbar, use the **File** → **Open** menu, or press **Ctrl+O**. You will then simply need to specify the path and name of your project.



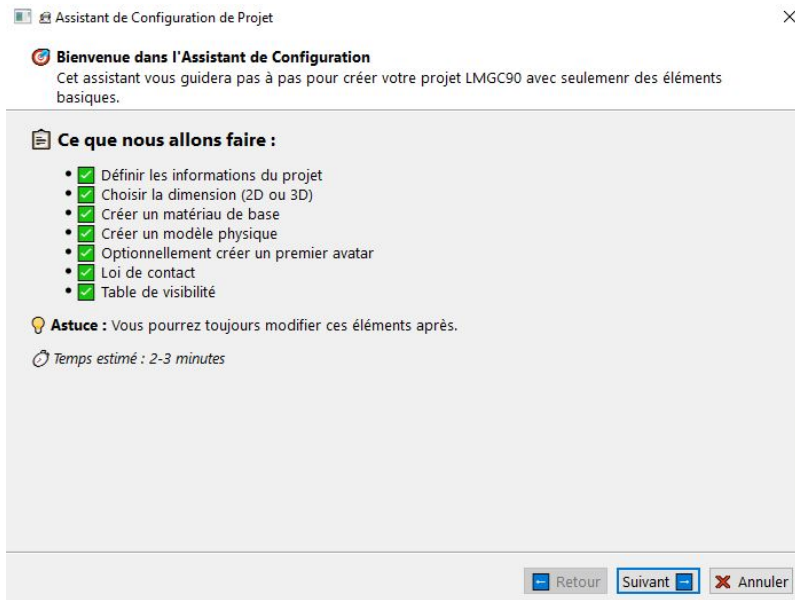
captures/ouvrir_projet.JPG

Save Used to save your projects to your hard drive. Click the **Save** button in the toolbar, use the **File** → **Save** menu, or use the keyboard shortcut **Ctrl+S**.

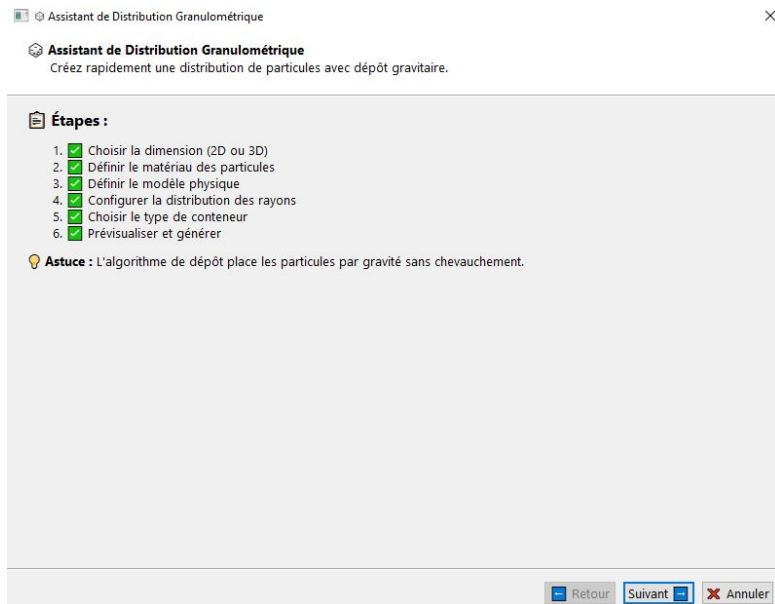
Wizards

Wizards guide the user step by step through the most common tasks. They can be used as many times as needed within the same project.

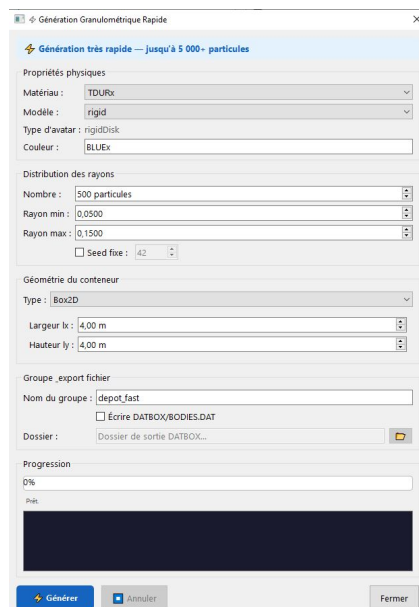
Wizard	Shortcut	Description
Project configuration	Ctrl+Shift+N	Guides step by step through the creation and initial configuration of a new project (material, model, reference avatar, contact law).
Granulometry pylmgc90	Ctrl+Shift+G	Generates a granulometric distribution with gravitational deposit via pylmgc90 routines. Recommended for assemblies of fewer than 8,000 avatars. Beyond that, the interface refresh may become slow.
Granulometry numpy _(beta)_	—	Generates and deposits avatars using numpy, without going through pylmgc90 routines. Recommended for assemblies of more than 5,000 avatars.
Deformable	Ctrl+Shift+D	Guides the creation or import of deformable elements (rectangular meshes, disks, spheres, cylinders, or external <code>.msh</code> / <code>.geo</code> files).
Masonry	Ctrl+Shift+M	Specialized in creating 2D and 3D brick stacks (<code>brick2D</code> / <code>brick3D</code>) according to different bonding patterns (standard, running bond, single stretcher bond, double stretcher bond, etc.).
Factory	Ctrl+Shift+F	Specialized in the 2D/3D avatar Factory, making them appear visually or in the computation.



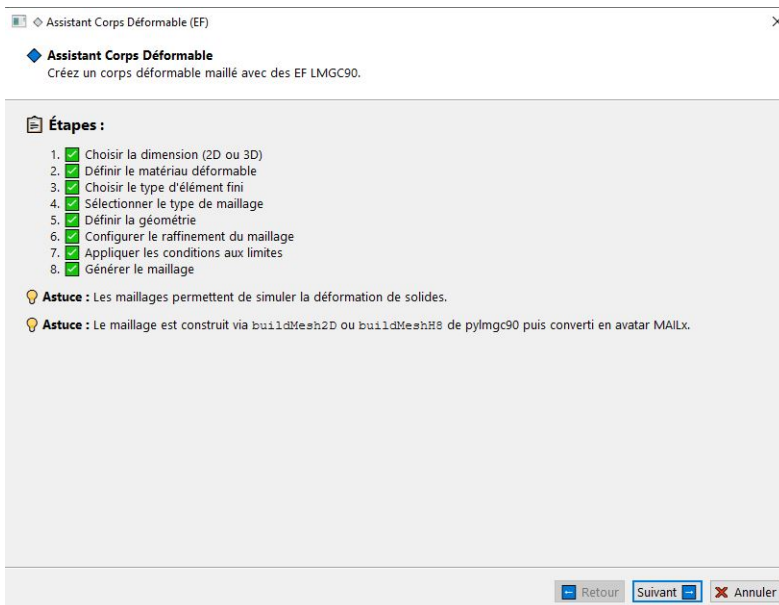
captures/assistant_projet.JPG



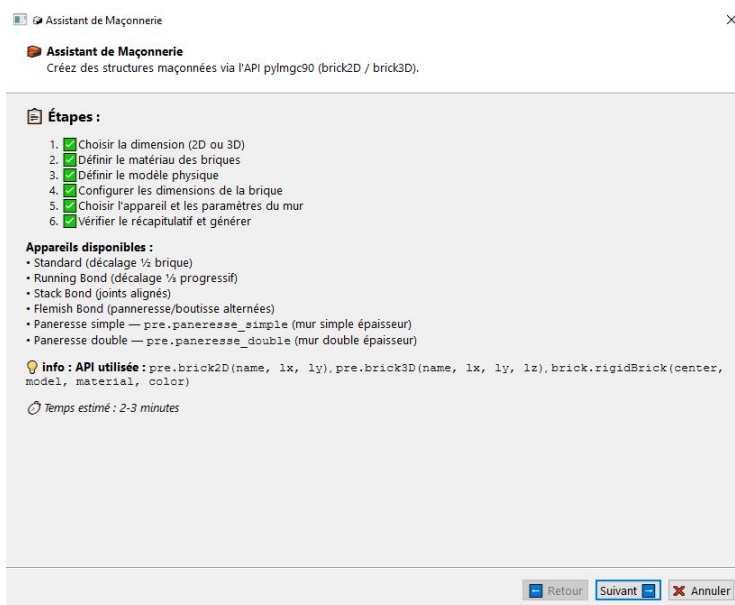
captures/assistant_granulo_pylmgc90.JPG



captures/assistant_granulo_numpy.JPG



captures/assistant_defor_page1.JPG



captures/assistant_maçon_page1.JPG

Note: wizards can be relaunched at any time during the session. Each run adds the generated elements to the existing project, without erasing what was previously created.

Duplicating elements with the same name causes errors, so be sure to name your elements differently.

Tools

Action	Shortcut	Description
Generate DATBOX	—	Generates the .dat files used by LMGC90 for the computation (DATBOX/).
Generate Python Script	—	Generates the pre-processing Python script (pre.py) reproducing the entire model built in the interface.

Action	Shortcut	Description
Convert lmgc90 script	Ctrl+Shift+C	A dialog box that scans a pylmgc90 python script and converts it into an .lmgc90 file beta version
Dynamic variables	Ctrl+V	Opens a dialog box allowing you to define reusable variables in the interface's numeric fields (radius, spacing, offset, etc.). It is also an inspection window for the properties of LMGC90 objects present in memory. See the Dynamic Variables page.
Preferences	Ctrl+,	Opens the application configuration dialog box. See the Preferences section.

Computation

Action	Shortcut	Description
Computation settings	Ctrl+F5	Opens the configuration dialog box for chipy routines: physics, contact detectors, extractions, control, inspection.
Run computation	F5	Launches the chipy computation directly from the interface, in a separate process so as not to block the interface.
Generate Computation Script	—	Generates the computation Python script (chipy.py) from the routine configuration.
View LMGC90 logs	F6	Displays the console output of the running computation in real time.
Application log	F7	Displays the internal log of LMGC90_GUI: unhandled errors, Python warnings, failed pylmgc90 calls. Useful for diagnosing problems that do not produce a visible message in the interface.

```

2026-03-10 13:51:12 [INFO] ] lmgc90_gui - =====
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - LMGC90_GUI - session démarrée le 2026-03-10T13:51:12.182912
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - Python 3.12.12 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 20:05:38)
[MSC v.1929 64 bit (AMD64)]
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - Platforms : win32
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - Exécutable : c:\Users\PC\anaconda3\envs\lmgc90\python.exe
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - Frozen : False
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - Fichier log : C:\
\Users\PC\AppData\Roaming\LMGC90_GUI\logs\lmgc90_gui_20260310_135112.log
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - =====
2026-03-10 13:51:12 [INFO] ] lmgc90_gui - main.py démarré

```

captures/journal_app.JPG

Tabs

Action	Shortcut	Description
Open	—	Opens a specific tab from the complete list. See the Creation tabs section.
Close others	—	Closes all open tabs except the active tab.
Close all (except essential)	—	Closes all non-essential tabs.
Default tabs	Ctrl+Alt+D	Restores the default tab layout.

Help

Action	Description
About	Displays information about the LMGC90_GUI version and dependencies.
Online help	Opens the online documentation in the default browser.

Toolbar

The toolbar groups together the most frequently used actions for quick access:

Button	Menu equivalent
New project	File → New
Open project	File → Open
Save project	File → Save
Generate Python Script	Tools → Generate Python Script
Generate DATBOX	Tools → Generate DATBOX _(since v0.2.6)_

2. Model Tree (left)

A fixed area displaying the **complete tree structure of the current model**. It updates automatically after each creation, modification, or deletion of an element.

Sections displayed

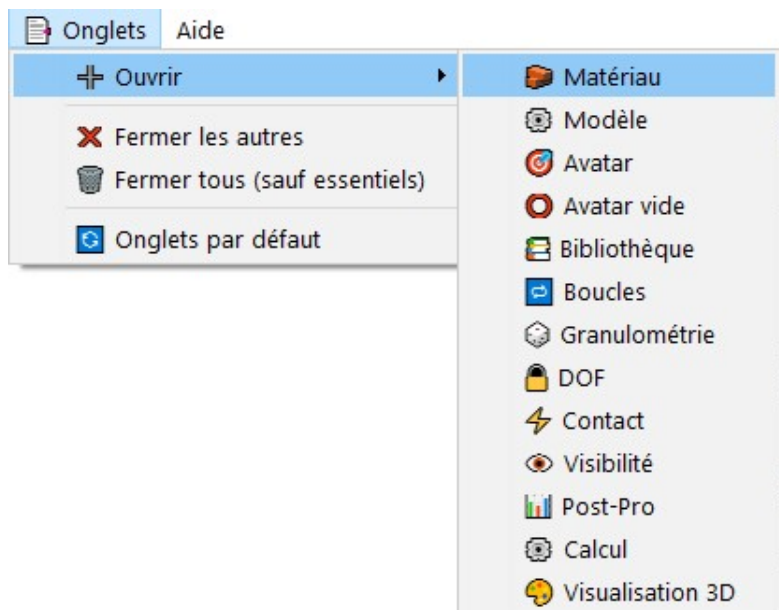
Section	Content
Materials	List of all materials defined in the project.
Models	List of all finite element models (physics, element, dimension).
Avatars	List of all bodies in the project: rigid, empty, deformable.
Avatar groups	Groups created by loops, granulometry, or the masonry wizard.
Contact laws	Defined contact behavior laws (friction, cohesion, stiffness).
Visibility tables	chipy visibility rules for avatars during the computation.
PostPro	Configured post-processing commands.

Features

- **Click on an element:** opens the corresponding tab in edit mode and loads the element into the form.
- **Right-click:** context menu with Edit, Delete, and Information actions depending on the selected element.
- **Hierarchical view:** avatar groups can be expanded to display the member avatars.

3. Creation Tabs (upper central area)

The main working area. Each tab is dedicated to a modeling step. To open a tab, use the **Tab** → **Open** menu and choose the desired tab.



captures/onglets.jpg

Tab	Shortcut	Description
Material	Ctrl+1	Creation and management of materials (RIGID, ELAS, ELAS_PLAS, THERMO_ELAS, PORO_ELAS, etc.).
Model	Ctrl+2	Definition of physical models and finite elements (MECAx, THERx, POROx, MULTI).
Avatar	Ctrl+3	Creation of standard rigid bodies: disk, jonc, polygon, rough wall, sphere, cylinder, polyhedron, etc.
Empty avatar	Ctrl+4	Creation of avatars with custom contactors, or addition of contactors to an existing deformable body.
Libraries	Ctrl+5	Preconfigured avatars (complex shapes, common assemblies) ready to be inserted into the project.
Loops	Ctrl+6	Parametric generation of series of avatars: circle, grid, line, spiral, or manual placement.
Granulometry	Ctrl+7	Generation of deposits with statistical distribution of radii and gravitational deposit.

Tab	Shortcut	Description
DOF	Ctrl+8	Boundary conditions: imposed translations, blocked rotations, imposed velocities, degree-of-freedom couplings.
Contact	Ctrl+9	Definition of contact behavior laws (Coulomb friction, cohesion, normal and tangential stiffness).
Visibility	—	Creation of chipy visibility tables to show or hide avatars during the computation.
Postpro	—	Configuration of post-processing commands: energy balance, body tracking, field extraction.
3D Visualization	—	Interactive display of the model's avatars with navigation, selection, and measurement modes.

Keyboard shortcuts: the keys **Ctrl+1** through **Ctrl+9** directly open the first nine tabs in the list.

4. Rendering Area (lower central area)

An area dedicated to visualization and computation outputs.

Available buttons

Button	Description
LMGC90 Visualization	Launches the integrated pylmgc90 visualization via <code>pre.visuAvatars()</code> . Opens an independent external window.
ParaView	Automatically opens the computation's output files in ParaView (by default <code>rigids.pvd</code>). Requires ParaView to be installed on the machine and a computation to have already been performed.

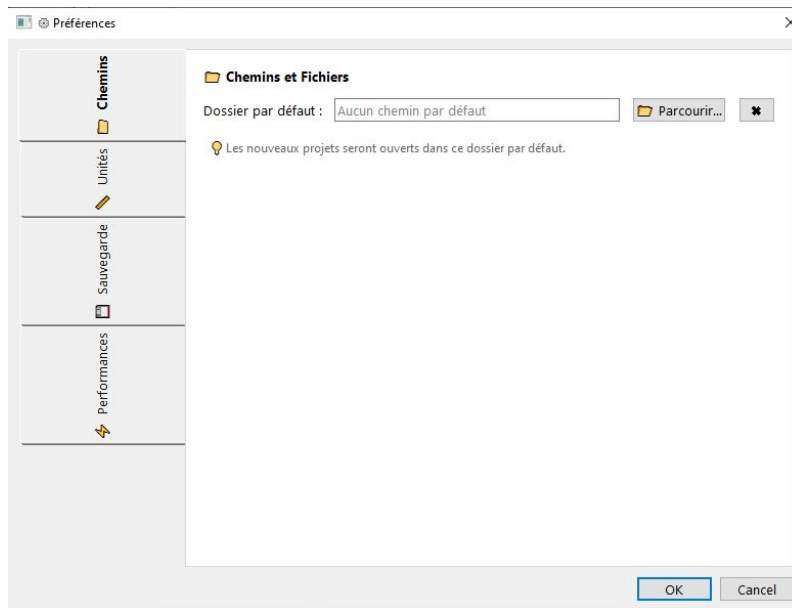
Interactive 3D viewer modes

Mode	Description
■■■ Navigation	Default mode: rotation (left-click + drag), zoom (scroll wheel), pan (right-click + drag).
■ Selection	Click on an avatar to highlight it (yellow highlight) and display its information in the status bar.
■ Ruler	Distance measurement: click on a first point (A) then a second point (B) to display the distance in meters.

The quick views **XY**, **XZ**, **YZ**, and **Iso** are accessible from the viewer's toolbar.

5. Preferences

Accessible via **Tools** → **Preferences** or the shortcut **Ctrl+,**. The preferences dialog box brings together the application's configuration settings.

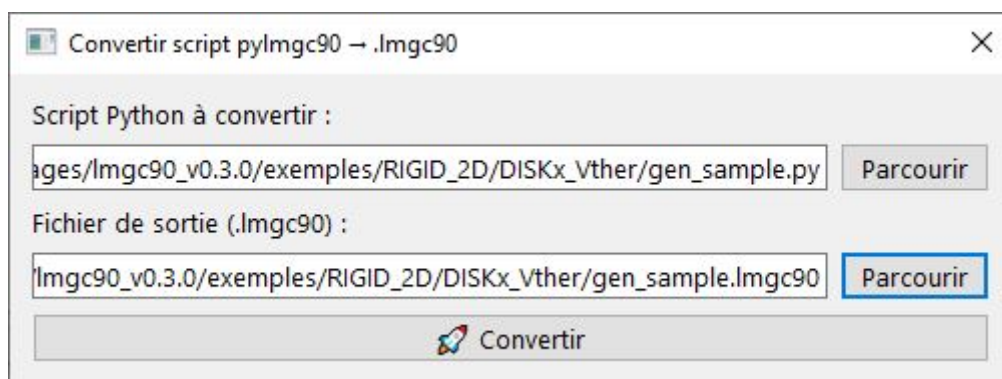


captures/preferences.JPG

Setting	Description
Projects folder	Default path used when opening and saving projects. Click Browse to change it.
Unit system	Choice between SI (meter, kilogram, second) and CGS (centimeter, gram, second). _(Not yet implemented in this version.)_
Automatic save	Options to enable automatic saving at regular intervals and when closing the application.
Recent projects history	Maximum number of projects kept in the File → Recent projects list.
Avatar display	Enables or disables the display of avatars in the model tree and in the Avatar tab table. Disabling this option improves performance on projects with a large number of avatars.

Convert

LMGC90_GUI includes a beta converter (python script) to .lmgc (JSON). You simply need to provide the path to your script, then the name of your output file, and click the "Convert" button.



captures/convert.JPG

Note: translation quality depends on the complexity of your script.

Keyboard Shortcuts Summary

Shortcut	Action
Ctrl+N	New project
Ctrl+O	Open a project
Ctrl+S	Save
Ctrl+Shift+S	Save as...
Ctrl+Q	Quit
Ctrl+Shift+N	Project configuration wizard
Ctrl+Shift+G	Granulometry pylmgc90 wizard
Ctrl+Shift+D	Deformable wizard
Ctrl+Shift+M	Masonry wizard
Ctrl+V	Dynamic variables
Ctrl+,	Preferences
Ctrl+F5	Computation settings
F5	Run computation
F6	View LMGC90 logs
F7	Application log
Ctrl+Alt+D	Default tabs
Ctrl+1 ... Ctrl+9	Open the corresponding tab

6. Status Bar

A horizontal strip at the bottom of the window displaying contextual messages about ongoing operations: creation of an avatar, generation of a script, measurement result in the 3D viewer, validation error, etc.

LMGC90_GUI is designed to be **intuitive** and **fully visual**, while maintaining full compatibility with traditional LMGC90 Python scripts.

Assistant de configuration

Project Configuration Wizard

The **Project Configuration Wizard** guides you step by step through the creation of a complete LMGC90 project with all its essential elements: material, model, test avatar, contact law, and visibility table.

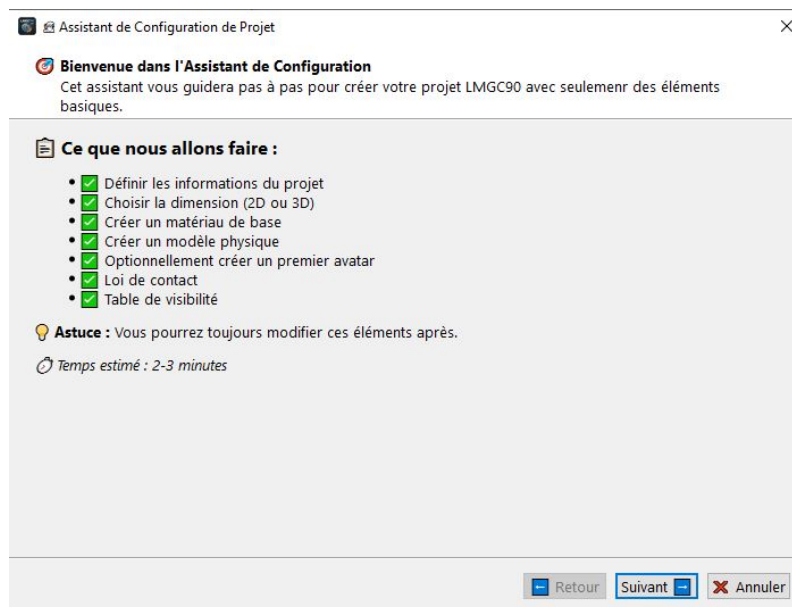
It is particularly useful for quickly starting a new project without having to configure each tab separately.

■ ■ *Estimated time: 2 to 3 minutes*

Launching the Wizard

Three ways to open the wizard:

Method	Action
Menu	File → Project Wizard... or Wizards → Project Configuration
Toolbar	Dedicated button (depending on the toolbar configuration)
Keyboard shortcut	Ctrl+Shift+N



Opening the wizard

Cancellation possible at any time: clicking ■ **Cancel** at any step closes the wizard without modifying the project. The project state (name, dimension, path) is fully restored.

Overview of Steps

The wizard consists of **9 pages** navigated sequentially. The **■ Back** and **Next ➡■** buttons allow free navigation between pages.

Step	Page	Description	Mandatory
0	Introduction	Presentation of the wizard	—
1	Project Information	Name and description	■ Yes (name required)
2	Dimension	2D or 3D	■ Yes
3	Material	Create or reuse a material	■ Yes
4	Model	Create or reuse a physical model	■ Yes
5	Avatar	Create a first test avatar	■ Optional
6	Contact Law	Define the contact behavior	■ Optional
7	Visibility Table	Define who interacts with whom	■ Optional (requires steps 5 and 6)
8	Summary	Verification before creation	—

Page 0 — Introduction

Welcome page presenting the upcoming steps. No input required.

Click **Next ➡■** to begin.

Page 1 — Project Information

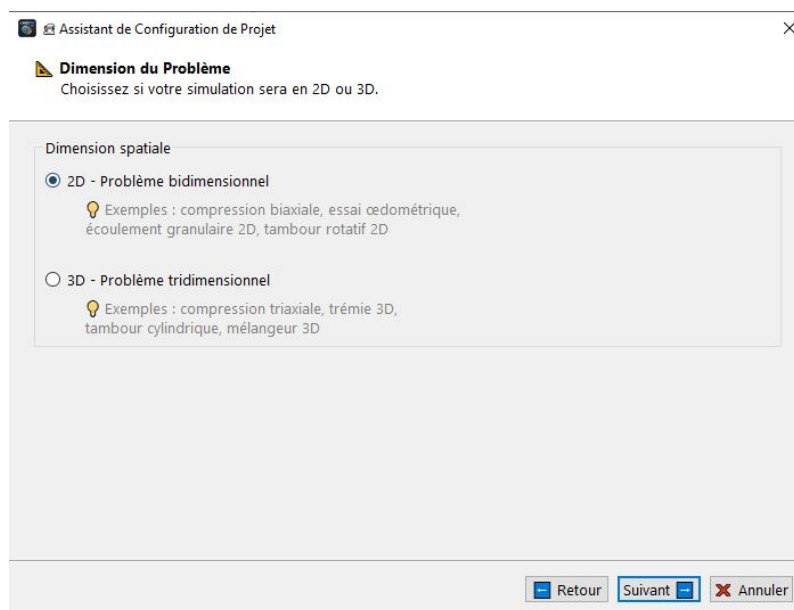
Project name page

Field	Description	Constraints
Project name	Project identifier. Used as the file name when saving.	Required — 50 characters maximum. The Next button is disabled as long as this field is empty.

Field	Description	Constraints
Description	Free text describing the purpose or context of the project.	Optional

Name required: the Name field is marked with an asterisk (*) in the wizard — it is a mandatory field. The **Next** ➡ button remains grayed out as long as it is empty.

Page 2 — Problem Dimension



Dimension page

Choose between two mutually exclusive options (radio buttons):

Choice	Internal Code	Usage Examples
2D — Two-dimensional problem	<code>dimension = 2</code>	Biaxial compression, oedometric test, 2D granular flow, 2D rotating drum
3D — Three-dimensional problem	<code>dimension = 3</code>	Triaxial compression, 3D hopper, cylindrical drum, 3D mixer

The **2D** value is selected by default.

Effect on subsequent steps: the dimension chosen here automatically determines: - The list of elements offered at the Model step (`Rxx2D` or `Rxx3D`) - The type of avatar offered at the Avatar step (`rigidDisk` or `rigidSphere`) - The body and contactor type in the visibility table (`RBDY2/DISKx` or `RBDY3/SPHER`)

Page 3 — Material

Assistant de Configuration de Projet

Matériau

Créez un matériau de base pour vos particules.

☒ Créer un matériau maintenant

Paramètres du matériau

Nom (max 5 car.) :

rockx

Type :

RIGID

Densité :

2500,00 kg/m³

Conseil : Pour des simulations granulaires simples, utilisez RIGID avec une densité typique de 2500 kg/m³ (sable/gravier).

Retour

Suivant

Annuler

Material page

This page offers two modes depending on the state of the project:

Mode A — Use an Existing Material (if the project already contains materials)

A dropdown list displays all the materials already defined in the Material tab. Select one of them to associate it with the project without creating a new one.

Mode B — Create a New Material (automatically checked if no material exists)

Check **Create a new material instead** to display the creation form:

Field	Description	Default Value
Name	Material identifier. 5 characters maximum.	rockx
Type	Type of mechanical behavior.	RIGID
Density	Density (kg/m³).	2500 kg/m³

Wizard tip: for simple granular simulations, use the **RIGID** type with a density of 2,500 kg/m³ (typical sand/gravel). Elastic properties are not configurable in the wizard — use the Material tab for the **ELAS**, **ELAS_PLAS**, etc. types.

Page 4 — Physical Model

Assistant de Configuration de Projet

Modèle Physique

Créez un modèle de comportement mécanique.

☒ Créer un modèle maintenant

Paramètres du modèle

Nom (max 5 car.) :

rigid

Physique :

MECAx

Élément :

Rxx2D

Conseil : Pour des corps rigides, utilisez Rxx2D (2D) ou Rxx3D (3D).

Retour

Suivant

Annuler

Model page

Same logic as the Material page: reuse an existing model or create a new one.

Mode A — Use an Existing Model

Dropdown list of models already defined in the Model tab.

Mode B — Create a New Model

Field	Description	Default Value
Name	Model identifier. 5 characters maximum.	rigid
Physics	Physics family.	MECAx (only option in the wizard)
Element	Finite element type. Adapted automatically to the dimension.	Rxx2D (2D) or Rxx3D (3D)

Wizard tip: for rigid bodies (DEM), use Rxx2D in 2D or Rxx3D in 3D. These elements have no numerical options and are the simplest to configure. For deformable finite element models, create the model directly in the Model tab after finishing the wizard.

Page 5 — First Avatar _(optional)_

Avatar page

This page is **optional**. It allows you to create a test avatar positioned at the origin (center = $[0, 0]$ in 2D or $[0, 0, 0]$ in 3D).

Check **Create a test avatar** to display the form:

Field	Description	Default Value
Type	Avatar type. Adapted automatically to the dimension.	rigidDisk (2D) or rigidSphere (3D)
Radius	Radius of the disk or sphere (m).	0.1 m
Color	5-character LMGC90 color code.	BLUEx

Dependency: the avatar automatically uses the material and model defined in the previous steps. If no valid material or model is available, the avatar will not be created even if the box is checked.

Position: the avatar is created at the origin of the coordinate system. Modify its position after creation via the Avatar tab.

Page 6 — Contact Law (optional)

Assistant de Configuration de Projet

Loi de Contact
Définissez comment les particules interagissent.

☐ Créer une loi de contact

Paramètres de la loi

Nom :

Type de loi :

Coefficient de friction :

IQS_CLB : Loi de contact avec friction de Coulomb
Friction typique : 0.3 (sable), 0.5 (gravier), 0.1 (surfaces lisses)

Contact law page

Defines the mechanical behavior during contacts between avatars. Check **Create a contact law** (checked by default) to display the form:

Field	Description	Default Value
Name	Law identifier. Up to 20 characters.	iqsc0
Law type	Type of contact behavior. See table below.	IQS_CLB
Friction coefficient	Visible only for laws with Coulomb friction.	0.3

Available Law Types

Type	Full Name	Friction	Description
IQS_CLB	Quasi-Static Unilateral Contact + Coulomb	■ Yes	Standard non-smooth rigid law. The most common for DEM simulations.
IQS_CLB_G0	IQS_CLB with zero gap	■ Yes	Variant with zero initial gap.
COUPLED_DOF	Coupled degrees of freedom	■ No	Kinematic coupling between bodies.
IQS_DS_CLB	Discrete Rigid Contact + Coulomb	■ No	Discrete law with normal and tangential stiffnesses.
IQS_MOHR_DS_CLB	Discrete Mohr-Coulomb	■ No	Mohr-Coulomb criterion for joints or brittle interfaces.
IQS_MAC_CZM	Cohesive Zone	■ No	Cohesive zone law for cracking.
ELASTIC_WIRE	Elastic cable	■ No	Unilateral tension link (cable).
BRITTLE_ELASTIC_WIRE	Brittle elastic cable	■ No	Cable with brittle failure beyond a threshold.
ELASTIC_ROD	Elastic rod	■ No	Bilateral link in tension and compression (rod).
ELASTIC_REPELL_CLB	Elastic repulsion + Coulomb	■ No	Repulsive contact with friction.

Typical Friction Values:

Material	Friction Coefficient
Smooth surfaces	0.1
Fine sand	0.3
Gravel	0.5
Rough concrete	0.6–0.8

Friction is only configurable for the IQS_CLB and IQS_CLB_G0 laws. For other types, the field is automatically hidden.

Page 7 — Visibility Table (optional)

Assistant de Configuration de Projet

Table de Visibilité

Définissez quels avatars peuvent interagir entre eux.

☐ Créer une table de visibilité

La table de visibilité définit quels contacteurs peuvent se voir et avec quelle loi.
Par défaut, tous les avatars de même couleur interagissent entre eux.

Configuration de visibilité

Couleur candidat : BLUEx

Couleur antagoniste : BLUEx

Distance d'alerte : 0,10 m

Configuration automatique :

• Corps : RBDY2 (2D) ou RBDY3 (3D)

• Contacteur : DISKx (2D) ou SPHER (3D)

• Loi : celle créée à l'étape précédente

Retour

Suivant

Annuler

Visibility page

The visibility table defines **which contactors can detect each other** and with which contact law. Check **Create a visibility table** (checked by default) to display the form:

Field	Description	Default Value
Candidate color	Color of the candidate contactors (active body).	BLUEx (synchronized with the avatar's color if step 5 is active)
Antagonist color	Color of the antagonist contactors (passive body).	BLUEx
Alert distance	Maximum contact detection distance (m). Beyond this, two bodies are not considered to be in potential contact.	0.1 m

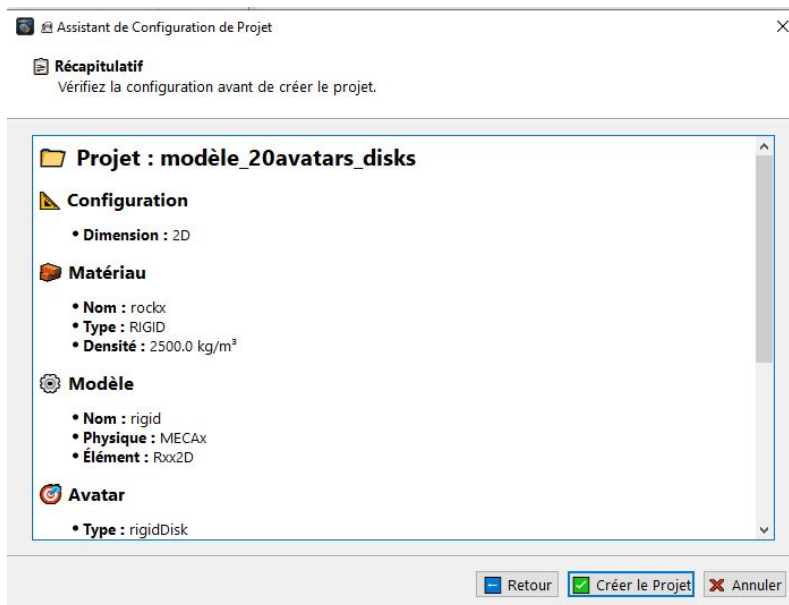
Automatic Configuration Based on Dimension:

Parameter	2D	3D
Body type	RBDY2	RBDY3
Contacteur type	DISKx	SPHER
Contact law	The one created at step 6	The one created at step 6

Dependency: the visibility table is only created if a contact law was defined at the previous step **and** an avatar was created at step 5. If either of these conditions is not met, the table is ignored even if the box is checked.

Color synchronization: if an avatar was created at step 5, the candidate and antagonist colors are automatically pre-filled with that avatar's color.

Page 8 — Summary



Summary page

The last page displays a complete summary of all the elements that will be created. Review the information before validating.

Summary Content

Section	Information Displayed
Project	Project name, dimension (2D/3D)
Material	Name, type, density — or selected existing material
Model	Name, physics, element — or selected existing model
Avatar	Type, radius — or "No avatar created"
Contact Law	Name, type, friction coefficient — or "No law created"
Visibility Table	Body, contactor, colors, applied law, alert distance — or "No table created"

Click **Create the Project** to finalize. All elements are created simultaneously and immediately appear in the model tree and the corresponding tabs.

In case of error: if creation fails (duplicate material name, invalid parameter, etc.), an error box is displayed with details of the problem. The project state is fully restored to what it was before the wizard was opened.

After Creation

Once the wizard is finished, all generated elements can be freely modified via their respective tabs:

Created Element	Tab to Edit
Material	Material (Ctrl+1) — select from the list and click  Edit
Model	Model (Ctrl+2)
Avatar	Avatar (Ctrl+3)
Contact Law	Contact (Ctrl+9)
Visibility Table	Visibility

Shortcuts Summary

Action	Shortcut
Open the wizard	Ctrl+Shift+N
Next page	Enter or Next 
Previous page	 Back
Cancel	 Cancel (restores the previous state)
Create the project	 Create the Project (last page)

Usage Tips

Start simple: for a first 2D LMGC90_GUI project, use the **RIGID** type for the material, **Rxx2D** for the model, create a test disk, and the **IQS_CLB** law with a friction of 0.3. The project will be functional in under 2 minutes.

Reuse existing elements: if a material or model has already been created in the current project, the wizard automatically offers it as the first option. There is no need to create a new one every time.

Complete afterward: the wizard creates basic elements. For your advanced configurations (deformable finite elements, plastic laws, boundary conditions, granular loops), use the specialized tabs directly after finishing the wizard.

Creating a Material

The **Material** tab explains how to create/edit/delete and configure a material in LMGC90_GUI.

Interface

The **Material** tab is divided into two parts:

- **List of materials** (top): a table displaying all the materials defined in the project with their name, type, density, and a preview of the properties. Materials used by at least one avatar are displayed in green.
- **Creation / editing form** (bottom): input fields adapted to the selected type.

Form Fields

Field	Description
Name	Unique identifier of the material. 5 characters maximum (internal LMGC90 constraint). Examples: TDURx , ROCKx , steel , BEton .
Type	Dropdown list of supported material types. Determines the expected properties in the next field.
Density	Density in kg/m ³ (SI system). Default value: 2800 .
Properties	Free-text field for type-specific parameters, in the format key=value , key=value . Automatically filled with a consistent suggestion each time the type is changed.

Automatic suggestion: each time a new type is selected, the Properties field and the name are automatically filled with typical values. These values serve as a starting point and must be adapted to the actual material.

Available Material Types

LMGC90_GUI offers **10 types** of materials, corresponding to the types accepted by `pre.material(materialType=...)` in `pymgc90`.

Note: types marked *_(advanced)_* — **DISCRETE**, **USER_MAT**, **EXTERNAL** — do not have automatic suggestions in the interface. Their parameters must be entered manually.

Parameters Table

Type	Full Name	Main Parameters	Compatible Physics
RIGID	Rigid body	_(none)_	MECAx
ELAS	Linear elastic	elas , young , nu , anisotropy	MECAx

Type	Full Name	Main Parameters	Compatible Physics
ELAS_DILA	Elastic with thermal dilation	elas, young, nu, anisotropy, dilatation, T_ref_meca	MECAx, THERx
VISCO_ELAS	Visco-elastic	elas, young, nu, anisotropy, viscous_model, viscous_young, viscous_nu	MECAx
ELAS_PLAS	Elasto-plastic	elas, young, nu, anisotropy, critere, isoh, iso_hard, isoh_coeff, cinh, visc	MECAx
THERMO_ELAS	Coupled thermo-elastic	elas, young, nu, anisotropy, dilatation, T_ref_meca, conductivity, specific_capacity	THERx, THMx
PORO_ELAS	Poro-elastic (Biot)	elas, young, nu, anisotropy, hydro_cpl, conductivity, specific_capacity	POROf, THMx
DISCRETE _(advanced)_	Discrete elements (mass-spring)	masses, stiffnesses, viscosities	MECAx
USER_MAT _(advanced)_	Custom constitutive law	density, file_mat	MECAx
EXTERNAL _(advanced)_	Interface with external code	_(defined by the external code)_	MECAx

Table of Typical Uses

Type	Typical Applications	Concrete Examples	Fields
RIGID	Discrete Element Method (DEM)	Grain stacks, granular flows, particle assemblies	Civil engineering, pharmaceuticals, food industry
ELAS	Structures in the elastic regime	Buildings, bridges, mechanical parts, metal structures	Civil engineering, mechanics
ELAS_DILA	One-sided thermal stresses	Structures subject to temperature variations, differential dilation	Construction, mechanics, electronics
VISCO_ELAS	Materials with viscous behavior	Polymers, asphalt, damping materials, seals	Roads, automotive, aerospace
ELAS_PLAS	Permanent plastic deformations	Metal forming, impact, damage, machining	Metallurgy, automotive, aerospace
THERMO_ELAS	Full thermo-mechanical coupling	Thermal dissipation, thermal shocks, braking	Electronics, automotive, nuclear
PORO_ELAS	Saturated porous media	Soil consolidation, oil reservoirs, aquifers, CO ₂ storage	Geotechnics, hydrogeology, oil
DISCRETE	Mass-spring-damper systems	Seismic isolators, suspensions, discrete elastic links	Earthquake engineering, automotive
USER_MAT	Custom constitutive laws	Specific materials, experience-derived laws	Research, innovative materials

Type	Typical Applications	Concrete Examples	Fields
EXTERNAL	Coupling with an external code	Interface with other simulation software	Multi-physics simulation

Detailed Properties by Type

RIGID — Rigid Body

No property parameter is required. The Properties field must remain empty.

```
Name : BRIQx
Type : RIGID
Density : 2000
Properties: (empty)
```

ELAS — Linear Elastic

```
elas='standard', young=2.1e11, nu=0.3, anisotropy='isotropic'
```

Parameter	Description	Typical Value
elas	Elastic formulation: always 'standard'.	'standard'
young	Young's modulus (Pa).	Steel: 2.1e11 · Concrete: 3e10 · Rock: 5e10
nu	Poisson's ratio (dimensionless).	0.2 to 0.35
anisotropy	Type of anisotropy: 'isotropic' or 'orthotropic'.	'isotropic'
G	Shear modulus (Pa)	Steel: 8.1e10

ELAS_DILA — Elastic with Thermal Dilation

```
elas='standard', young=3e10, nu=0.2, anisotropy='isotropic', dilatation=1.2e-5, T_ref_meca=20.0
```

Parameter	Description	Typical Value
elas	Elastic formulation.	'standard'
young	Young's modulus (Pa).	3e10
nu	Poisson's ratio.	0.2
anisotropy	Anisotropy.	'isotropic'
dilatation	Linear thermal expansion coefficient (K ⁻¹).	1e-5 to 2e-5
T_ref_meca	Mechanical reference temperature (°C or K) — zero thermal strain at this value.	20.0

VISCO_ELAS — Visco-Elastic

```
elas='standard', anisotropy='isotropic', young=1.17e11, nu=0.35,
viscous_model='KelvinVoigt', viscous_young=1.17e9, viscous_nu=0.35
```

Parameter	Description	Values
elas	Elastic formulation.	'standard'
young	Young's modulus of the elastic branch (Pa).	1.17e11
nu	Elastic Poisson's ratio.	0.35
anisotropy	Anisotropy.	'isotropic'
viscous_model	Rheological model. KelvinVoigt = spring and damper in parallel (reversible creep).	'KelvinVoigt' . 'none'
viscous_young	Young's modulus of the viscous branch (Pa).	1.17e9
viscous_nu	Poisson's ratio of the viscous branch.	0.35

ELAS_PLAS — Elasto-Plastic

```
elas='standard', anisotropy='isotropic', young=2.1e11, nu=0.3,
critere='Von-Mises', isoh='linear', iso_hard=2.5e8, isoh_coeff=1e9,
cinh='none', visc='none'
```

Parameter	Description	Values
elas	Elastic formulation.	'standard'
young	Young's modulus (Pa).	2.1e11
nu	Poisson's ratio.	0.3
anisotropy	Anisotropy.	'isotropic' . 'orthotropic'
critere	Plasticity criterion.	'Von-Mises' . 'none'
isoh	Type of isotropic hardening.	'none', 'linear'
iso_hard	Initial yield strength σ_y (Pa).	2.5e8
isoh_coeff	Isotropic hardening modulus H (Pa).	1e9
cinh	Kinematic hardening.	'none'
visc	Viscoplasticity.	'none'

THERMO_ELAS — Coupled Thermo-Elastic

```
elas='standard', young=3e10, nu=0.2, anisotropy='isotropic',
dilatation=1.2e-5, T_ref_meca=20.0, conductivity=1.8, specific_capacity=880.0
```

Parameter	Description	Typical Value
elas	Elastic formulation.	'standard'
young	Young's modulus (Pa).	3e10
nu	Poisson's ratio.	0.2
anisotropy	Anisotropy.	'isotropic'
dilatation	Thermal expansion coefficient (K^{-1}).	1.2e-5
T_ref_meca	Mechanical reference temperature.	20.0

Parameter	Description	Typical Value
conductivity	Thermal conductivity (W/m/K) or 'field'	
specific_capacity	Specific heat capacity (J/kg/K) or 'field'	

PORO_ELAS — Poro-Elastic (Biot)

```
elas='standard', young=5e7, nu=0.3, anisotropy='isotropic',
hydro_cpl=0.8, conductivity=1e-8, specific_capacity=1e-10
```

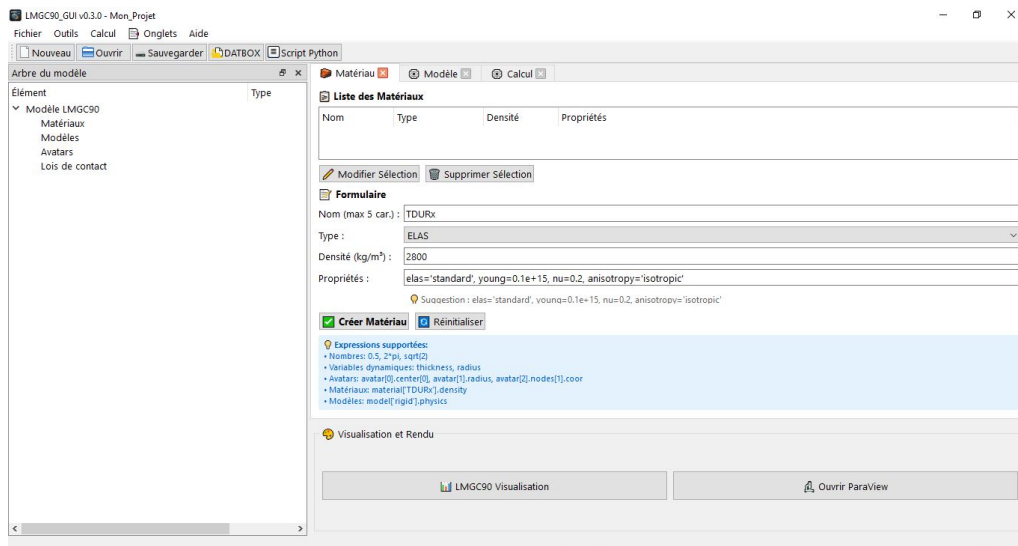
Parameter	Description	Typical Value
elas	Elastic formulation.	'standard'
young	Young's modulus of the solid skeleton (Pa).	5e7
nu	Poisson's ratio of the skeleton.	0.3
anisotropy	Anisotropy.	'isotropic'
hydro_cpl	Biot coupling coefficient (0 to 1).	0.8
conductivity	Hydraulic conductivity (m/s) or 'field'	
specific_capacity	Hydraulic storage capacity (Pa ⁻¹) or 'field'	

Creation Example — Elastic Steel

1. Open the **Material** tab (**Ctrl+1**).
2. Select the **ELAS** type from the dropdown list — the Properties field fills in automatically.
3. Modify the values in the Properties field:

```
elas='standard', young=2.1e11, nu=0.3, anisotropy='isotropic'
```

4. Set the density to **7850** kg/m³.
5. Enter a name: **steel** or **ACIEx**.
6. Click **Create Material**.

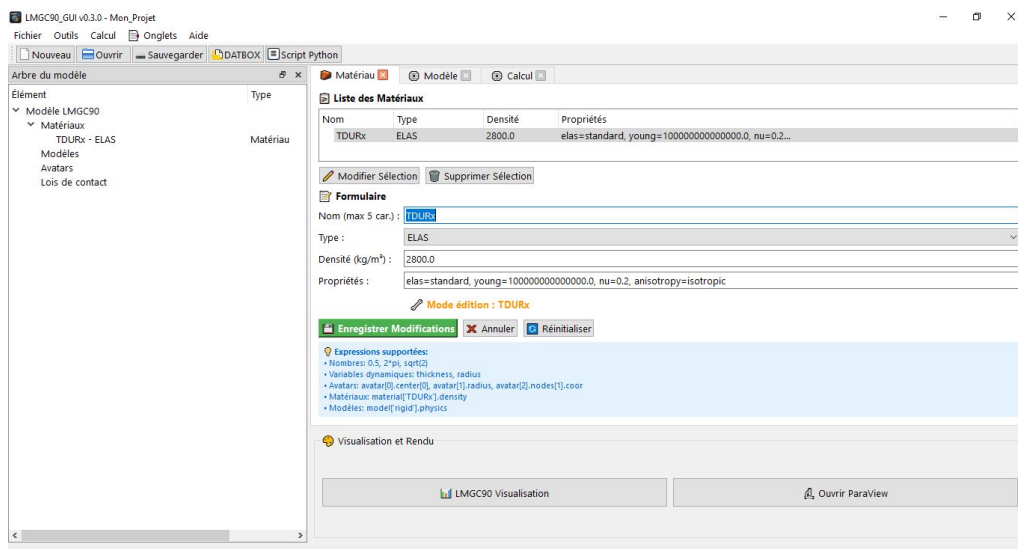


Creating a steel material

Editing and Deletion

In the materials list table, select the material to edit, then click the **Edit Selection** button. All the material's data is loaded into the form in **Edit** mode.

- Make your changes in the fields.
- Click **Save Changes** to confirm.
- Click **Cancel** to discard the changes and return to normal mode.



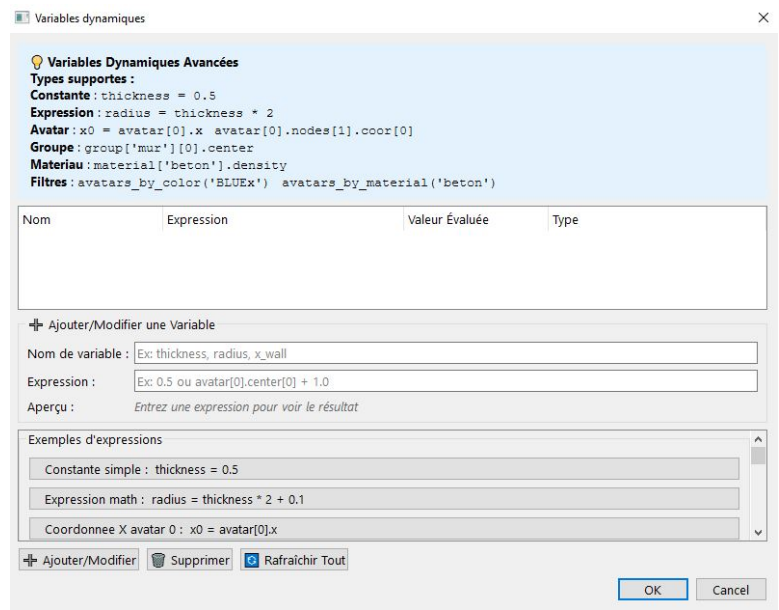
Editing a material

Deletion: a material used by at least one avatar cannot be deleted directly. A warning message indicates the affected avatars. You must first reassign these avatars to another material, or delete them.

Dynamic Variables

Dynamic variables allow you to define reusable values or expressions in all the numeric fields of the interface, including the material Properties field.

Open the dialog box via **Tools** → **Dynamic Variables** or the shortcut **Ctrl+V**.



Dynamic variables

How to Create a Variable

1. In the dialog box, click on one of the examples to load it as a base (optional).
2. Enter the variable's **name** (e.g.: **young_ref**).
3. Enter the **value or expression** (e.g.: **2.1e11**).
4. Click **Add or edit**.
5. Click **OK** to close.

The dynamic variables can now be used in any field of the application by simply typing its name. See Dynamic Variables.

Expression Examples

Entered Expression	Result	Usage
<code>young_ref = 2.1e11</code>	Numeric constant	<code>young=young_ref</code> in the properties
<code>young_beton = young_ref / 7</code>	Computed expression	Concrete/steel stiffness ratio
<code>nu_courant = 0.3</code>	Constant	Default Poisson's ratio
<code>radius = avatar[0].radius * 2</code>	Property of an existing avatar	Radius derived from another avatar

Note: expressions are evaluated via **SafeEvaluator**, which allows standard Python mathematical operations (+, -, <i>, /, </i>*, **math.sqrt(...)**, etc.) as well as access to the properties of the project's avatars and materials.

Tips

- **Name limited to 5 characters:** names in LMGC90 are limited to five characters. Prefer short codes such as **steel**, **BEton**, **GRAN1**.

-
- **RIGID material with no properties:** for rigid bodies, the Properties field must remain empty. Only the density is used to compute the mass and moment of inertia.
 - **'field' value:** for `conductivity` and `specific_capacity` in the `THERMO_ELAS` and `PORO_ELAS` types, the value `'field'` indicates that this parameter is defined by the finite element model rather than by a scalar constant.
 - **Material / model consistency:** the material type must be compatible with the physics of the model associated with the avatar. For example, a `PORO_ELAS` material must be associated with a model of `POROX` or `MULTI` physics.
 - **Dynamic variables:** use `Ctrl+V` to quickly access variables and avoid repetitive entry in the Properties fields.

Creating a Model

The model defines the **finite element solver** that will be used for the deformable bodies of the project. It associates a physics, a finite element type, a spatial dimension, and numerical options.

Interface

The **Model** tab (**Ctrl+2**) is divided into two parts:

- **List of models** (top): a table displaying all the models defined in the project with their name, physics, element, and dimension. Models used by at least one avatar are displayed in green.
- **Creation / editing form** (bottom): input fields whose Options section adapts automatically to the selected element.

Form Fields

Field	Description
Name	Unique identifier of the model. 5 characters (internal LMGC90 constraint). Examples: rigid , MECAX , elas2 , ther3 .
Physics	Family of physics being solved. Determines the list of available elements. See section Available Physics.
Dimension	Spatial dimension: 2 (2D) or 3 (3D). The list of elements updates automatically.
Element	Finite element type. The list depends on BOTH the selected physics and dimension.
Options	Numerical parameters automatically displayed depending on the element. Rigid elements (Rxx2D , Rxx3D) have no options.

Automatic update: each time the physics or dimension changes, the list of available elements is reloaded. If the previously selected element exists in the new list, it is kept.

Available Physics

All four physics types are implemented in LMGC90_GUI.

Code	Full Name	Description	Compatible Materials
MECAX	Solid mechanics	Deformation, stress, dynamics.	RIGID , ELAS , ELAS_DILA , VISCO_ELAS , ELAS_PLAS
THERX	Thermal	Heat diffusion, convection, radiation.	THERMO_ELAS
POROX	Poromechanics	Solid / fluid coupling following Biot's theory.	PORO_ELAS
MULTI	Thermo-hydro-mechanical (THM)	Full mechanical + thermal + hydraulic coupling.	PORO_ELAS , THERMO_ELAS

Available Elements by Physics

MECAx — 2D Mechanical Elements

Element	Geometry	Nodes	Order	Description
Rxx2D	Point	1	—	2D rigid body. No options.
T3xxx	3-node triangle	3	1	Standard linear triangle.
T3Lxx	3-node triangle	3	1	Enriched linear triangle (incompatible modes).
T6xxx	6-node triangle	6	2	Quadratic triangle.
DKTxx	3-node triangle	3	1	Discrete Kirchhoff triangle (thin plates).
Q4xxx	4-node quadrangle	4	1	Standard bilinear quadrangle.
Q4P0x	4-node quadrangle	4	1	Bilinear quadrangle + constant pressure (quasi-incompressible).
Q8xxx	8-node quadrangle	8	2	Serendipity quadrangle.
Q8Rxx	8-node quadrangle	8	2	Serendipity quadrangle with reduced integration.
Q9xxx	9-node quadrangle	9	2	Biquadratic Lagrange quadrangle.
BARxx	2-node segment	2	1	1D bar / truss.
SPRG2	2-node segment	2	1	2D spring (discrete=yes__ added automatically).

MECAx — 3D Mechanical Elements

Element	Geometry	Nodes	Order	Description
Rxx3D	Point	1	—	3D rigid body. No options.
TE4xx	4-node tetrahedron	4	1	Linear tetrahedron.
TE4Lx	4-node tetrahedron	4	1	Enriched linear tetrahedron (F-bar).
TE10x	10-node tetrahedron	10	2	Quadratic tetrahedron.
H8xxx	8-node hexahedron	8	1	Trilinear hexahedron.
H20xx	20-node hexahedron	20	2	Serendipity hexahedron.
H20Rx	20-node hexahedron	20	2	Serendipity hexahedron with reduced integration.
PRI6x	6-node prism	6	1	Linear prism.
SHB6x	6-node prism	6	1	SHB6 solid-shell prism.
PRI15	15-node prism	15	2	Quadratic prism.
BARxx	2-node segment	2	1	1D bar / truss.

Element	Geometry	Nodes	Order	Description
SPRG3	2-node segment	2	1	3D spring (discrete=yes__ added automatically).

THERx — 2D Thermal Elements

Element	Nodes	Order	Specific Thermal Options
Rxx2D	1	—	none
T3xxx	3	1	mass_storage, convection, radiation
T6xxx	6	2	mass_storage, convection, radiation
DKTxx	3	1	mass_storage, convection, radiation
Q4xxx	4	1	mass_storage, convection, radiation
Q4P0x	4	1	mass_storage, convection, radiation
Q8xxx	8	2	mass_storage, convection, radiation
Q8Rxx	8	2	mass_storage, convection, radiation
SPRG2	2	1	mass_storage only
S2xth	2	1	mass_storage only — 1D thermal segment

THERx — 3D Thermal Elements

Element	Nodes	Order	Specific Thermal Options
Rxx3D	1	—	none
TE4xx	4	1	mass_storage, convection, radiation
TE10x	10	2	mass_storage, convection, radiation
H8xxx	8	1	mass_storage, convection, radiation
H20xx	20	2	mass_storage, convection, radiation
H20Rx	20	2	mass_storage, convection, radiation
PRI6x	6	1	mass_storage, convection, radiation
PRI15	15	2	mass_storage, convection, radiation
SPRG3	2	1	mass_storage only

POROX — Poromechanical Elements (mixed displacement-pressure elements)

Element	Dim.	Geometry	Nodes	Order	Description
T33xx	2D	Triangle	3	1	Mixed P1/P1 triangle.
T63xx	2D	Triangle	6	2	Mixed P2/P1 triangle — satisfies the LBB condition.
Q44xx	2D	Quadrangle	4	1	Mixed Q1/Q1 quadrangle.
Q84xx	2D	Quadrangle	8	2	Mixed Q2/Q1 quadrangle — satisfies the LBB condition.
TE44x	3D	Tetrahedron	4	1	Mixed P1/P1 tetrahedron.
TE104	3D	Tetrahedron	10	2	Mixed P2/P1 tetrahedron — satisfies the LBB condition.
H88xx	3D	Hexahedron	8	1	Mixed Q1/Q1 hexahedron.
H208x	3D	Hexahedron	20	2	Mixed Q2/Q1 hexahedron — satisfies the LBB condition.

MULTI — 2D and 3D THM Elements

Same mixed elements as POROx, plus H8xxx in 3D for uniform interpolation of the three coupled fields.

Element	Dim.	Description
T33xx	2D	Mixed P1/P1 triangle.
T63xx	2D	Mixed P2/P1 triangle.
Q44xx	2D	Mixed Q1/Q1 quadrangle.
Q84xx	2D	Mixed Q2/Q1 quadrangle.
TE44x	3D	Mixed P1/P1 tetrahedron.
TE104	3D	Mixed P2/P1 tetrahedron.
H8xxx	3D	Trilinear hexahedron (uniform THM interpolation).
H88xx	3D	Mixed Q1/Q1 hexahedron.
H208x	3D	Mixed Q2/Q1 hexahedron.

Summary Table by Use Case

Use Case	Dimension	Recommended Elements	Note
2D rigid bodies (DEM)	2D	Rxx2D	No options
3D rigid bodies (DEM)	3D	Rxx3D	No options
2D springs / bars	2D	SPRG2, BARxx	discrete=yes__ auto
3D springs / bars	3D	SPRG3, BARxx	discrete=yes__ auto

Use Case	Dimension	Recommended Elements	Note
2D structures — standard accuracy	2D	Q4xxx, T3xxx	Fast, suited to structured meshes
2D structures — complex geometry	2D	T6xxx, Q8xxx	Unstructured automatic meshes
2D thin plates	2D	DKTxx	Kirchhoff formulation
2D quasi-incompressibility	2D	Q4P0x	Constant pressure per element
3D structures — structured mesh	3D	H8xxx, H20xx	Hexahedra recommended
3D structures — automatic mesh	3D	TE10x, TE4xx	Adaptive tetrahedra
3D thick shells	3D	SHB6x, PRI6x	Solid-shells
2D thermal	2D	Q4xxx, T3xxx	THERx physics
3D thermal	3D	H8xxx, TE4xx	THERx physics
1D thermal segment	2D / 3D	S2xt th	THERx physics only
2D poromechanics (Biot)	2D	T63xx, Q84xx	LBB satisfied — recommended
3D poromechanics (Biot)	3D	TE104, H208x	LBB satisfied — recommended
3D coupled THM	3D	H8xxx, TE104	MULTI physics

Model Options

MECAx Options — Element-Specific

These three options are displayed for all non-rigid MEXA elements.

kinematic — Kinematic Assumption

Value	Description	Use Case
small	Small strains (HPP). Linear displacement-strain relationship. Reference geometry assumed constant.	Concrete, elastic steel, most civil engineering structures.
large	Large strains. Geometrically nonlinear relationship. Configuration updated at each step.	Rubber, soft materials, forming, impact, crash.

formulation — Lagrangian Formulation

Relevant only for kinematic=large.

Value	Description
UpdtL	Updated Lagrangian. The reference configuration is updated at each step. Cauchy stresses. Recommended for continuous large strains (metals).
Total	Total Lagrangian. The reference configuration remains the initial configuration. Piola-Kirchhoff stresses. Recommended for reversible large strains (elastomers).

mass_storage — Mass Matrix Storage

Value	Description	Use Case
<code>lump_</code>	Lumped mass. Diagonal matrix. Immediate inversion. Less accurate at high frequencies.	Explicit dynamics, small time-step schemes.
<code>coher</code>	Coherent mass. Full integrated matrix. More accurate. More costly to invert.	Implicit dynamics, modal analysis, high frequencies.

MECAx Options — Common to All Elements (except rigid)

These three options are displayed in addition to the element-specific options.

material — Local Constitutive Law

Value	Behavior	Associated Materials
<code>elas_</code>	Standard linear elasticity	<code>ELAS</code> , <code>ELAS_DILA</code> , <code>RIGID</code>
<code>elasd</code>	Damageable elasticity	<code>ELAS_PLAS</code> (with damage variable)
<code>J2iso</code>	Isotropic J2 plasticity	<code>ELAS_PLAS</code> (<code>isoh</code> ='linear' or 'nonlinear')
<code>J2mix</code>	Mixed J2 plasticity (isotropic + kinematic)	<code>ELAS_PLAS</code> (<code>isoh</code> and <code>cinh</code> both active)
<code>kvisc</code>	Kelvin-Voigt visco-elasticity	<code>VISCO_ELAS</code>

anisotropy — Anisotropy

Value	Description
<code>iso__</code>	Isotropic. Identical properties in all directions. 2 parameters: E (Young), ν (Poisson).
<code>ortho</code>	Orthotropic. Different properties along 3 principal directions. Requires the 9 elastic constants (E_x , E_y , E_z , ν_{xy} , ν_{yz} , ν_{xz} , G_{xy} , G_{yz} , G_{xz}). Suited to composites, wood, laminated materials.

external_model — External Model

Value	Description
<code>MatL_</code>	Internal LMGC90 law (standard behavior). Default value.
<code>Demfi</code>	DemFi interface — coupling with an external DEM model.
<code>Umat_</code>	UMAT interface — ABAQUS-type user routine (Fortran / C).
<code>no__</code>	Disabled.
<code>yes__</code>	Enabled (depending on the option's context).

THERx Options

These options are displayed for all non-rigid thermal elements.

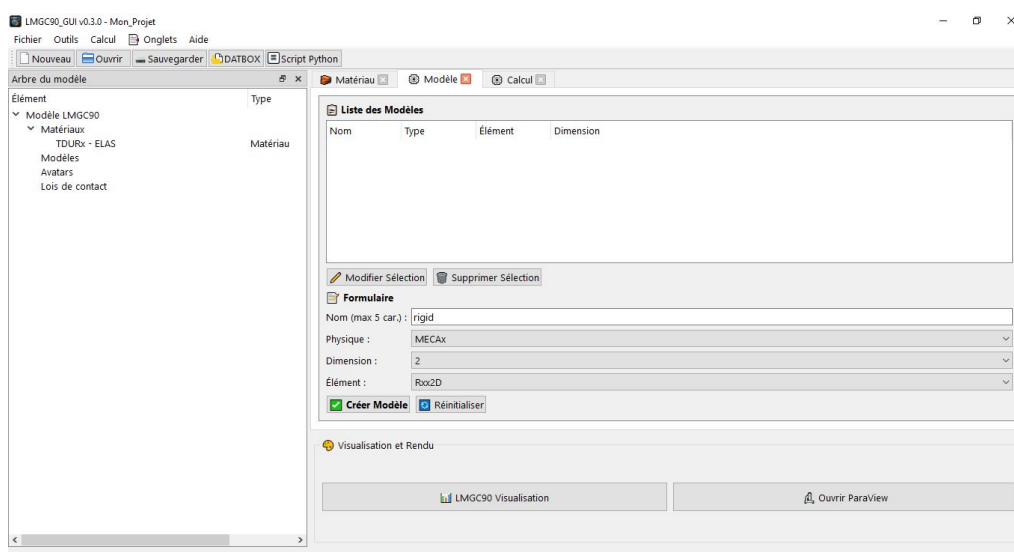
Option	Values	Description
<code>mass_storage</code>	<code>lump_</code> · <code>coher</code>	Storage of the thermal capacity matrix. Same meaning as in MECAX.

Option	Values	Description
convection	no__ · yes__	Enables surface thermal convection terms.
radiation	no__ · yes__	Enables surface thermal radiation terms (Stefan-Boltzmann law).
anisotropy	iso__ · ortho	Anisotropy of thermal conductivity.
external_model	MatL_ · Demfi · Umat_ · no__ · yes__	Interface with an external thermal model.

Note: SPRG2, SPRG3, and S2xth only have the mass_storage option. The convection and radiation options are not displayed for these 1D elements.

Creation Example — 2D Rigid Body

1. Open the **Model** tab (**Ctrl+2**).
2. Enter the name: **rigid**.
3. Select the physics: **MECAx**.
4. Select the dimension: **2**.
5. Select the element: **Rxx2D** — the Options section disappears (no options for rigid elements).
6. Click **■ Create Model**.



Creating a 2D rigid model

Creation Example — 2D Elastic Mechanics

1. Name: **elas2**.
2. Physics: **MECAx** · Dimension: **2** · Element: **Q4xxx**.
3. Options displayed automatically:
 - **kinematic** → **small**
 - **formulation** → **UpdtL**
 - **mass_storage** → **lump_**
 - **material** → **elas_**
 - **anisotropy** → **iso__**

• `external_model` → `MatL_`

4. Click **Create Model**.

Editing and Deletion

Select a model in the list, then click **Edit Selection** to load it in **Edit** mode. Make your changes, then click **Save Changes**, or **Cancel** to discard the changes.

Deletion: a model used by at least one avatar cannot be deleted directly. The warning dialog box lists the affected avatars.

Material / Physics Compatibility Table

Material	MECAx	THERx	POROX	MULTI
RIGID	■	■	■	■
ELAS	■	■	■	■
ELAS_DILA	■	■	■	■
VISCO_ELAS	■	■	■	■
ELAS_PLAS	■	■	■	■
THERMO_ELAS	■	■	■	■
PORO_ELAS	■	■	■	■
DISCRETE	■	■	■	■
USER_MAT	■	■	■	■
EXTERNAL	■	■	■	■

Material / `material` Option Compatibility Table

Material	<code>elas_</code>	<code>elasd</code>	<code>J2iso</code>	<code>J2mix</code>	<code>kvisc</code>
RIGID	■	—	—	—	—
ELAS	■	—	—	—	—
ELAS_DILA	■	■	—	—	—
VISCO_ELAS	—	—	—	—	■
ELAS_PLAS	—	■	■	■	—
THERMO_ELAS	■	■	—	—	—

Tips

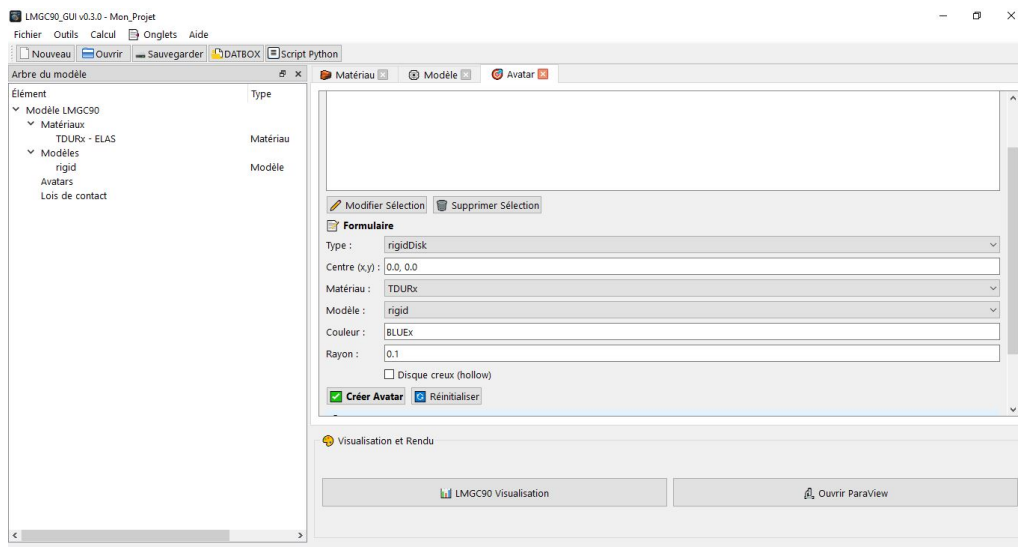
- **Name limited to 5 characters:** LMGC90 silently ignores additional characters. Prefer `rigid`, `elas2`, `ther3`, `poro2`.
- **Rxx element:** rigid bodies exclusively use `Rxx2D` (2D) or `Rxx3D` (3D). These elements have no numerical options.
- **Springs:** for `SPRG2` and `SPRG3`, the option `discrete=yes__` is added automatically at creation — there is no need to enter it manually.
- **LBB in poromechanics:** for consolidation or pressure diffusion computations, prefer higher-order elements (`T63xx`, `Q84xx`, `TE104`, `H208x`) which satisfy the Ladyzhenskaya-Babuška-Brezzi condition and avoid spurious pressure

oscillations.

- **kinematic=large with formulation**: the **formulation** field only has an effect if **kinematic=large**. In small strains, the value of **formulation** is ignored by the solver.
- **Unused models**: models not associated with an avatar appear in black in the list. They have no effect on the computation.

Creating an Avatar (Simple Rigid Body)

Avatar tab Allows you to create, modify, and delete the rigid bodies of the project: disks, spheres, joncs, polygons, walls, cylinders, polyhedra, and more. Each avatar is defined by a **type**, a **center**, a **material**, a **model**, and **geometric parameters** specific to its type.



captures/avatar_disque.JPG

General Interface

The tab is divided into two areas:

- **Avatar list** (top): a tree displaying all the avatars in the project with their index, type, color, and center. Double-click on a row to edit it. Right-click to access the context menu (Edit, Delete, Information).
- **Creation / editing form** (bottom): fields adapted to the selected avatar type.

List of Avatars by Dimension

2D Avatars

pylmgc90 Type	Short Description	Key Parameters
<code>rigidDisk</code>	Rigid disk	<code>r</code> , <code>is_hollow</code>
<code>rigidJonc</code>	Rigid ellipse	<code>axe1</code> , <code>axe2</code>
<code>rigidPolygon</code>	Rigid polygon	<code>generation_type</code> , <code>nb_vertices</code> , <code>radius</code> or <code>vertices</code>
<code>rigidOvoidPolygon</code>	Rigid ovoid	<code>ra</code> , <code>rb</code> , <code>nb_vertices</code>
<code>rigidDiscreteDisk</code>	Discrete disk	<code>r</code>

pylmgc90 Type	Short Description	Key Parameters
<code>rigidCluster</code>	Disk cluster	<code>r</code> , <code>nb_disk</code>
<code>roughWall</code>	Rough wall	<code>l</code> , <code>r</code> , <code>nb_vertex</code>
<code>fineWall</code>	Fine wall	<code>l</code> , <code>r</code> , <code>nb_vertex</code>
<code>smoothWall</code>	Smooth wall	<code>l</code> , <code>h</code> , <code>nb_polyg</code>
<code>granuloRoughWall</code>	Granular wall	<code>l</code> , <code>rmin</code> , <code>rmax</code> , <code>nb_vertex</code>

3D Avatars

pylmgc90 Type	Short Description	Key Parameters
<code>rigidSphere</code>	Rigid sphere	<code>r</code>
<code>rigidPlan</code>	Rigid plane	<code>axe1</code> , <code>axe2</code> , <code>axe3</code>
<code>rigidCylinder</code>	Rigid cylinder	<code>r</code> , <code>h</code>
<code>rigidPolyhedron</code>	Rigid polyhedron	<code>generation_type</code> , <code>nb_vertices</code> , <code>radius</code> or <code>vertices + faces</code>
<code>roughWall3D</code>	3D rough wall	<code>lx</code> , <code>ly</code> , <code>r</code>
<code>granuloRoughWall3D</code>	3D granular wall	<code>lx</code> , <code>ly</code> , <code>rmin</code> , <code>rmax</code>

Fields Common to All Types

Field	Description
Type	pylmgc90 avatar type. Determines the additional fields displayed.
Center	Coordinates of the reference center. Format <code>x</code> , <code>y</code> in 2D or <code>x</code> , <code>y</code> , <code>z</code> in 3D. Accepts Python expressions (<code>avatar[0].x + 0.5</code>).
Material	Selection from the materials defined in the Materials tab.
Model	Selection from the models defined in the Models tab.
Color	LMGC90 display color in 5 characters. See the color list below.

Available LMGC90 Colors

Examples of colors you could use:

Code	Color
<code>BLUEX</code>	Blue
<code>REDxx</code>	Red
<code>VERTx</code>	Green
<code>JAUNx</code>	Yellow
<code>GRAYx</code>	Gray
<code>BLACx</code>	Black
<code>WHITx</code>	White
<code>ORANx</code>	Orange
<code>CYANx</code>	Cyan
<code>MAGEx</code>	Magenta

Code	Color
VIOLx	Purple
ROSEx	Pink

2D Avatar Types

1. rigidDisk — 2D Rigid Disk

2D rigid circular body. This is the most common type for granular simulations.

Parameter	Description	Example
r (radius)	Disk radius (m).	0.1
is_hollow	If checked, creates a hollow disk (<code>is_Hollow=True</code>). Useful for rigid rings.	checkbox

rigidJonc — 2D Rigid Jonc / Ellipse

2D rigid elliptical body. Defined by two half-axes.

Parameter	Description	Example
axe1	Main half-axis (m) — length.	0.15
axe2	Secondary half-axis (m) — width.	0.05

rigidPolygon — 2D Rigid Polygon

2D rigid polygonal body. Three generation modes available depending on `generation_type`.

Parameter	Description	Values
generation_type	Shape generation mode.	regular · full · bevel
nb_vertices	Number of vertices (for <code>regular</code> and <code>full</code>).	3 to 20
radius	Radius of the circumscribed circle (m) — used for <code>regular</code> . Not used for <code>full</code> and <code>bevel</code> .	0.1
vertices	Explicit list of vertices <code>[[x1,y1],[x2,y2],...]</code> — used for <code>full</code> and <code>bevel</code> .	<code>[[-0.1, -0.1], [0.1, -0.1], [0., 0.1]]</code>

Generation modes:

- `regular`: regular polygon (all sides equal). Defined by `nb_vertices` and `radius` (radius of the circumscribed circle).
- `full`: arbitrary polygon from an explicit list of vertices. The radius is not used.
- `bevel`: polygon with automatic beveling of corners to avoid contact singularities.

rigidOvoidPolygon — 2D Rigid Ovoid

2D rigid ovoid body (polygonal ellipse). Polygonal approximation of an ellipse.

Parameter	Description	Example
<code>ra</code>	Main half-axis (m).	<code>0.15</code>
<code>rb</code>	Secondary half-axis (m).	<code>0.08</code>
<code>nb_vertices</code>	Number of vertices of the polygonal approximation.	<code>20</code>

rigidDiscreteDisk — 2D Rigid Discrete Disk

2D rigid circular body with discrete kinematics. Same geometry as a `rigidDisk`, but with a `xKSID`-type contactor (discrete disk). Used in some advanced discrete models.

Parameter	Description	Example
<code>r</code> (radius)	Disk radius (m).	<code>0.1</code>

rigidCluster — 2D Rigid Disk Cluster

2D rigid body composed of several disks rigidly linked together. Allows the creation of complex non-convex shapes.

Parameter	Description	Example
<code>r</code>	Radius of each disk in the cluster (m).	<code>0.05</code>
<code>nb_vertices</code> (nb_disk)	Number of disks in the cluster.	<code>4</code>

The parameter is named `nb_disk` in the generated `pylmgc90` script (not `nb_vertices`).

roughWall — 2D Rough Wall

2D rough wall made up of aligned disks. Used for confining walls with geometric roughness.

Parameter	Description	Example
<code>l</code>	Total length of the wall (m).	<code>2.0</code>
<code>r</code>	Radius of the disks forming the roughness (m).	<code>0.05</code>
<code>nb_vertex</code>	Number of disks along the wall. Default: <code>10</code> .	<code>20</code>

fineWall — 2D Fine Wall

2D fine wall made up of very small disks. Same parameters as `roughWall` but with finer roughness.

Parameter	Description	Example
<code>l</code>	Total length of the wall (m).	<code>2.0</code>
<code>r</code>	Radius of the disks (m). Typically very small (0.001 to 0.01).	<code>0.005</code>
<code>nb_vertex</code>	Number of disks. Default: <code>10</code> .	<code>50</code>

smoothWall — 2D Smooth Wall

2D smooth wall defined by a half-length and a height. **CLxxx** contactor (continuous surface).

Parameter	Description	Example
l	Half-length of the wall (m) — the total length is $2 \times l$.	1.0
h	Half-height (thickness) of the wall (m).	0.01
nb_polyg	Number of polygonal segments. Default: 10 .	20

granuloRoughWall — 2D Granular Rough Wall

2D wall with random roughness generated by a granulometric distribution of disks.

Parameter	Description	Example
l	Total length of the wall (m).	2.0
rmin	Minimum radius of the disks (m).	0.01
rmax	Maximum radius of the disks (m).	0.05
nb_vertex	Number of disks. Default: 10 .	30

3D Avatar Types

rigidSphere — 3D Rigid Sphere

3D rigid spherical body. 3D equivalent of **rigidDisk**.

Parameter	Description	Example
r (radius)	Sphere radius (m).	0.1

rigidPlan — 3D Rigid Plane

3D rigid flat surface. Defined by three direction vectors forming a local frame.

Parameter	Description	Example
axe1	Vector of the plane's first axis (local X direction).	[1.0, 0.0, 0.0]
axe2	Vector of the plane's second axis (local Y direction).	[0.0, 1.0, 0.0]
axe3	Normal to the plane (local Z direction).	[0.0, 0.0, 1.0]

The three vectors must form a direct orthonormal basis.

rigidCylinder — 3D Rigid Cylinder

3D rigid cylindrical body.

Parameter	Description	Example
<code>r</code> (radius)	Cylinder radius (m).	<code>0.1</code>
<code>h</code>	Height (axial length) of the cylinder (m). Default: <code>1.0</code> if absent.	<code>0.5</code>

rigidPolyhedron — 3D Rigid Polyhedron

3D rigid polyhedral body. Two generation modes available depending on `generation_type`.

Parameter	Description	Values
<code>generation_type</code>	Generation mode.	<code>regular</code> · <code>vertices</code>
<code>nb_vertices</code>	Number of vertices (for <code>regular</code>).	<code>8</code> (cube), <code>12</code> (icosahedron)...
<code>radius</code>	Radius of the regular polyhedron (m) — for <code>regular</code> .	<code>0.1</code>
<code>vertices</code>	Explicit list of 3D vertices <code>[[x,y,z],...]</code> — for <code>vertices</code> .	<code>[[-1,-1,-1], [1,-1,-1], ...]</code>
<code>faces</code>	Face connectivity <code>[[i,j,k],...]</code> (in <code>wall_params</code>) — for <code>vertices</code> .	<code>[[0,1,2], [2,3,0], ...]</code>

Generation modes:

- `regular`: regular polyhedron (similar to a polygonal sphere). Defined by `nb_vertices` and `radius`.
- `vertices`: arbitrary polyhedron defined by an explicit list of vertices and face connectivity.

roughWall3D — 3D Rough Wall

3D rough wall made up of spheres aligned on a rectangular surface.

Parameter	Description	Example
<code>lx</code>	Wall dimension in X (m).	<code>2.0</code>
<code>ly</code>	Wall dimension in Y (m).	<code>2.0</code>
<code>r</code>	Radius of the roughness spheres (m).	<code>0.05</code>

granuloRoughWall3D — 3D Granular Rough Wall

3D rough wall with randomly sized spheres.

Parameter	Description	Example
<code>lx</code>	Dimension in X (m).	<code>2.0</code>
<code>ly</code>	Dimension in Y (m).	<code>2.0</code>
<code>rmin</code>	Minimum radius of the spheres (m).	<code>0.01</code>
<code>rmax</code>	Maximum radius of the spheres (m).	<code>0.05</code>

General Notes

Python expressions in numeric fields: all numeric fields (center, radius, dimensions) accept Python expressions evaluated via `SafeEvaluator`. Examples: `avatar[0].radius <i> 2, thickness + 0.1, math.sqrt(2) </i> r_base`.

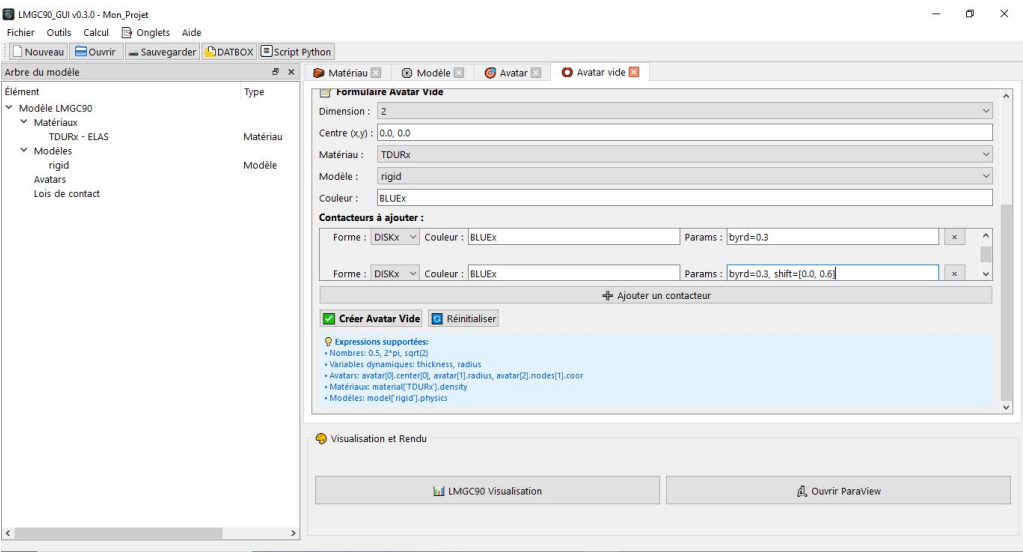
RIGID material: rigid avatars must use a material of type `RIGID`. An elastic material can technically be assigned, but has no mechanical effect on a rigid body.

Rxx model: rigid avatars use a model with the `Rxx2D` (2D) or `Rxx3D` (3D) element. These elements have no numerical options.

Unused avatars: avatars whose material or model has been deleted remain in the list but generate an error when the script is created. Validation is performed before generation.

Empty Avatar (Advanced Customization)

Empty Avatar tab Allows you to create **avatars with custom contactors**: either an empty body (**emptyAvatar**) fully defined by its contactors, or the addition of contactors to an **existing deformable body** (FEM mesh). This is the most flexible tool in LMGC90_GUI for advanced cases that do not fit into the standard rigid avatar types.



captures/avatar_vide_2disques.JPG

Two Operating Modes

The **Mode** field at the top of the form switches between two distinct behaviors:

Mode	Description
Empty avatar (emptyAvatar)	Creates a new pylmgc90 body fully assembled from its contactors. The body is rigid, but its geometry is defined manually.
Existing deformable body	Adds contact contactors to a deformable body (FEM mesh) already created via the mesh wizard. Calls <code>body.addContactors()</code> directly on the pylmgc90 object.

Mode 1 — Empty Avatar (emptyAvatar)

Principe

An empty avatar is a rigid body whose geometry is not predefined. pylmgc90 assembles it in three steps:

1. Creation of an empty body `pre.avatar(dimension=...)`
2. Addition of a rigid bulk `pre.rigid2d()` or `pre.rigid3d()`

3. Addition of a main node and the **contactors** that define the shape

The inertial properties (mass, moment of inertia) are computed automatically from the contactors' geometry via `body.computeRigidProperties()`.

Form Fields

Field	Description
Dimension	2 or 3. Determines the list of available contactor shapes and the format of the center.
Center (x,y) or (x,y,z)	Coordinates of the reference main node. Accepts Python expressions.
Material	RIGID-type material assigned to the body.
Model	Model with the Rxx2D (2D) or Rxx3D (3D) element.
Color	Color for LMGC90 interaction (5 characters); not very important.

Mode 2 — Existing Deformable Body

Principle

This mode does not create a new avatar — it enriches a deformable body (FEM mesh) already present in the project by adding surface contactors to it. This is necessary so that the deformable body can interact with other bodies (rigid or deformable).

The function called on the `pylmgc90` object is: `body.addContactors(shape=..., color=..., **params)`.

Fields Specific to This Mode

Field	Description
Deformable body	Dropdown list of MESH_DEFORMABLE-type bodies present in the project. The displayed format is #index – geometry (material/model).
Group (group=)	group parameter passed to <code>addContactors()</code> . Determines which node group of the mesh the contactor is applied to (e.g.: 102 for group index 102). Leave empty to apply to all nodes.

Important: the deformable body must have been created and rebuilt in memory (present in `_pylmgc_bodies`) before adding contactors. If the project was just loaded from a JSON file without rebuilding, the `pylmgc90` body does not yet exist in memory and adding contactors will fail.

Managing Contactors

Each row in the **Contactors to add** list corresponds to a `body.addContactors()` call. Click **Add a contactor** to create a new row. Click **x** to delete a row.

Columns of a Contactor Row

Column	Description
Shape	pylmgc90 contactor type. The list adapts to the mode (empty avatar or deformable) and the dimension.
Color	Contactor color (5-character LMGC90 code). Independent of the body's color, but very important for contact detection.
Params	Geometric parameters of the contactor in the format key=value, key=value . Automatically filled with a suggestion based on the chosen shape.

Available Contactor Shapes

2D Empty Avatar — shapes_2d

Shape	Description	Parameters	Default Suggestion
DISKx	2D disk	byrd = radius	byrd=0.3
xKSID	2D discrete disk	byrd = radius	byrd=0.3
JONCx	2D jonc / ellipse	axe1 = long half-axis, axe2 = short half-axis	axe1=1.0, axe2=0.1
POLYG	2D polygon	nb_vertices = number of vertices, vertices = list [[x,y],...]	nb_vertices=4, vertices=[[-1.,-1.],[1.,-1.],[1.,1.],[-1.,1.]]
PT2Dx	2D point node (FEM)	none	(empty)

3D Empty Avatar — shapes_3d

Shape	Description	Parameters	Default Suggestion
SPHER	3D sphere	byrd = radius	byrd=0.3
PLANx	3D plane	axe1, axe2, axe3 = axis dimensions	axe1=1.0, axe2=1.0, axe3=0.1
CYLND	3D cylinder	byrd = radius, High = height	byrd=0.5, High=1.0
DNLYC	3D hollow cylinder	byrd = radius, High = height	byrd=0.5, High=1.0
POLYR	3D polyhedron	nb_vertices = number of vertices, vertices = list [[x,y,z],...]	nb_vertices=8, vertices=[[-1,-1,-1],[1,-1,-1],...]
PT3Dx	3D point node (FEM)	none	(empty)

2D Deformable Body — mesh_shapes_2d

These shapes are intended to be added to a 2D FEM mesh. They define surface contactors for rigid-deformable interactions.

Shape	Description	Usage
ALpxx	Line contactor for 2D FEM masonry	ALpMECAx interactions (CLALp / MECAX)
CLxx	2D continuous line contactor	DKMECAx interactions (disk / MECAX)
DISKL	Disk on a 2D FEM node	Disk-disk interaction on a mesh
PT2TL	2D transmission point	FEM node-to-node coupling

3D Deformable Body — mesh_shapes_3d

Shape	Description	Usage
ASpxx	Surface contactor for 3D FEM spheres	SPMECAx interactions (sphere / 3D MECAx)
CSpxx	3D continuous surface contactor	Generic 3D rigid-deformable interactions
PT3Dx	3D FEM point node	3D FEM node-to-node coupling

Parameter Details by Shape

DISKx / xKSID / SPHER — Disk, Discrete Disk, Sphere

```
byrd=0.3
```

Parameter	Description
byrd	Radius of the contactor (m). Corresponds to the contact radius used in the detectors.

JONCx — 2D Jonc / Ellipse

```
axe1=1.0, axe2=0.1
```

Parameter	Description
axe1	Main half-axis (m) — long axis of the ellipse.
axe2	Secondary half-axis (m) — short axis of the ellipse.

POLYG — 2D Polygon

```
nb_vertices=4, vertices=[[-1., -1.], [1., -1.], [1., 1.], [-1., 1.]]
```

Parameter	Description
nb_vertices	Number of vertices of the polygon.
vertices	List of local vertex coordinates $[[x1, y1], [x2, y2], \dots]$. The coordinates are relative to the center of the body. The vertices must be in trigonometric (counter-clockwise) order.

PLANx — 3D Plane

```
axe1=1.0, axe2=1.0, axe3=0.1
```

Parameter	Description
axe1	Dimension along the first axis of the plane (m).
axe2	Dimension along the second axis of the plane (m).
axe3	Thickness of the plane (m) — used for computing the inertial properties.

CYLND / DNLYC — 3D Cylinder

```
byrd=0.5, High=1.0
```

Parameter	Description
byrd	Radius of the cylinder (m).
High	Height (axial length) of the cylinder (m). Note the capital letter.

POLYR — 3D Polyhedron

```
nb_vertices=8, vertices=[[-1., -1., -1.], [1., -1., -1.], [1., 1., -1.], [-1., 1., -1.],  
[-1., -1., 1.], [1., -1., 1.], [1., 1., 1.], [-1., 1., 1.]]
```

Parameter	Description
nb_vertices	Number of vertices of the polyhedron.
vertices	List of 3D coordinates for each vertex <code>[[x, y, z], ...]</code> . Local coordinates relative to the center.

For a convex polyhedron, the vertices can be provided in any order — `pylmgc90` computes the convex hull. For a non-convex polyhedron, the order of the faces must be consistent.

PT2Dx / PT3Dx — Point Nodes

No parameters. These contactors represent a contact point at a node.

```
(Params field empty)
```

Complete Examples

2D Empty Avatar — 6-Vertex Polygonal Body

```
Mode : Empty avatar (emptyAvatar)  
Dimension : 2  
Center : 0.0, 0.5  
Material : BRIQx  
Model : rigid  
Color : REDxx  
  
Contactor 1 :  
Shape : POLYG  
Color : REDxx  
Params : nb_vertices=6, vertices=[[-0.1, -0.05], [0.1, -0.05], [0.15, 0.0],  
[0.1, 0.05], [-0.1, 0.05], [-0.15, 0.0]]
```

3D Empty Avatar — Body with a Sphere and a Cylinder

```
Mode : Empty avatar (emptyAvatar)
Dimension : 3
Center : 0.0, 0.0, 0.5
Material : ACIER
Model : rig3D
Color : CYANx

Contactor 1 :
Shape : SPHER
Color : CYANx
Params : byrd=0.2

Contactor 2 :
Shape : CYLND
Color : GRAYx
Params : byrd=0.05, High=0.8
```

Adding Contactors to a Deformable Body

```
Mode : Existing deformable body
Deformable body : #3 – Rectangle (beton/MECAx)
Group (group=) : 102

Contactor 1 :
Shape : CLxx
Color : BLUEx
Params : (empty)
```

Interface — List of Empty Avatars

The list at the top of the tab displays only avatars of type **EMPTY_AVATAR**. The columns are:

Column	Description
#	Index of the avatar in the project's overall avatar list.
Color	LMGC90 color code of the body.
Center	Coordinates of the reference center, rounded to 2 decimal places.
Contactors	Number of contactors defined on this body.

Context menu (right-click):

-  **Edit** — loads the avatar into the form for editing.
-  **Delete** — deletes the avatar after confirmation. Refused if the avatar is referenced by a loop or a group.
-  **Information** — displays a dialog box with the details of all contactors.

Important Notes

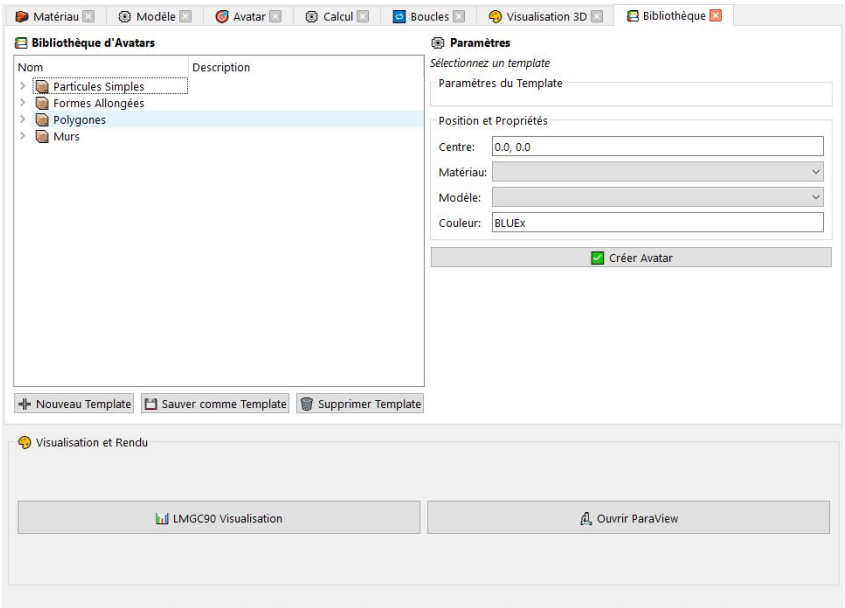
An empty avatar requires at least one contactor. Creation is refused if the contactor list is empty.

Multiple contactors. An empty avatar can have as many contactors as needed, of different shapes. Each contactor generates a distinct `body.addContactors(...)` line in the script.

Library — Preconfigured Avatars

The **Library** tab (📖) provides a catalog of preconfigured avatars, organized by category. It allows you to quickly insert common shapes into the project without having to manually enter the geometric parameters.

This tab **refreshes automatically** every time the dimension changes in the Model tab: the available templates adapt to the project's current dimension (2D or 3D).



General view of the Library tab

Interface Organization

The tab is divided into two panels:

Panel	Description
Left — Template tree	Tree structure of available avatars, organized by category. Click on an entry to display its properties on the right.
Right — Properties	Displays the name, avatar type, description, and default parameters of the selected template. Also allows you to configure the center, material, model, and color before insertion.

Available Templates

Templates are organized by category. The list changes depending on whether the project's dimension is **2D** or **3D**.

2D Templates

Simple Particles

Template	pylmgc90 Type	Default Parameters	Description
Small Disk	<code>rigidDisk</code>	<code>radius = 0.05 m</code>	Rigid disk with a small radius, typical of fine granular simulations.
Medium Disk	<code>rigidDisk</code>	<code>radius = 0.10 m</code>	Rigid disk with an intermediate radius.
Large Disk	<code>rigidDisk</code>	<code>radius = 0.20 m</code>	Rigid disk with a large radius, for sizable particles.

Elongated Shapes

Template	pylmgc90 Type	Default Parameters	Description
Horizontal Cylinder	<code>rigidJonc</code>	<code>axe1 = 2.0 m, axe2 = 0.1 m</code>	Elliptical jonc elongated horizontally, aspect ratio 2:1.
Vertical Cylinder	<code>rigidJonc</code>	<code>axe1 = 2.0 m, axe2 = 0.1 m</code>	Elliptical jonc elongated vertically.

`axe1` is the long half-axis, `axe2` the short half-axis (same parameters as `rigidJonc` in the Avatar tab).

Regular Polygons

Template	pylmgc90 Type	Default Parameters	Description
Triangle	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=3, radius=0.1 m</code>	Equilateral triangle inscribed in a circle of radius 0.1 m.
Square	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=4, radius=0.1 m</code>	Regular square inscribed in a circle of radius 0.1 m.
Pentagon	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=5, radius=0.1 m</code>	Regular pentagon.
Hexagon	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=6, radius=0.1 m</code>	Regular hexagon — a shape widely used in DEM for compact assemblies.
Rectangle	<code>rigidPolygon</code>	<code>generation_type='full', vertices defined explicitly, radius=0.15 m</code>	Rectangle 0.30 m × 0.10 m defined by a list of vertices.

Walls

Template	pylmgc90 Type	Default Parameters	Description
Horizontal Wall	<code>fineWall</code>	<code>l=2.0 m, r=0.1 m, nb_polyg=20</code>	Thin horizontal wall of length 2 m, made up of 20 polygonal segments.

3D Templates

Simple Particles

Template	pylmgc90 Type	Default Parameters	Description
Small Sphere	<code>rigidSphere</code>	<code>radius = 0.05 m</code>	Rigid sphere with a small radius.
Medium Sphere	<code>rigidSphere</code>	<code>radius = 0.10 m</code>	Rigid sphere with an intermediate radius.
Large Sphere	<code>rigidSphere</code>	<code>radius = 0.20 m</code>	Rigid sphere with a large radius.

3D Shapes

Template	pylmgc90 Type	Default Parameters	Description
3D Cylinder	<code>rigidCylinder</code>	<code>radius=0.05 m, h=0.2 m</code>	Right cylinder of radius 0.05 m and height 0.2 m.
Ground Plane	<code>rigidPlan</code>	<code>axe1=2.0 m, axe2=2.0 m, axe3=0.1 m</code>	Rigid horizontal plane of 2 m × 2 m, used as a floor or ceiling.

Complex Assemblies

In addition to the individual templates, the library offers **assemblies of several avatars** generated in a single action. These assemblies create several coordinated bodies in the project.

2D Disk Cluster

Creates a 2D rigid body of type `rigidCluster` — an aggregation of rigidly linked disks, forming a non-convex particle.

Parameter	Description	Default Value
Center	Position of the cluster's reference center.	<code>[0.0, 0.0]</code>
Material	Material of type <code>RIGID</code> .	—
Model	Model with element <code>Rxx2D</code> .	—
Main radius	Radius of each disk making up the cluster (m).	<code>0.1 m</code>
Number of disks (<code>nb_disk</code>)	Number of disks in the cluster.	<code>5</code>

2D Dumbbell

Creates an empty avatar (`emptyAvatar`) composed of three contactors: two disks (`DISKx`) at the ends and a joint (`JONCx`) in the center, forming a dumbbell shape.

Parameter	Description	Default Value
Center	Position of the dumbbell's center.	<code>[0.0, 0.0]</code>
Total length	Distance between the centers of the two disks (m).	<code>0.3 m</code>
Disk radius	Radius of the two end pieces (m).	<code>0.05 m</code>

Automatically Generated Contactors:

Contactor	Shape	Parameters
Left disk	DISKx	byrd = radius, positioned at -length/2
Right disk	DISKx	byrd = radius, positioned at +length/2
Central body	JONCx	axe1 = length, axe2 = radius × 0.3

2D Rectangular Box (Box Container)

Creates a rectangular container open at the top, composed of **3 smooth walls** (`smoothWall`): a bottom wall and two side walls.

Parameter	Description	Default Value
Total width	Interior horizontal dimension of the box (m).	—
Height	Vertical dimension of the box (m).	—
Wall thickness	Thickness of each wall (m).	—
Center	Position of the center of the box.	—

Bodies Created:

Body	Type	Position
Bottom wall	<code>smoothWall</code>	<code>center_y - height/2</code>
Left wall	<code>smoothWall</code>	<code>center_x - width/2</code>
Right wall	<code>smoothWall</code>	<code>center_x + width/2</code>

2D V-Hopper

Creates a conical hopper composed of **2 inclined walls** (`rigidPolygon` with `generation_type='full'`), forming a funnel open at the top and bottom.

Parameter	Description
Top width	Upper opening of the hopper (m).
Bottom width	Lower opening of the hopper (m).
Height	Total height of the hopper (m).
Center	Position of the geometric center.

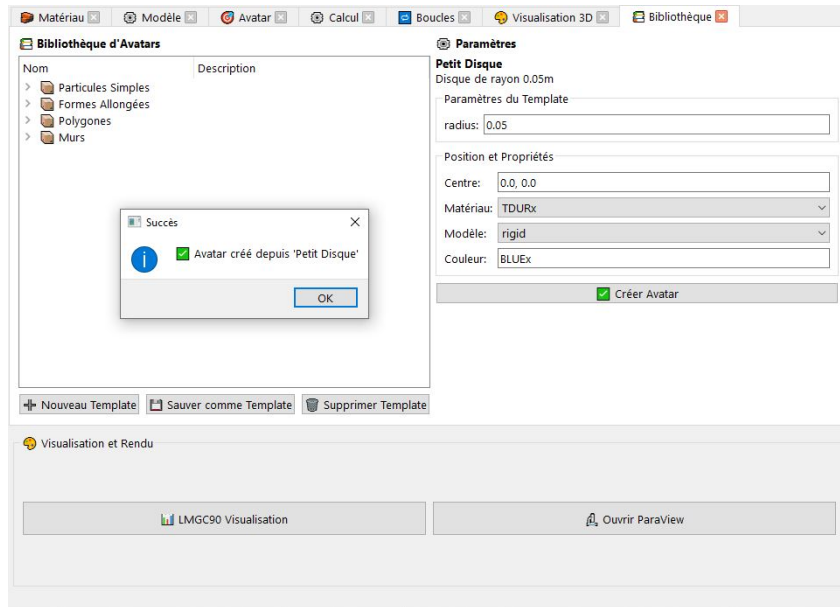
Bodies Created: left wall and right wall, each defined by 4 vertices calculated automatically from the dimensions.

Creating an Avatar from a Template

1. Select a template in the tree on the left — its properties are displayed in the panel on the right.
2. Configure the fields in the right panel:
 - **Center:** insertion coordinates (x, y) in 2D or (x, y, z) in 3D. Accepts Python expressions.
 - **Material:** select from the materials defined in the Material tab.

- **Model:** select from the models defined in the Model tab.
 - **Color:** 5-character LMGC90 color code.
3. Click **Create Avatar**.

The avatar is added to the project's list and appears in the model tree.

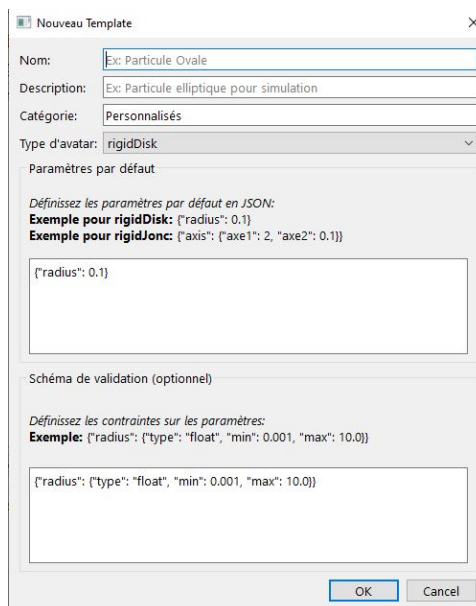


Creating an avatar from a template

Creating a New Template

It is possible to save an existing avatar as a new template to reuse it in other projects.

1. Click the **New template** button.
2. The template creation dialog box opens.
3. Enter the name and description of the template.
4. The geometric parameters of the selected avatar are picked up automatically.
5. Confirm to add the template to the library.



Important Notes

Current dimension: the list of templates automatically adapts to the project's dimension. If the dimension is changed in the Model tab, the Library tab refreshes and displays only the compatible templates (2D or 3D).

Material and model required: a material and a model must be defined in the project before an avatar can be created from a template. If no material or model is available, an error message is displayed.

Default parameters: templates use reasonable default values. These values must be adjusted according to the units and scale of the model. In particular, radii and lengths in meters must be consistent with the granulometry and dimensions of the scene.

Assemblies: complex assemblies (box, hopper, dumbbell) create several avatars simultaneously in the project. They all appear in the model tree and can be modified individually after their creation.

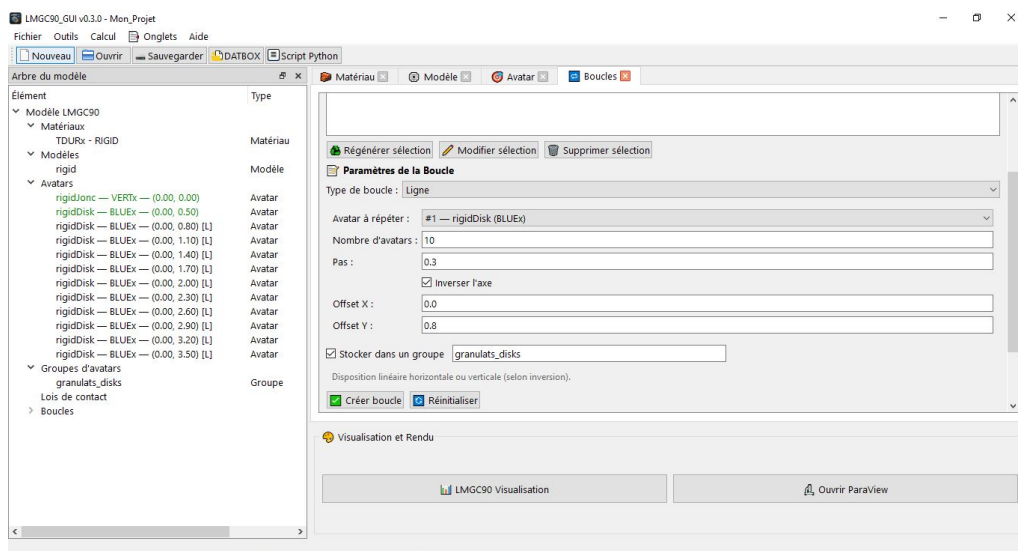
Python expressions: the Center field accepts Python expressions evaluated via `SafeEvaluator`, as in all other tabs of the interface: `avatar[0].x + 0.5`, `math.sqrt(2) * r`, etc.

Parametric Avatar Generation (Loops)

The **Loops** tab (**Ctrl+6**) allows you to automatically create series of avatars based on geometric patterns or Python expressions. It offers two complementary mechanisms:

- **Geometric loops**: automatic positioning of a model avatar according to a pattern (Circle, Grid, Line, Spiral).
- **Generic for loops**: generation of avatars, materials, models, contact laws, visibility tables, or DOF operations via Python expressions evaluated at each iteration.

The created loops are saved in the project and reconstructed when reloaded. They are also exported into the generated Python script.



captures/boucle_disk_ligne.JPG

Part 1 — Geometric Loops

Principe

A geometric loop takes a **model avatar** already defined in the Avatar tab, computes a list of positions according to the chosen pattern, then creates an identical copy of the avatar at each position. All the attributes of the model avatar are copied (type, material, model, radius, color, etc.).

Form Fields

Field	Description
Model avatar	Index of the avatar whose properties will be copied to each position. Select from the dropdown list of existing avatars.
Loop type	Geometric placement pattern. See the 4 types below.
Number of elements	Number of avatars to create.

Field	Description
Offset X	Offset of the pattern's center along X (m).
Offset Y	Offset of the pattern's center along Y (m).
Group	Name of the avatar group in which to store the generated avatars. Left empty to not create a group.

Types of Geometric Loops

Circle

Distributes **count** avatars uniformly on a circle of radius **radius**, centered at (**offset_x**, **offset_y**).

Parameter	Description	Default
Radius	Radius of the placement circle (m).	—
Offset X / Y	Center of the circle.	0.0

Typical use: ring particles, rotating drum, bolt circle, circular granulometry.

Grid

Distributes **count** avatars on a square grid, with a spacing **step** between elements.

Parameter	Description	Default
Step	Distance between two neighboring avatars on the grid (m).	—
Offset X / Y	Bottom-left corner of the grid.	0.0

*The grid is always square. If **count** is not a perfect square, the actual number of avatars created is n_side^2 . Example: **count**=10 → 4×4 grid = 16 avatars.*

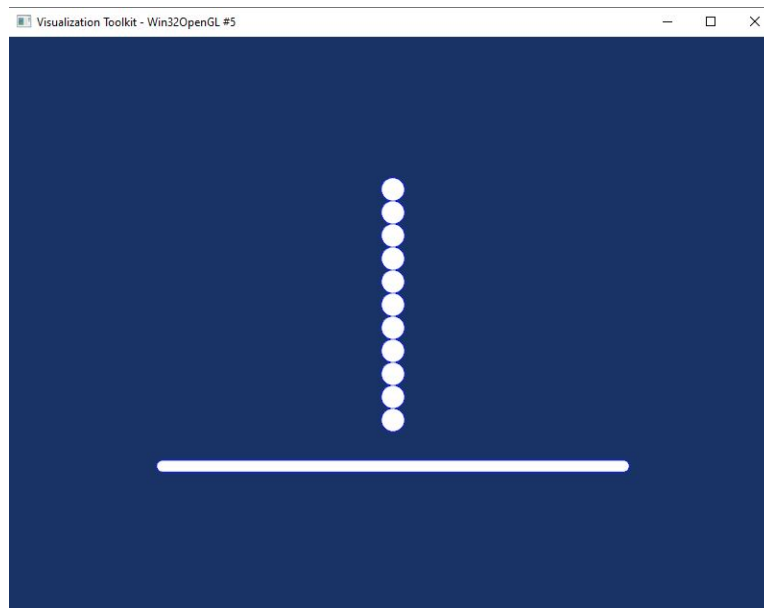
Typical use: regular particle stacking, perforated plate, square lattice.

Line

Aligns **count** avatars in a straight line, separated by a step **step**, in the X or Y direction.

Parameter	Description	Default
Step	Distance between two consecutive avatars (m).	—
Offset X / Y	Position of the first avatar.	0.0
Invert axis	If checked, the line is oriented along Y (vertical) instead of X (horizontal).	unchecked

Typical use: row of posts, line of particles, simple wall.



captures/rendu_boucle_disk_ligne.JPG

Spiral

Arranges **count** avatars on an Archimedean spiral, with an initial radius **radius** that grows by a factor **spiral_factor** at each iteration.

Parameter	Description	Default
Initial radius	Starting radius of the spiral (m).	—
Spiral factor	Radius increment per element (m).	—
Offset X / Y	Center of the spiral.	0.0

Typical use: mixers, spiral deposits, decorative arrangements.

Managing Geometric Loops

Creating a Loop

Fill in the form and click **■ Generate the loop**. The avatars are created immediately in the project.

Editing a Loop

Select a loop in the list, modify the parameters, and click **■ Update**. The old avatars are deleted and recreated with the new parameters.

Deleting a Loop

Select a loop in the list and click **■■ Delete**. All avatars generated by this loop are automatically deleted.

Cascading deletion: deleting a loop deletes **all** the avatars it generated (indices recorded in **loop.generated_indices**), in reverse order so as not to shift the other indices.

Part 2 — Generic **for** Loops

Principe

Generic **for** loops allow you to create series of elements by varying a **loop variable** over an interval defined by Python expressions. The **element type** to create is chosen among: avatar, material, model, contact law, visibility table, or DOF operation.

Paramètres de la Boucle

Type de boucle : For

Configuration Boucle For

Variable de boucle : i

Début : 0

Fin : 10

Step : 1

Type d'élément : avatar

Template JSON

Exemples de templates :

- Avatar : {"avatar_type": "rigidDisk", "center": "[i*0.5, 0]", "material_name": "TDURx", "model_name": "rigid", "radius": "0.1+i*0.01"}
- Matériau : {"name": "MAT "+str(i), "material_type": "RIGID", "density": "2800+i*100"}
- Variables : Utilisez 'i' (variable de boucle) et vos variables dynamiques

{ "avatar_type": "rigidDisk", "center": "[i*0.5, 0]", "material_name": "TDURx", "model_name": "rigid", "radius": "0.1+i*0.01" }

captures/for_generique.JPG

At each iteration, the **configuration template** is evaluated with the loop variable injected into the evaluation context.

Form Fields

Field	Description	Example
Loop variable	Name of the Python variable available in the template.	i, k, n
Start	Python expression for the starting value.	0, 1, n_start
End	Python expression for the ending value (excluded).	10, count, n_start + 5
Step	Python expression for the increment.	1, 2, -1
Element type	Type of the object to create at each iteration.	see table below
Template	Configuration of the element, with the loop variable usable in the values.	see sections by type
Group	Name of the avatar group (avatars only).	ma_ligne

Available Evaluation Context

The expressions in the template and the loop bounds have access to the following elements:

Symbol	Description
i (or the loop variable)	Current value of the iteration
math	Full Python math module
sqrt, pi, e	Math shortcuts
abs, min, max, sum, len	Standard Python functions
str, int, float	Type conversions
Dynamic variables	All variables defined in the Tools → Dynamic Variables menu

Available Element Types

avatar Type — Avatar Creation

Creates an avatar at each iteration. All the parameters of the Avatar tab are available in the template.

Template Parameters:

Key	Description	Example
avatar_type	pylmgc90 type of the avatar.	"rigidDisk"
center	Python expression returning a list <code>[x, y]</code> or <code>[x, y, z]</code> . The loop variable can be used.	<code>[i * 0.1, 0.0]</code>
material_name	Name of the material (string).	"TDURx"
model_name	Name of the model (string).	"rigid"
color	LMGC90 color code (5 characters).	"BLUEx"
radius	Python expression for the radius (m).	<code>"0.05 + i * 0.01"</code>
axis	Dict <code>{axe1: expr, axe2: expr}</code> for joints / planes.	<code>{"axe1": "1.0", "axe2": "0.1"}</code>
nb_vertices	Expression for the number of vertices (polygons).	"6"
generation_type	"regular", "full", or "bevel".	"regular"
vertices	Python expression returning the list of vertices.	<code>"[[-0.1, -0.1], [0.1, -0.1], [0.1, 0.1], [-0.1, 0.1]]"</code>
wall_params	Dict of wall parameters.	<code>{"l": "2.0", "r": "0.05"}</code>
contactors	List of contactors (same format as the Empty Avatar tab).	
is_hollow	Boolean — creates a hollow disk.	false

Generated Script:

```
for i in range(0, 10, 1):
    center = [i * 0.1, 0.0]
    body = pre.rigidDisk(
        center=center,
        model=mods['rigid'],
        material=mats['TDURx'],
        color='BLUEx',
        r=0.05 + i * 0.01
    )
    bodies.addAvatar(body)
    bodies_list.append(body)
```

Typical use: line of disks with increasing radii, grid of heterogeneous avatars, series of bodies with varying parameters.

material Type — Material Creation

Creates a material at each iteration. Useful for generating a family of materials with graded properties.

Template Parameters:

Key	Description	Example
<code>name</code>	Python expression for the name (string, 5 chars max).	<code>"str('mat') + str(i)"</code>
<code>material_type</code>	LMGC90 material type.	<code>"RIGID"</code>
<code>density</code>	Python expression for the density (kg/m³).	<code>"2800 + i * 100"</code>
<code>properties</code>	Dict of additional properties.	<code>{"young": "1e9 + i * 1e8"}</code>

Generated Script:

```
for i in range(0, 5, 1):
    mat_name = str('mat') + str(i)
    density_val = 2800 + i * 100
    mats[mat_name] = pre.material(
        name=mat_name,
        materialType='RIGID',
        density=density_val
    )
    materials.addMaterial(mats[mat_name])
```

model Type — Model Creation

Creates a finite element model at each iteration.

Template Parameters:

Key	Description	Example
<code>name</code>	Python expression for the name.	<code>"'mod' + str(i)"</code>
<code>physics</code>	Physics.	<code>"MECAx"</code>
<code>element</code>	Finite element.	<code>"Rxx2D"</code>
<code>dimension</code>	Dimension (2 or 3).	<code>2</code>
<code>options</code>	Dict of numerical options.	<code>{}</code>

contactLaw Type — Contact Law Creation

Creates a contact law at each iteration. Useful for generating laws with different friction coefficients.

Template Parameters:

Key	Description	Example
<code>name</code>	Python expression for the name.	<code>"'law' + str(i)"</code>
<code>law_type</code>	Law type (<code>IQS_CLB</code> , etc.).	<code>"IQS_CLB"</code>
<code>friction</code>	Python expression for the friction coefficient.	<code>"0.1 + i * 0.05"</code>

Generated Script:

```
for i in range(0, 5, 1):
    law_name = 'law' + str(i)
    laws[law_name] = pre.tact_behav(
        name=law_name,
        law='IQS_CLB',
        fric=0.1 + i * 0.05
    )
    tacts.addBehav(laws[law_name])
```

visibility Type — Visibility Table Creation

Creates a visibility rule (contact detection table) at each iteration.

Template Parameters:

Key	Description	Example
candidate_body	Candidate body (RBDY2, RBDY3).	"RBDY2"
candidate_contactor	Candidate contactor (DISKx, etc.).	"DISKx"
candidate_color	Expression for the candidate color.	"'BLUEx'"
antagonist_body	Antagonist body.	"RBDY2"
antagonist_contactor	Antagonist contactor.	"DISKx"
antagonist_color	Expression for the antagonist color.	"'REDx'"
behavior_name	Name of the associated contact law.	"'law' + str(i)"
alert	Alert distance (m).	"0.05 + i * 0.01"

dof Type — DOF Operations

Applies a boundary condition (imposed degree of freedom) to one or more avatars at each iteration.

Template Parameters:

Key	Description	Example
operation_type	pylmgc90 operation type.	"imposeDrivenDof"
target_type	Always "avatar".	"avatar"
target_value	Python expression for the index of the target avatar.	"i", "i + 10"
parameters	Dict of pylmgc90 kwargs.	{"component": "[1, 2]", "dofty": "\"vlocy\""}"

Generated Script:

```
for i in range(0, 5, 1):  
    bodies_list[i].imposeDrivenDof(component=[1, 2], dofty="vlocy")
```

Managing for Loops

for loops are listed separately from geometric loops. The same operations are available: create, edit (regenerates the elements), delete (removes the generated elements).

Avatar Groups

Both types of loops can store the generated avatars in a **named group**. A group is a list of avatar indices accessible throughout the project under the given name.

Groups allow you to:

- apply a DOF condition to all the avatars of a group at once
- target a set of avatars in visibility tables

- distinguish multiple avatar populations for data extraction (chipy `GetBodyVector`)

If the Group field is left empty, no group is created and the avatars are simply added to the global list.

Summary of Parameters by Geometric Loop Type

Type	Required Parameters	Optional Parameter	Formula
Circle	count, radius	offset_x, offset_y	$x = \text{offset_x} + \text{radius} \times \cos(2\pi i / \text{count})$
Grid	count, step	offset_x, offset_y	$x = \text{offset_x} + (i \% n_side) \times \text{step}$
Line	count, step	offset_x, offset_y, invert_axis	$x = \text{offset_x} + i \times \text{step}$ (or Y if inverted)
Spiral	count, radius, spiral_factor	offset_x, offset_y	$r = \text{radius} + i \times \text{spiral_factor}$

Important Notes

Non-deletable model avatar: an avatar used as a model by a geometric loop cannot be deleted as long as the loop exists. A warning message is displayed.

Cascading update: editing a loop (geometric or `for`) deletes and recreates all its avatars. If other elements of the project reference these avatars (DOF conditions, visibility tables), they may be invalidated.

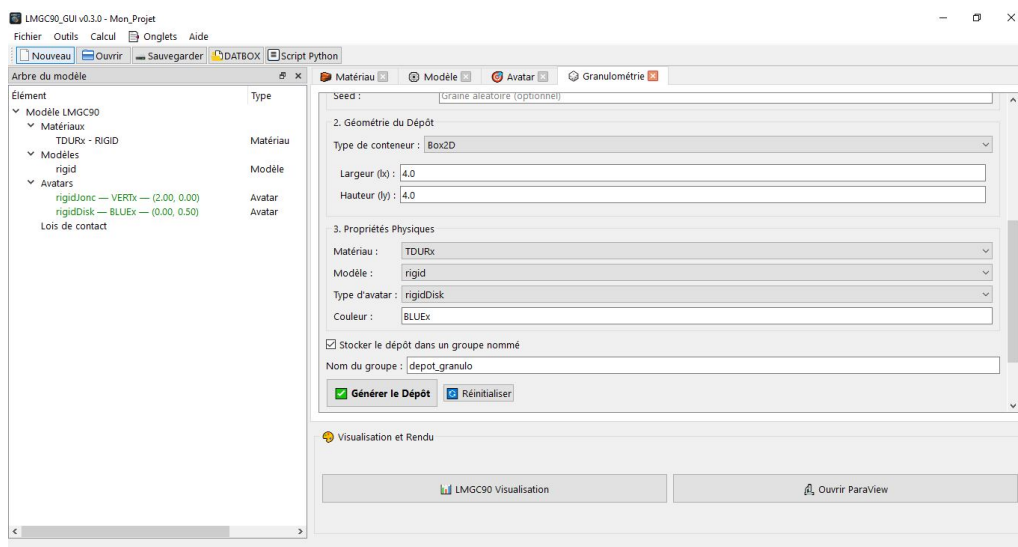
Python expressions: in `for` loops, expressions are evaluated via `SafeEvaluator`. Malformed expressions generate an error message without blocking the project. Dynamic variables defined in **Tools** → **Dynamic Variables** (`Ctrl+V`) are accessible in all expressions.

3D dimension: geometric loops generate 2D `[x, y]` or 3D `[x, y, z]` centers depending on the project's dimension. 3D avatars (spheres, polyhedra) can be generated in a `for` loop with explicitly 3D centers in the template.

Number of elements and performance: creating more than a few thousand avatars via a loop can slow down the interface. For large assemblies (> 5,000 avatars), use the granulometry generator or the dedicated wizards.

Granulometry

The **Granulometry** tab (**Ctrl+7**) allows you to create, manage, and delete granular distributions directly from the interface, without going through the wizard. It is suited for quick generations from avatars and materials already defined in the project.



Example of a granular deposit of disks and jons

General Interface

The tab is divided into two areas:

- **List of distributions** (top): a table displaying all the granulometric generations of the project with their container type, number of particles, min/max radii, and associated group. Double-click to edit. Right-click to access the context menu.
- **Creation / editing form** (bottom): configuration fields for the distribution.

Form Fields

Model Avatar

Field	Description
Model avatar	Selection of an existing rigid avatar in the project (defined in the Avatar tab). All its attributes are copied onto each generated particle: type (rigidDisk or rigidSphere), material, model, color.

The model avatar must be of type `rigidDisk` (2D) or `rigidSphere` (3D). It indirectly defines the material, model, and color of all the generated particles.

Particle Distribution

Field	Description	Range	Default
Number of particles	Number of particles requested from the deposit algorithm. The number actually placed may be lower if the container is saturated.	10 to 10,000	200
Minimum radius (rmin)	Minimum radius of the particles (m).	0.001 to 10.0	0.05 m
Maximum radius (rmax)	Maximum radius of the particles (m). Must be strictly greater than rmin.	0.001 to 10.0	0.15 m

Performance: beyond ~1,500 avatars, the interface refresh can slow down significantly. For large assemblies, use the **granulometry wizard** (Ctrl+Shift+G) or **numpy generation** (Wizards menu).

Reproducibility

Field	Description	Default
Use a seed	Checkbox. If enabled, radii and positions are generated identically on every run.	unchecked
Seed value	Integer from 0 to 999,999. Passed to <code>pre.granulo_Random(seed=...)</code> .	12345

Deposit Container

Selection of the container type and its dimensions. 2D containers are available regardless of the project's dimension — adapt according to the chosen model avatar type.

2D Containers

Container	Parameters	pylmgc90 Function	Description
Box2D	lx (m), ly (m)	<code>pre.depositInBox2D(radii, lx, ly)</code>	Rectangular box. Gravitational deposit from the top.
Disk2D	r (m)	<code>pre.depositInDisk2D(radii, r)</code>	Circular disk.
Couette2D	rint (m), rext (m)	<code>pre.depositInCouette2D(radii, rint, rext)</code>	Couette cell — annular space between two cylinders.
Drum2D	r (m)	<code>pre.depositInDrum2D(radii, r)</code>	Rotating circular drum.

3D Containers

Container	Parameters	pylmgc90 Function	Description
Box3D	lx, ly, lz (m)	<code>pre.depositInBox3D(radii, lx, ly, lz)</code>	Rectangular parallelepiped box.

Container	Parameters	pylmgc90 Function	Description
Sphere3D	r (m)	<code>pre.depositInSphere3D(radii, r)</code>	3D sphere.
Cylinder3D	r (m)	<code>pre.depositInCylinder3D(radii, r)</code>	3D cylinder.

Options

Field	Description	Default
Color	5-character LMGC90 color code. Overrides the model avatar's color if filled in.	BLUEx
Store in a group	If checked, all generated avatars are recorded in a group named in <code>state.avatar_groups</code> .	checked
Group name	Group identifier. Automatic default: <code>granulo_{container_type}</code> (e.g. <code>granulo_box2d</code>).	<code>granulo_box2d</code>

Generating a Distribution

Click **Generate the deposit** to launch generation. The algorithm:

1. Randomly draws `nb` radii within `[rmin, rmax]` via `pre.granulo_Random`.
2. Positions the particles in the chosen container via `pre.depositInXxx`.
3. Creates a `rigidDisk` or `rigidSphere` avatar for each placed particle.
4. Saves the configuration in `state.granulo_generations`.
5. Emits the `granulo_generated` signal to refresh the interface.

The `granulo_generated` signal triggers a full refresh of the model tree and all tabs.

Managing Distributions

Editing a Distribution

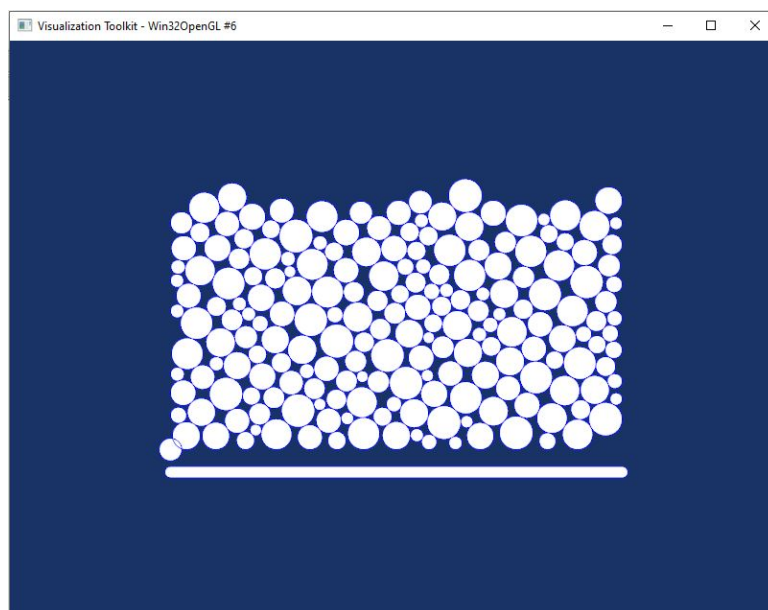
Select a distribution in the list and click **Edit**. The configuration is loaded into the form in **Edit** mode. Modify the parameters and click **Save** to regenerate. The old avatars are deleted and replaced.

Deleting a Distribution

Select it and click **Delete**. All avatars generated by this distribution are automatically deleted (indices recorded in `GranuloGeneration.generated_indices`). The `granulo_deleted` signal is emitted.

Configuration Example

Field	Example Value
Model avatar	#0 — rigidDisk / TDURx / rigid
Number of particles	400
Minimum radius	0.05 m
Maximum radius	0.075 m
Container	Box2D — lx=4.0 m, ly=4.0 m
Color	BLUEx
Group	granulo_box2d



Rendering of the granular deposit

Notes

Actual vs. requested count: the number of particles actually placed (`_nb_remaining`) may be lower than the number requested if the container is too small or the packing density is reached.

Performance limit: the tab calls `add_avatar()` for each particle, which emits one signal per avatar. Beyond ~1,500 avatars, the interface slows down. For larger assemblies, use the **granulometry wizard**, which inserts avatars directly into the `pylmgc90` containers without an individual signal.

Automatic group: if no group name is entered, a name is automatically generated in the form `granulo_{container_type}`.

Granulometry Wizard — pylmgc90

The **granulometric distribution wizard** guides you step by step through the creation of a deposit of rigid particles (2D disks or 3D spheres) via the official `pylmgc90` API. It automatically creates or reuses the material and model, configures the radius distribution and the deposit container, then generates all the avatars at once in an optimized way.

Access: menu **Wizards** → **Granulometry Wizard** · shortcut `Ctrl+Shift+G`

Recommended for: up to about 8,000 particles. Beyond that, the interface may slow down. Use numpy generation for very large assemblies.

Overview of Steps

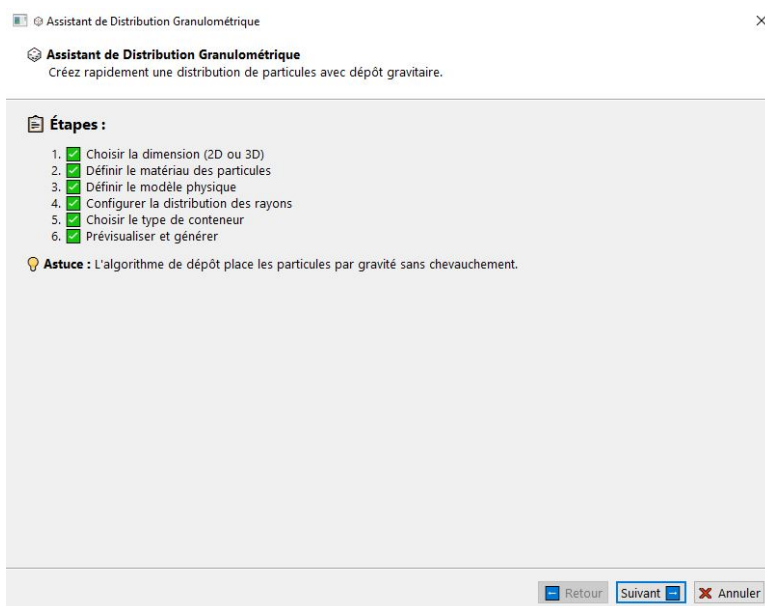
The wizard consists of **7 active pages** (the Preview page is disabled in the current version).

Page	Title	Description
0	Introduction	Presentation of the algorithm
1	Dimension	2D (disks) or 3D (spheres)
2	Material	Create or reuse a RIGID material
3	Model	Create or reuse a Rxx2D / Rxx3D model
4	Distribution	Number of particles, radii, seed
5	Container	Geometry and dimensions of the deposit
6	Summary	Verification before generation

Page 0 — Introduction

Presents the principle of the algorithm in two steps:

1. `pre.granulo_Random(nb, r_min, r_max, seed)` — randomly generates **nb** radii within $[r_min, r_max]$ following a uniform distribution.
2. `pre.depositInXxx(radii, ...)` — performs a gravitational deposit of the particles in the chosen container, without overlap.



captures/assistant_granulo_page1.JPG

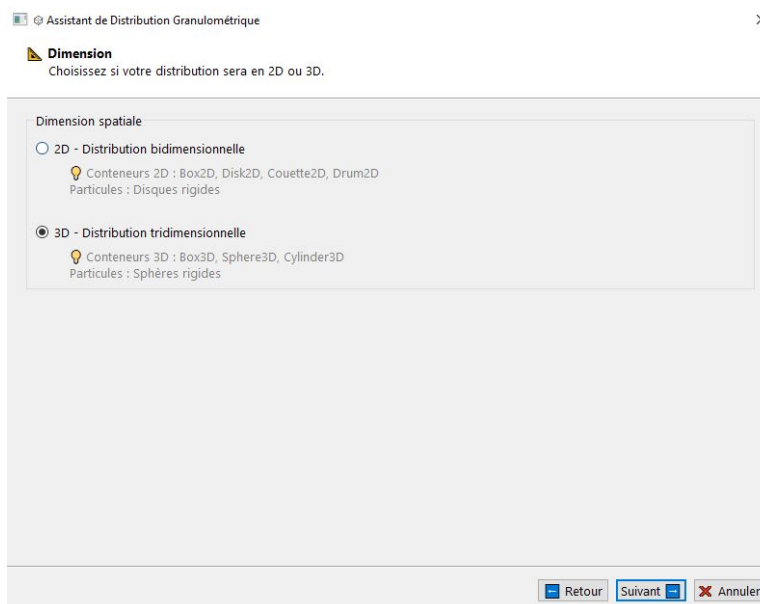
Click **Next** ➡ to begin.

Page 1 — Dimension

Choice	Particle Type Generated	Available Containers
2D	rigidDisk	Box2D, Disk2D, Couette2D, Drum2D
3D	rigidSphere	Box3D, Sphere3D, Cylinder3D

The **2D** value is selected by default.

The dimension determines the type of avatar created, the list of containers available on page 5, and the model element (Rxx2D or Rxx3D).



Choosing the dimension

Page 2 — Particle Material

Two modes:

Mode A — Use an Existing Material (suggested if materials already exist)

Dropdown list of all the materials in the project.

Mode B — Create a New Material (automatically checked if none exist)

Field	Description	Default
Name	Identifier, 5 characters maximum.	TDURx
Density	Density (kg/m³). Range: 100 to 20,000.	2500 kg/m³

*The type is always **RIGID** — granular particles are rigid bodies.*

Assistant de Distribution Granulométrique

Matériau des Particules
Créez un nouveau matériau ou utilisez un existant.

Source du matériau
☒ Créer un nouveau matériau

Nouveau matériau
Nom : TDURx
Densité : 2500,00 kg/m³

Retour Suivant Annuler

Creating the material

Page 3 — Physical Model

Two modes:

Mode A — Use an Existing Model

Dropdown list of the project's models.

Mode B — Create a New Model

Field	Value
Name	5 characters maximum. Default: rigid .
Physics	MECAx (automatic)
Element	Rxx2D (2D) or Rxx3D (3D) — adapted automatically

Assistant de Distribution Granulométrique

Modèle Physique
Créez un nouveau modèle ou utilisez un existant.

Source du modèle

☒ Créer un nouveau modèle

Nouveau modèle

Nom : rigid

Retour
Suivant
Annuler

Creating the model

Page 4 — Particle Distribution

Number of Particles

Field	Description	Range	Default
Number requested	Number of particles to generate. The number actually placed may be lower if the container is saturated.	10 to 10,000	200

A visual indicator signals the density:

- **Orange** (< 100): low density
- **Blue** (100 to 499): medium density
- **Green** (≥ 500): high density

Radius Distribution

Field	Description	Range	Default
Minimum radius	Smallest radius of the particles (m).	0.001 to 10.0	0.05 m
Maximum radius	Largest radius (m). Must be > Rmin.	0.001 to 10.0	0.15 m
Rmax/Rmin ratio	Calculated automatically. Indicates the range of polydispersity. A ratio ≥ 3 produces a very spread-out distribution.	—	3.00

A **real-time histogram** (20 bins, 200 preview radii) visualizes the size distribution before generation.

Reproducibility

Field	Description	Default
Use a seed	Checkbox. Makes generation reproducible.	unchecked
Seed value	Integer from 0 to 999,999. Passed to <code>pre.granulo_Random(seed=...)</code> .	12345

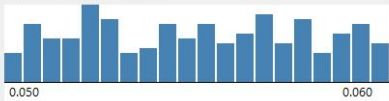
Assistant de Distribution Granulométrique

Distribution des Particules
Définissez le nombre et la taille des particules.

Nombre de particules
Nombre demandé : 500
Niveau : Élevé

Distribution des rayons
Rayon minimum : 0,05 m
Rayon maximum : 0,06 m
Ratio Rmax/Rmin : 1,20

Aperçu de la distribution :



Reproductibilité
☐ Utiliser une graine aléatoire 12345

Retour Suivant Annuler

Radius distribution

Page 5 — Deposit Container

The list of containers automatically adapts to the dimension chosen on page 1.

2D Containers

Box2D — 2D Rectangular Box

Parameter	Description	Default
Width (lx)	Horizontal dimension of the box (m).	4.0 m
Height (ly)	Vertical dimension of the box (m).	4.0 m

pylmgc90 Function: `pre.depositInBox2D(radii, lx, ly)` **Usage:** biaxial compression test, soil column, shear cell.

Disk2D — 2D Circular Disk

Parameter	Description	Default
Radius (r)	Radius of the container disk (m).	2.0 m

pylmgc90 Function: `pre.depositInDisk2D(radii, r)` **Usage:** circular silo, 2D drum.

Couette2D — 2D Couette Cell

Annular space between an inner cylinder of radius `rint` and an outer cylinder of radius `rext`.

Parameter	Description	Default
Inner radius (<code>rint</code>)	Radius of the inner cylinder (m).	2.0 m
Outer radius (<code>rext</code>)	Radius of the outer cylinder (m). Must be > <code>rint</code> .	4.0 m

pylmgc90 Function: `pre.depositInCouette2D(radii, rint, rext)` **Usage:** Couette rheometer, effective viscosity measurement, annular flow.

Drum2D — 2D Rotating Drum

Parameter	Description	Default
Radius (<code>r</code>)	Radius of the drum (m).	2.0 m

pylmgc90 Function: `pre.depositInDrum2D(radii, r)` **Usage:** rotating mixer, ball mill, dryer.

3D Containers

Box3D — 3D Rectangular Box

Parameter	Description
Width (<code>lx</code>)	Dimension along X (m).
Depth (<code>ly</code>)	Dimension along Y (m).
Height (<code>lz</code>)	Dimension along Z (m).

pylmgc90 Function: `pre.depositInBox3D(radii, lx, ly, lz)` **Usage:** 3D triaxial test, 3D soil model.

Sphere3D — 3D Container Sphere

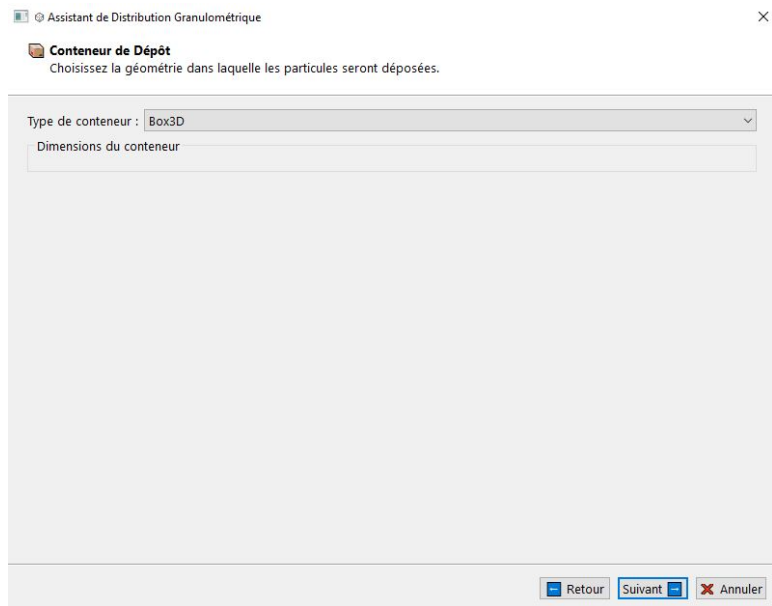
Parameter	Description
Radius (<code>r</code>)	Radius of the container sphere (m).

pylmgc90 Function: `pre.depositInSphere3D(radii, r)`

Cylinder3D — 3D Container Cylinder

Parameter	Description
Radius (<code>r</code>)	Radius of the cylinder (m).

pylmgc90 Function: `pre.depositInCylinder3D(radii, r)` **Usage:** cylindrical column, silo, 3D oedometric test.



Selecting the Box3D container

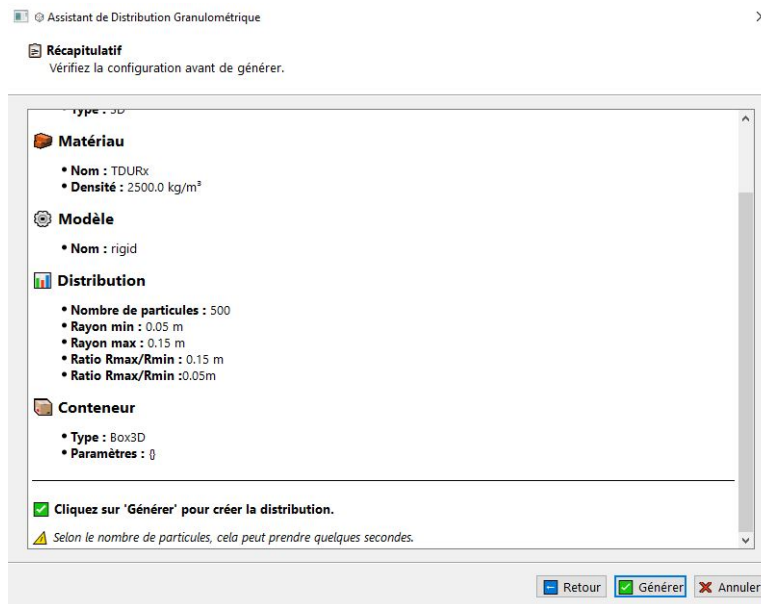
Page 6 — Summary

Displays a complete table before generation:

Section	Information
Dimension	2D or 3D
Material	Name and density (new) or existing material
Model	Name (new) or existing model
Distribution	Number of particles, Rmin, Rmax, Rmax/Rmin ratio
Seed	Value if enabled
Container	Type, parameters (lx, ly, r, rint, rext...)

Click **Generate** to launch. A confirmation message indicates success.

In case of error: the project state (name, path, dimension) is fully restored.



Summary

Generation Result

Element Created	Description
Material	Added to the Material tab (if created). Type RIGID .
Model	Added to the Model tab (if created). MECAX + Rxx2D / Rxx3D .
Particle avatars	One rigidDisk (2D) or rigidSphere (3D) per placed particle. Color BLUEx . Origin AvatarOrigin.GRANULO .
Automatic group	Group granulo_{container} (e.g. granulo_box3d).
GranuloGeneration	Configuration saved in state.granulo_generations for script reconstruction.

Optimization: the wizard inserts avatars directly into the `pymgc90` containers (`_bodies_container`, `_pymgc_bodies`) without emitting a signal per particle — which allows processing several thousand particles without slowing down the interface.

Important Notes

Reproducibility: two generations with the same parameters produce different results. Setting a seed guarantees the exact identity of the results — essential for parametric studies.

Rmax/Rmin ratio: a high ratio (> 3) produces a polydisperse assembly where small particles fill the gaps between large ones, increasing compactness. A ratio close to 1 gives a quasi-monodisperse assembly with a lower maximum compactness.

12,000-particle limit: beyond this, refreshing the model tree and the tabs can take several seconds. For very large assemblies, use numpy generation.

Numpy Granulometry Generation **_(beta)_**

Numpy granulometry generation is a fast generation mode suited for very large assemblies (> 5,000 particles). It opens as a simple dialog (**QDialog**) — with no steps, no pages to go through — and uses a **background computation thread** (**GranuloWorker**) so as not to block the interface during the deposit.

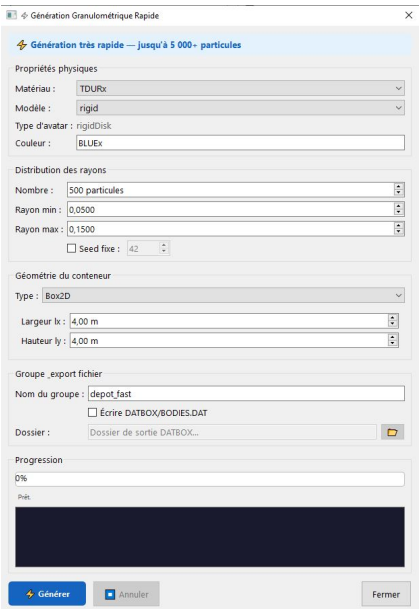
Access: menu **Wizards** → **■ Numpy granulometry generation... (beta)**

Recommended for: assemblies of **5,000 particles and more**. Below this threshold, the **pylmgc90 wizard** (**Ctrl+Shift+G**) is preferable since it uses the official **pylmgc90** deposit routines directly.

Status: **beta** feature. The positions generated may differ slightly from those produced by **pylmgc90's pre.depositInBox2D**, since the placement algorithm is different. The result is nonetheless physically consistent (no overlap).

Access

Click the **Wizards** menu, then **■ Numpy granulometry generation... (beta)**. The dialog opens directly — no step-by-step wizard.



Numpy granulometry generation dialog

Differences from the pylmgc90 Wizard

Characteristic	pylmgc90 Wizard (Ctrl+Shift+G)	Numpy Generation (beta)
Interface	7-page wizard	Single dialog
Placement algorithm	pre.depositInXxx — pylmgc90 physical gravitational deposit	GranuloGenerator via numpy — algorithmic placement
Computation	Main thread (can block the UI)	Background thread (GranuloWorker) — UI not blocked
Progress bar	No	Yes — real-time progress
Recommended limit	~8,000 particles	> 5,000 particles, no practical limit
Material / model	Creation or reuse	Uses existing materials and models

Characteristic	pylmgc90 Wizard (Ctrl+Shift+G)	Numpy Generation (beta)
3D containers	Box3D, Sphere3D, Cylinder3D	2D only (Box2D, Disk2D, Couette2D, Drum2D)
Reproducibility	Yes (seed)	Yes (seed)

Dialog Fields

Distribution Parameters

Field	Description	Range	Default
Number of particles	Number of particles requested from the placement algorithm.	10 to 100,000+	1000
Minimum radius (rmin)	Minimum radius of the particles (m).	0.001 to 10.0	0.05 m
Maximum radius (rmax)	Maximum radius of the particles (m). Must be > rmin.	0.001 to 10.0	0.10 m

Material and Model

Field	Description
Material	Dropdown list of existing materials in the project. Select the material to apply to the particles.
Model	Dropdown list of existing models in the project.

Unlike the pylmgc90 wizard, this dialog does not create a new material or a new model — they must already exist in the project.

Deposit Container

Only **2D** containers are available in the current version.

Container	Parameters	Description
Box2D	lx (m), ly (m)	Rectangular box.
Disk2D	r (m)	Circular disk.
Couette2D	r _{int} (m), r _{ext} (m)	Couette cell — annular space.
Drum2D	r (m)	Rotating drum.

Reproducibility

Field	Description	Default
Use a seed	Checkbox. Makes generation reproducible.	unchecked
Seed value	Integer from 0 to 999,999.	12345

Options

Field	Description	Default
Color	5-character LMGC90 color code.	BLUEx
Group	Name of the group in which to store the generated avatars (<code>state.avatar_groups</code>).	granulo_numpy

Internal Operation

Computation Thread — `GranuloWorker`

Generation runs in a separate thread (`GranuloWorker`, a subclass of `QThread`) so as not to block the interface.

Signals Emitted by `GranuloWorker`:

Signal	Parameters	Trigger
progress_updated	(nb_done, nb_total, message)	At each computation step — updates the progress bar
data_ready	list[{center, radius}]	Computation finished — sends the data to the main thread
error_occurred	str	On exception — displays an error message

Execution Sequence:

```
1. GranuloFastDialog.exec()
↓
2. GranuloWorker.start() ← secondary thread
↓
3. GranuloGenerator.generate(config)
   ■■■ pre.granulo_Random(nb, rmin, rmax, seed)
   ■■■ pre.depositInXxx(radii, ...)
↓
4. GranuloWorker.data_ready.emit(particles_data)
↓
5. Main thread: creation of the avatars
↓
6. GranuloFastDialog.granulo_generated.emit()
↓
7. MainWindow._refresh_all()
```

Data Returned by `GranuloWorker`

Each particle is sent to the main thread as a dictionary:

```
{
'center': [x, y], # numpy coordinates → Python list
'radius': 0.075 # radius as a float
}
```

Placement Algorithm

`GranuloGenerator.generate()` calls the same `pymgc90` routines as the Granulometry tab:

```
radii = pre.granulo_Random(config.nb_particles, config.radius_min, config.radius_max, config.seed)
nb_remaining, coor = pre.depositInBox2D(radii, params['lx'], params['ly'])
coor.shape = [coor.size // 2, 2]
radii = radii[nb_remaining]
return nb_remaining, coor, radii
```

The difference from the wizard is that the computation is offloaded to a thread (**QThread**), which prevents the interface from freezing on very large assemblies.

Progress and Cancellation

During generation, the dialog displays:

- A **progress bar** updated by **progress_updated** (current value / total).
- A **status message** describing the current step (e.g.: "Computing the granulometric deposit...", "Deposit computed successfully").
- A **Cancel** button that calls **GranuloWorker.stop()** to interrupt the computation.

Generation Result

Element Created	Description
Particle avatars	One rigidDisk per placed particle. Origin AvatarOrigin.GRANULO .
Group	Group named in state.avatar_groups (default: granulo_numpy).
GranuloGeneration	Configuration saved in state.granulo_generations .
Signal	granulo_generated emitted → MainWindow._refresh_all()

Avatars are created in the **main thread** after receiving the **data_ready** signal — creation remains synchronous, but the heavy computation (physical deposit) is asynchronous.

Important Notes

Material and model required: unlike the **pylmgc90** wizard, the dialog does not offer to create a material or a model. Create them beforehand in the Material tab (e.g. **TDURx**, **RIGID**, 2500 kg/m³) and in the Model tab (e.g. **rigid**, **MECAX**, **Rxx2D**).

2D containers only: the current beta version does not support 3D containers (**Box3D**, **Sphere3D**, **Cylinder3D**). For 3D generation, use the **pylmgc90** wizard.

Identical results on reload: the generated positions and radii are saved via **GranuloGeneration** in the project. When reloading, the deposit is **recomputed** from the parameters — if no seed was set, the positions will be different. Set a seed to guarantee exact identity between sessions.

Thread and resources: if the dialog is closed during the computation, the **GranuloWorker** thread continues running in the background until it finishes. Wait for the computation to finish, or click **Cancel** before closing.

Conditions aux limites (DOF)

Boundary Conditions (DOFs)

The **DOF** tab (**Ctrl+5**) allows you to apply mechanical, thermal, or initial boundary conditions to the avatars of the project. Each operation is recorded in `state.operations`, applied immediately to the `pylmgc90` objects, and exported in the generated Python script.

Operating Principle

A **DOF operation** (`DOFOperation`) is made up of:

Field	Description
<code>operation_type</code>	Nature of the operation: <code>translate</code> , <code>rotate</code> , <code>imposeDrivenDof</code> , <code>imposeInitValue</code>
<code>target_type</code>	Target: <code>'avatar'</code> (single index) or <code>'group'</code> (group name)
<code>target_value</code>	Avatar index (int) or group name (str)
<code>parameters</code>	Dictionary of parameters passed directly to the <code>pylmgc90</code> method

Tab Interface

The tab is organized into two areas:

- **Operations list** (top): a table of all recorded operations with their type, target, and main parameters. Double-click to edit. Right-click to access the context menu.
- **Creation / editing form** (bottom): fields that adapt dynamically to the chosen operation type.

Target Selection

Field	Description
Target type	<code>Avatar</code> (numeric index) or <code>Group</code> (group name).
Target	If Avatar: index of the avatar in the list (0-based). If Group: dropdown list of all groups defined in the project (loops, granulometry, masonry, etc.).

All groups created in the Loops, Granulometry, and Masonry tabs automatically appear in the list of available groups.

The Four Operations

1. `translate` — Rigid Displacement

Moves all the nodes of the avatar by a translation vector. A purely geometric operation — it does not impose a kinematic condition for the simulation.

pymgc90 signature:

```
avatar.translate(dx=0., dy=0., dz=0.)
```

Parameters:

Parameter	Type	Default	Description
<code>dx</code>	float	<code>0.0</code>	Translation along the X axis (m)
<code>dy</code>	float	<code>0.0</code>	Translation along the Y axis (m)
<code>dz</code>	float	<code>0.0</code>	Translation along the Z axis (m) — 3D only

Example:

```
bodies[0].translate(dx=0.5, dy=0.0)
```

The avatar's position in `state.avatars` is automatically resynchronized after the translation (`_sync_avatar_position`). This updates the display in the model tree and the 3D viewer.

2. rotate — Rigid Rotation

Applies a rotation to the avatar about a given center. Two description modes are available: Euler angles or axis-angle.

pymgc90 signature:

```
avatar.rotate(
    description='Euler', # or 'axis'
    phi=0., theta=0., psi=0., # Euler angles (rad) – Euler mode
    alpha=0., # angle (rad) – axis mode
    axis=[0., 0., 1.], # rotation axis – axis mode
    center=[0., 0., 0.] # center of rotation (m)
)
```

Common parameters:

Parameter	Type	Default	Description
<code>description</code>	str	<code>'Euler'</code>	Mode: <code>'Euler'</code> or <code>'axis'</code>
<code>center</code>	list[3]	<code>[0., 0., 0.]</code>	Center of rotation in absolute coordinates (m)

'Euler' mode parameters:

Parameter	Type	Default	Description
<code>phi</code>	float	<code>0.0</code>	1st Euler angle — rotation about Z (rad)
<code>theta</code>	float	<code>0.0</code>	2nd Euler angle — rotation about X (rad)
<code>psi</code>	float	<code>0.0</code>	3rd Euler angle — rotation about Z (rad)

The three rotations are applied successively: first `phi` about Z, then `theta` about X, then `psi` about Z.

'axis' mode parameters:

Parameter	Type	Default	Description
<code>alpha</code>	float	<code>0.0</code>	Rotation angle (rad)
<code>axis</code>	list[3]	<code>[0., 0., 1.]</code>	Direction vector of the rotation axis

Examples:

```
# 90° rotation about Z centered at the origin (axis mode)
bodies[0].rotate(description='axis', alpha=1.5708, axis=[0., 0., 1.], center=[0., 0., 0.])

# 45° Euler rotation in the XY plane
bodies[2].rotate(description='Euler', phi=0.7854, theta=0., psi=0., center=[1.0, 0.5, 0.])
```

Tip: for masonry walls, 'axis' rotation with `axis=[0,0,1]` allows you to create building corners. The angle is entered in degrees in the interface and automatically converted to radians.

3. `imposeDrivenDof` — Driven DOF

Imposes a **driven** degree of freedom on the nodes of a group of the avatar. The imposed value can be constant, sinusoidal with a ramp, or defined by a time-evolution file.

`pylmgc90` signature:

```
avatar.imposeDrivenDof(
    group='all',
    component=1,
    description='predefined',
    dofty='vlocy',
    ct=0.,
    amp=0.,
    omega=0.,
    phi=0.,
    rampi=1.,
    ramp=0.,
    evolutionFile=''
)
```

Parameters:

Parameter	Type	Default	Description
<code>group</code>	str	<code>'all'</code>	Node group of the avatar. <code>'all'</code> = all nodes. For deformable bodies: <code>'down'</code> , <code>'up'</code> , <code>'left'</code> , <code>'right'</code> , <code>'front'</code> , <code>'rear'</code> .
<code>component</code>	int or list	<code>1</code>	DOF component(s). See table below.
<code>description</code>	str	<code>'predefined'</code>	Time mode: <code>'predefined'</code> (analytical formula) or <code>'evolution'</code> (file).
<code>dofity</code>	str	<code>'vlocy'</code>	DOF type. See table below.
<code>ct</code>	float	<code>0.0</code>	Constant value.
<code>amp</code>	float	<code>0.0</code>	Cosine amplitude.
<code>omega</code>	float	<code>0.0</code>	Angular frequency (rad/s).
<code>phi</code>	float	<code>0.0</code>	Cosine phase (rad).

Parameter	Type	Default	Description
<code>rampi</code>	float	1.0	Initial value of the multiplicative ramp.
<code>ramp</code>	float	0.0	Ramp slope (s ⁻¹).
<code>evolutionFile</code>	str	' '	Path to a time-evolution file (*.evol). Used if <code>description='evolution'</code> .

Components (component)

Value	Meaning
1	Translation along X (or DOF 1)
2	Translation along Y (or DOF 2)
3	Translation along Z (or DOF 3) — 3D only
[1, 2]	X and Y simultaneously (locks the horizontal plane)
[1, 2, 3]	Full 3D locking

DOF Types (dofty)

Physics	dofty	Description
MECAx	'vlocy'	Imposed velocity (m/s)
MECAx	'force'	Imposed force (N)
THERx	'temp'	Imposed temperature (K or °C depending on convention)
THERx	'flux'	Imposed thermal flux (W/m²)
PORox	'vlocy'	Filtration velocity (m/s)
PORox	'force'	Imposed pressure
MULTI	'prim_'	Primal variable (pressure, temperature...)
MULTI	'dual_'	Dual variable (flux, force...)

General Formula ('predefined' mode)

The value imposed at time t is:

$$f(t) = [ct + \text{amp} \times \cos(\omega \times t + \phi)] \times \text{clamp}(\text{rampi} + \text{ramp} \times t)$$

where $\text{clamp}(x) = \text{sign}(x) \times \min(|x|, 1)$ — the ramp is bounded to 1 to avoid non-physical amplifications.

Scenario	Parameters	Behavior
Locked (zero velocity)	<code>ct=0, dofty='vlocy'</code>	Fixed nodes throughout the simulation
Constant displacement	<code>ct=v0, dofty='vlocy'</code>	Constant velocity <code>v0</code> m/s
Sinusoidal oscillation	<code>amp=A, omega=ω, phi=φ</code>	$v(t) = A \times \cos(\omega t + \phi)$
Gradual startup	<code>ct=v0, rampi=0, ramp=1/t_ramp</code>	Linear rise from 0 to <code>v0</code> over <code>t_ramp</code> seconds
Arbitrary evolution	<code>description='evolution', evolutionFile='myfile.evol'</code>	Value interpolated from the file

Examples

```
# Lock the base of an FE mesh (zero velocity in X and Y)
mesh.imposeDrivenDof(group='down', component=[1, 2], dofty='vlocy', ct=0.)

# Impose a constant vertical displacement of 0.001 m/s
bodies[3].imposeDrivenDof(group='all', component=2, dofty='vlocy', ct=0.001)

# Sinusoidal oscillation in X, frequency 5 Hz, amplitude 0.01 m/s
bodies[5].imposeDrivenDof(
group='all', component=1, dofty='vlocy',
ct=0., amp=0.01, omega=31.416, phi=0.
)

# Thermal loading from an evolution file
mesh.imposeDrivenDof(
group='up', component=1, dofty='temp',
description='evolution', evolutionFile='temperature.evol'
)
```

4. `imposeInitValue` — Initial Value

Imposes an **initial condition** (position or velocity) on the nodes of a group, only at time $t = 0$. Does not impose any constraint during the computation.

pylmgc90 signature:

```
avatar.imposeInitValue(
group='all',
component=1,
value=0.
)
```

Parameters:

Parameter	Type	Default	Description
<code>group</code>	str	'all'	Node group.
<code>component</code>	int or list	1	DOF component(s) (same convention as <code>imposeDrivenDof</code>).
<code>value</code>	float	0.0	Initial value to impose (m/s, m, K depending on the type).

Difference from `imposeDrivenDof`: `imposeInitValue` is only active at $t = 0$. The avatar is then free to move afterward. `imposeDrivenDof` maintains the constraint throughout the simulation.

Managing Operations

Creating an Operation

Fill in the form and click **Apply**. The operation is applied immediately to the pylmgc90 objects and saved in `state.operations`. The `operation_applied` signal is emitted, triggering a full refresh of the interface and the 3D viewer.

Editing an Operation

Double-click on an operation in the list, or select it and click **Edit**. The form is loaded with the operation's values. Modify it and click **Save** to update it (`update_dof_operation`). The operation is reapplied.

Deleting an Operation

Select it and click **Delete**. The operation is removed from `state.operations` (`remove_dof_operation`). The `operation_deleted` signal is emitted.

Warning: deleting an operation does not undo its effect on the avatars (translations and rotations already applied remain in place). To undo a translation, create an inverse translation.

Group Support

The group dropdown list automatically includes all groups defined in the project:

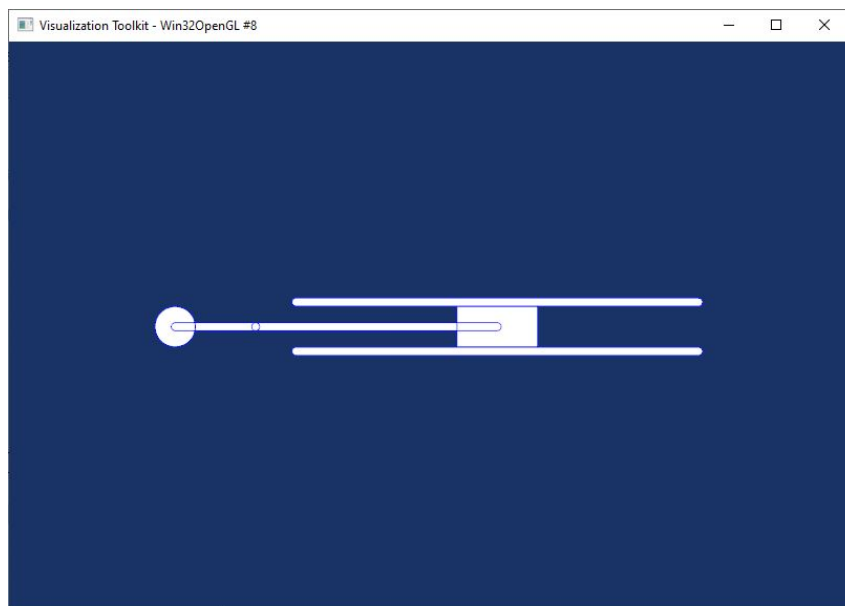
Source	Example Names
Geometric loops	<code>ma_ligne</code> , <code>anneau_particules</code>
<code>for</code> loops	name entered in the loop form
Granulometry	<code>granulo_box2d</code> , <code>granulo_couette2d</code>
Masonry	<code>mur_briques</code> , <code>mur_facade</code>
FE mesh DOF groups	<code>down</code> , <code>up</code> , <code>left</code> , <code>right</code> , <code>front</code> , <code>rear</code>

Applying an operation to a group executes the same operation on every avatar in the group:

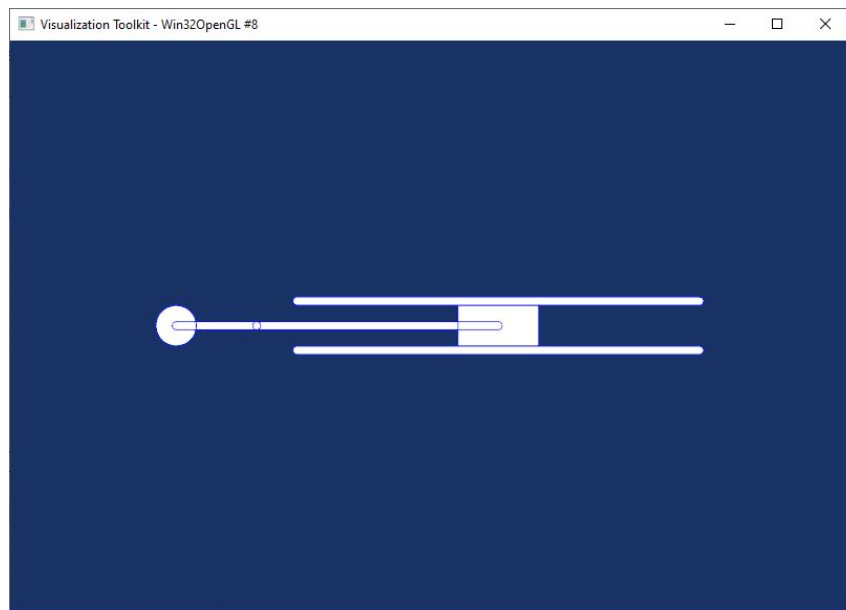
```
# Example generated script for a group:
for av in group_granulo_box2d:
    av.imposeDrivenDof(group='all', component=1, dofty='vlocy', ct=0.0)
```

Example — Slider-Crank (`slider_crank.lmgc90`)

The example provided among the project examples illustrates the application of four boundary conditions to the avatars of the slider-crank mechanism:



Operation	Type	Target	Typical Parameters
Crank rotation	<code>imposeDrivenDof</code>	Crank avatar	<code>component=3,</code> <code>dofty='vlocy', ct=ω</code>
Slider locking	<code>imposeDrivenDof</code>	Slider avatar	<code>component=2,</code> <code>dofty='vlocy', ct=0.</code>
Crank-rod pivot	<code>imposeInitValue</code>	Rod avatar	<code>component=[1,2],</code> <code>value=0.</code>
Initial position	<code>translate</code>	Crank avatar	<code>dx=x0, dy=y0</code>



captures/exemple_slider_crank.JPG

Important Notes

Order of operations: operations are applied in the order in which they are created. A translation followed by a rotation gives a different result than a rotation followed by the translation. The order in `state.operations` is respected both when applying the operations and in the generated script.

No undo: `translate` and `rotate` modify the physical coordinates of the `pymgc90` nodes. There is no Undo button — to reverse an operation, create an opposite one (inverse translation, rotation of opposite sign).

`imposeDrivenDof` vs `imposeInitValue`: use `imposeDrivenDof` with `ct=0` and `dofty='vlocy'` to **lock** a DOF throughout the simulation. Use `imposeInitValue` only to define an **initial condition** with no constraint over time.

Evolution files: the `.evol` file must be placed in the project's `DATBOX/` directory. It contains two columns: time (s) and value, separated by spaces or tabs. The number of lines must cover the entire simulation duration.

Components for deformable bodies: for FE meshes, `component` directly accepts the integers 1, 2, 3 (nodal displacement DOFs). For rigid bodies, `component=1` corresponds to translation in X, `component=2` in Y, `component=3` in Z. The rotation component is `component=4` (2D) or 4-6 (3D) depending on the physics.

Contact Laws

The **Contact** tab (**Ctrl+9**) allows you to create, modify, and delete pyImgc90 **contact laws** (**tact_behav**). Each law defines the mechanical behavior at the boundary between two bodies in contact. A law must be created here **before** it can be referenced in a visibility table (Visibility tab).

Operating Principle

A **contact law** (**ContactLaw**) is a pyImgc90 object created by `pre.tact_behav(name, law, ...)`. It is identified by a unique name within the project and characterizes the physics of the contact: friction, cohesion, stiffness, damage...

Generated pyImgc90 call:

```
law_IQS_CLB = pre.tact_behav(
name='IQS_CLB',
law='IQS_CLB',
fric=0.3
)
tacts.addBehav(law_IQS_CLB)
```

Tab Interface

The tab consists of three areas:

- **List of laws** (top): a tree with columns Name, Type, Friction, Properties. Laws **referenced** by at least one visibility table appear in **green**. Right-click to Edit, Delete, or display Information.
- **Creation / editing form** (middle): fields that adapt automatically to the chosen type.
- **Contextual help** (bottom): description and list of parameters for the selected type.

Buttons

Button	Mode	Action
■ Create Law	Creation	Validates the form and creates the law.
■ Save Changes	Editing	Updates the selected law.
■ Cancel	Editing	Returns to creation mode without saving.
■ Reset	All	Clears the form and restores default values.
—■ Edit Selection	All	Loads the selected law into the form.
■■ Delete Selection	All	Deletes the law (with reference check).

Form Fields

Field	Description
Name	Unique identifier of the law (20 characters max). Used in visibility tables. Default: law01 .
Category	Filters the Type combo box. See the 4 categories below.
Type	pylmgc90 law type. Specific fields are shown/hidden dynamically depending on the type.

The 4 Law Categories

Category 1 — Rigid / Rigid

Laws applicable between two rigid bodies (**RBDY2** / **RBDY3**).

IQS_CLB — Quasi-Static Inequality Coulomb (the most common)

Standard Coulomb law. Unilateral contact with dry Coulomb friction. Non-smooth Quasi-Static Inequality (IQS) approach.

Parameter	Interface Label	Description	Default
fric	Friction coefficient	Coulomb coefficient μ . The tangential force cannot exceed $\mu \times$ normal force.	0.3

Typical **fric** values:

Material	fric
Sand / glass beads	0.3 – 0.5
Concrete / rocks	0.5 – 0.8
Polished metals	0.1 – 0.3
Wood / plastic	0.3 – 0.6
Frictionless (sliding)	0.0

pylmgc90 call:

```
law = pre.tact_behav(name='loi1', law='IQS_CLB', fric=0.3)
```

IQS_CLB_g0 — Coulomb with Initial Gap

Identical to **IQS_CLB** but initializes the initial geometric gap between contactors to **g0=0**. Useful when bodies are already in contact at **t = 0** (with no initial interpenetration).

Parameter	Description	Default
fric	Coulomb friction coefficient.	0.3

Applications: pre-tightened mechanical assemblies, progressive compression, contacts with initial roughness.

IQS_DS_CLB — Coulomb with Static/Dynamic Stiffness

Discrete law with two distinct contact stiffnesses: a static stiffness (before sliding) and a dynamic one (during sliding).

Parameter	Interface Label	Description	Default
fric	Friction coefficient	Coulomb coefficient.	0.3
stfr	Static contact stiffness	Static normal stiffness (N/m).	1e8
dyfr	Dynamic contact stiffness	Dynamic normal stiffness (N/m).	1e8

Applications: braking systems ($\mu_{\text{static}} > \mu_{\text{dynamic}}$), tectonic plate sliding, mechanisms with self-induced vibrations.

IQS_MOHR_DS_CLB — Mohr-Coulomb with Cohesion

Mohr-Coulomb criterion including normal and tangential cohesion. Allows modeling materials with initial adhesion (cement, wet clay).

Parameter	Interface Label	Description	Default
fric	Friction coefficient	Coulomb coefficient μ .	0.3
stfr	Static stiffness	Stiffness before cohesion failure (N/m).	1e8
dyfr	Dynamic stiffness	Stiffness after failure (N/m).	1e8
cohn	Normal cohesion	Tensile adhesion force (Pa).	0.0
coht	Tangential cohesion	Additional tangential resistance (Pa).	0.0

Applications: geomechanics (clays, cohesive soils), wet granular materials, powders with van der Waals forces.

IQS_MAC_CZM — MAC Cohesive Zone (rigid/rigid)

Mohr-Coulomb-Allix-Corigliano cohesive zone model. Simulates progressive failure and delamination between rigid bodies.

Parameter	Interface Label	Description	Default
stfr	Static stiffness	Tangential stiffness before damage (N/m).	1e10
dyfr	Dynamic stiffness	Normal stiffness before damage (N/m).	1e10
cn	Normal strength	Normal tensile strength (Pa).	1e6
ct	Tangential strength	Shear strength (Pa).	1e6
b	Mixing parameter	Mode I/II coupling (0 = pure mode I, 1 = equipartition).	1.0
w	Fracture energy	Critical fracture energy (J/m ²).	0.01

Applications: bond failure, cracking, composite delamination.

RST_CLB — Restitution + Coulomb

Contact with a restitution coefficient (elastic or partially elastic impacts) and Coulomb friction.

Parameter	Description	Default
fric	Coulomb friction coefficient.	0.3

Applications: ball impacts, bouncing, mechanical shocks.

Category 2 — Rigid / Deformable (or Def / Def)

Laws applicable between a rigid body (**RBDY2/RBDY3**) and a deformable FE body (**MAILX**), or between two deformable bodies.

GAP_SGR_CLB — Gap Contact + Coulomb (rigid/deformable)

Standard law for rigid/deformable contact. Manages the initial gap between the rigid surface and the FE mesh.

Parameter	Description	Default
fric	Coulomb friction coefficient.	0.3

Applications: tool/workpiece contact, impact, indentation, FE compression.

GAP_SGR_CLB_g0 — GAP with g0 Initialization

Identical to **GAP_SGR_CLB** with the gap initialized to zero. Use if the bodies are initially in tangential contact.

Parameter	Description	Default
fric	Coulomb friction coefficient.	0.3

GAP_MOHR_DS_CLB — Mohr-Coulomb Gap (rigid/deformable)

Mohr-Coulomb criterion with gap management for rigid/deformable contact.

Parameter	Description	Default
fric	Coulomb coefficient.	0.3
stfr	Static stiffness (N/m).	1e8
dyfr	Dynamic stiffness (N/m).	1e8
cohn	Normal cohesion (Pa).	0.0
coht	Tangential cohesion (Pa).	0.0

MAC_CZM — MAC Cohesive Zone (rigid/deformable or def/def)

MAC cohesive zone model applied to rigid/deformable interfaces. Same parameters as **IQS_MAC_CZM**.

Parameter	Description	Default
stfr, dyfr	Stiffnesses (N/m).	1e10
cn, ct	Normal and tangential strengths (Pa).	1e6
b	Mixing parameter.	1.0
w	Fracture energy (J/m²).	0.01

MAL_CZM — MAL Cohesive Zone (rigid/deformable or def/def)

Variant of the CZM model based on the MAL (Mixed Augmented Lagrangian) formulation. Same parameters as `MAC_CZM`.

Category 3 — Point / Point

Laws applicable between point contactors (`PT2Dx`, `PT3Dx`, FE nodes). Model wire links or discrete rods.

`ELASTIC_WIRE` — Elastic Cable

Link active only in **tension** (cable inextensible in one direction). Simple cable model.

Parameter	Interface Label	Description	Default
<code>stiffness</code>	Axial stiffness	Cable stiffness EA (N).	<code>1e6</code>
<code>prestrain</code>	Pre-strain	Initial pre-tension (dimensionless, e.g. <code>0.01</code> = 1%).	<code>0.0</code>

Applications: suspension cables, anchor tie rods, fiber reinforcements.

`BRITTLE_ELASTIC_WIRE` — Brittle Elastic Cable

Elastic cable that fails in a brittle manner (without plastic deformation) when the stress exceeds `sigc`.

Parameter	Interface Label	Description	Default
<code>stiffness</code>	Axial stiffness	Cable stiffness EA (N).	<code>1e6</code>
<code>prestrain</code>	Pre-strain	Initial pre-tension.	<code>0.0</code>
<code>sigc</code>	Failure strength	Maximum stress before brittle failure (Pa).	<code>1e6</code>

Applications: brittle fibers, glass fibers, reinforcement failure.

`ELASTIC_ROD` — Elastic Rod

Rigid rod that can work in **both tension and compression** (unlike the cable). Linear rod model.

Parameter	Description	Default
<code>stiffness</code>	Axial stiffness EA (N).	<code>1e6</code>
<code>prestrain</code>	Initial pre-strain.	<code>0.0</code>

Applications: truss structures, point-to-point rigid links, stiffeners.

`VOIGT_ROD` — Voigt Visco-Elastic Rod

Rod with visco-elastic behavior (spring + damper in parallel — Kelvin-Voigt model).

Parameter	Interface Label	Description	Default
<code>stiffness</code>	Axial stiffness	Spring stiffness EA (N).	<code>1e6</code>
<code>viscosity</code>	Viscosity	Viscous damping coefficient (N·s).	<code>1e3</code>

Parameter	Interface Label	Description	Default
<code>prestrain</code>	Pre-strain	Initial pre-tension.	<code>0.0</code>

Applications: dampers, structures with viscous dissipation, visco-elastic soil models.

Category 4 — Any (any / any)

Universal laws applicable regardless of the contactor pair.

`COUPLED_DOF` — Degree-of-Freedom Coupling

Perfect coupling (zero velocity and displacement jump at the interface). Rigid kinematic link.

No parameters required.

Applications: perfect link between two bodies, DOF coupling in multi-body assemblies.

`NORMAL_COUPLED_DOF` — Normal Direction Coupling

Coupling only in the direction normal to the contact. Allows free tangential sliding.

No parameters required.

`ELASTIC_REPELL_CLB` — Elastic Repulsion + Coulomb

Soft contact via elastic penalization in the normal direction, with Coulomb friction. Regularization method (alternative to strict IQS laws).

Parameter	Interface Label	Description	Default
<code>fric</code>	Friction coefficient	Coulomb coefficient.	<code>0.3</code>
<code>Kn</code>	Normal stiffness	Normal penalty stiffness (N/m).	<code>1e8</code>

Applications: soft contacts, gentle penalization, models with controlled interpenetration.

Summary Table of the 18 Laws

Law	Category	Parameters	Main Use
<code>IQS_CLB</code>	Rig/Rig	<code>fric</code>	Standard granular contact
<code>IQS_CLB_g0</code>	Rig/Rig	<code>fric</code>	Contact with zero initial gap
<code>IQS_DS_CLB</code>	Rig/Rig	<code>fric, stfr, dyfr</code>	Static/dynamic friction
<code>IQS_MOHR_DS_CLB</code>	Rig/Rig	<code>fric, stfr, dyfr, cohn, coht</code>	Cohesive materials
<code>IQS_MAC_CZM</code>	Rig/Rig	<code>stfr, dyfr, cn, ct, b, w</code>	Failure, delamination
<code>RST_CLB</code>	Rig/Rig	<code>fric</code>	Impacts with restitution
<code>GAP_SGR_CLB</code>	Rig/Def	<code>fric</code>	FE tool/workpiece contact
<code>GAP_SGR_CLB_g0</code>	Rig/Def	<code>fric</code>	FE contact with zero gap

Law	Category	Parameters	Main Use
GAP_MOHR_DS_CLB	Rig/Def	fric, stfr, dyfr, cohn, coht	FE cohesive interface
MAC_CZM	Rig/Def	stfr, dyfr, cn, ct, b, w	FE cohesive zone
MAL_CZM	Rig/Def	stfr, dyfr, cn, ct, b, w	MAL cohesive zone
ELASTIC_WIRE	Pt/Pt	stiffness, prestrain	Cable (tension only)
BRITTLE_ELASTIC_WIRE	Pt/Pt	stiffness, prestrain, sigc	Brittle cable
ELASTIC_ROD	Pt/Pt	stiffness, prestrain	Elastic rod
VOIGT_ROD	Pt/Pt	stiffness, viscosity, prestrain	Visco-elastic rod
COUPLED_DOF	Any/Any	_(none)_	Perfect rigid link
NORMAL_COUPLED_DOF	Any/Any	_(none)_	Normal coupling only
ELASTIC_REPELL_CLB	Any/Any	fric, Kn	Soft penalty contact

Managing Laws

Creating a Law

Fill in the form and click **Create Law**. The law is created via `add_contact_law()`, which calls `LMGC90Bridge.create_contact_law() → pre.tact_behav(...) → tacts.addBehav(...)`. The `law_created` signal is emitted → `_refresh_all()`. Created laws are stored in `state.contact_laws` (a list of `ContactLaw`) and classified by `ContactLawType` (enum) and `CONTACT_LAW_CATEGORIES` (category → laws dict).

Editing a Law

Double-click in the list or click **Edit Selection**. The form switches to edit mode (the **Create** button is replaced by **Edit**). Modify the values and click **Save Changes** (`update_contact_law()`). Press **Cancel** to return to creation mode without saving.

Renaming: renaming a law automatically updates the `behavior_name` field of all visibility tables that reference it.

Deleting a Law

Select it and click **Delete Selection** (`remove_contact_law()`). If the law is referenced by one or more visibility tables, a warning lists the references and blocks the deletion. Delete the relevant visibility tables first.

Law Information

Right-click → **Information** displays a dialog box with the type, all the parameters, and the list of visibility tables that use it.

Visual Indicator in the List

Name Color	Meaning
Green	Law referenced by at least one visibility table
Black (normal)	Law not yet referenced

Important Notes

Consistency with the visibility table: the chosen law must be compatible with the body/contactor types of the visibility table. A `GAP_SGR_CLB` law will not work with an `RBDY2/DISKx` — `RBDY2/DISKx` (rigid/rigid) pair — use `IQS_CLB` in that case.

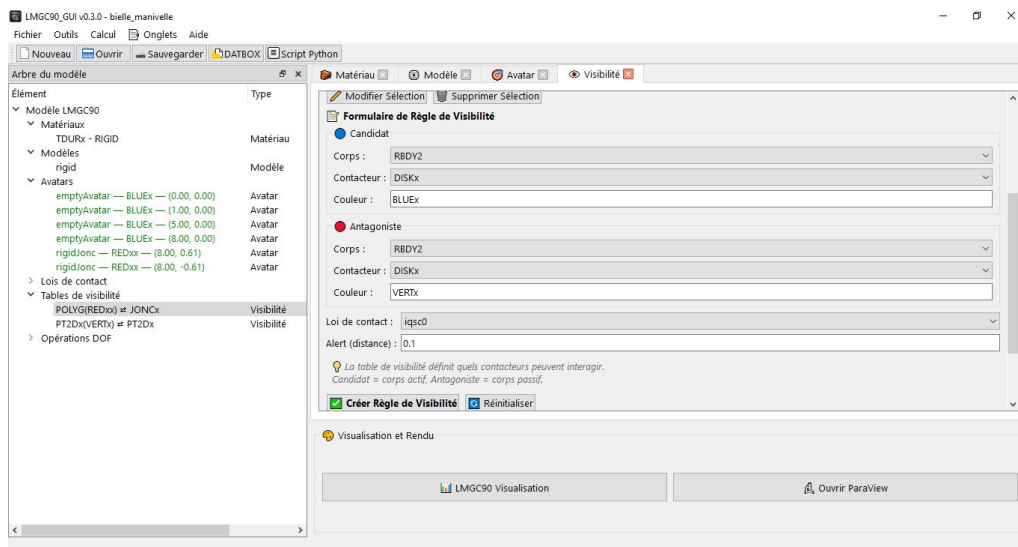
Consistency with the contact detector: the law type must match the detector enabled in the Computation tab. For example, `ELASTIC_WIRE` requires the `PT2Lx` (point/line) detector or similar.

Numerical values: the fields accept Python scientific notation (`1e8`, `1e-3`, `3.14e6`). Values are evaluated at the time the law is created.

Protected deletion: a law used by at least one visibility table cannot be deleted directly. This protection prevents reference errors in the project.

Visibility Tables

The **Visibility** tab (**Ctrl+9**) allows you to define pylmgc90's **visibility tables** ([see_table](#)): each rule declares which contactors can detect each other and with which contact law. This is a **mandatory** step before generating the DATBOX — without a visibility table, no contact interaction is computed.



captures/table_visibilite.JPG

Operating Principle

A **visibility table** ([see_table](#)) is the mechanism by which pylmgc90 knows which bodies must be tested for contact detection. It links a **candidate body** (which can be penetrated) and an **antagonist body** (never penetrated), specifying for each pair the color of the contactors involved and the contact law to apply.

Generated pylmgc90 call:

```
see_0 = pre.see_table(
CorpsCandidat='RBDY2',
candidat='DISKx',
colorCandidat='BLUEx',
CorpsAntagoniste='RBDY2',
antagoniste='DISKx',
colorAntagoniste='BLUEx',
behav=law_IQS_CLB,
alert=0.1
)
sees.addSeeTable(see_0)
```

The **VisibilityRule** structure in the project stores exactly these 8 parameters.

Tab Interface

The tab is divided into two areas:

- **List of rules** (top): a table of all visibility tables defined in the project, with candidate body, contactor, colors, antagonist body, law, and alert distance. Double-click to edit.
 - **Creation / editing form** (bottom): 8 fields corresponding to the parameters of `pre.see_table`.
-

Form Fields

Candidate Body

Field	Description	Common Values
Candidate body (<code>CorpsCandidat</code>)	Type of the body carrying the candidate contactor.	<code>RBDY2</code> (2D rigid body), <code>RBDY3</code> (3D rigid body), <code>MAILx</code> (FE deformable body)
Candidate contactor (<code>candidat</code>)	Shape of the contactor on the candidate body. Must match the shape declared in the Empty Avatar tab.	See the contactor table below
Candidate color (<code>colorCandidat</code>)	5-character LMGC90 color code of the candidate contactor. Must match exactly the color of the contactor declared on the body.	<code>BLUEx</code> , <code>REDxx</code> , <code>VERTx</code> , <code>GRAYx...</code>

Antagonist Body

Field	Description	Common Values
Antagonist body (<code>CorpsAntagoniste</code>)	Type of the body carrying the antagonist contactor.	<code>RBDY2</code> , <code>RBDY3</code> , <code>MAILx</code>
Antagonist contactor (<code>antagoniste</code>)	Shape of the antagonist contactor.	See the contactor table
Antagonist color (<code>colorAntagoniste</code>)	Color of the antagonist contactor. Must match the color of the contactor declared on the antagonist body.	<code>BLUEx</code> , <code>REDxx...</code>

Law and Alert

Field	Description	Default
Behavior (<code>behav</code>)	Name of the contact law to apply for this pair. Select from the dropdown list of laws defined in the Contact tab. The law must exist before creating the rule.	<code>_(first available law)_</code>
Alert distance (<code>alert</code>)	Detection distance (m). Two contactors less than <code>alert</code> apart are considered potentially in contact and are tested. Range: 0.001 to 10.0 m.	<code>0.1 m</code>

Available Body Types

Type	Description	Usage
RBDY2	2D rigid body (<code>rigidDisk</code> , <code>rigidJonc</code> , <code>rigidPolygon...</code>)	2D simulations — particles, masonry, mechanisms
RBDY3	3D rigid body (<code>rigidSphere</code> , <code>rigidPolyhedron...</code>)	3D simulations — spheres, polyhedra
MAILx	FE deformable body (<code>MESH_DEFORMABLE</code>)	Rigid/deformable or deformable/deformable interactions

Available Contactor Types

2D Contactors (for RBDY2)

Contactor	Description	pylmgc90 Parameter
DISKx	Circular disk	<code>r</code> (radius)
XKSID	Oriented disk (anti-disk)	<code>r</code>
JONCx	Elliptical jonc / capsule	<code>axe1</code> , <code>axe2</code>
POLYG	2D convex polygon	<code>nb_vertices</code> , <code>vertices</code>
CLxxx	Contact line (masonry)	length determined by the brick
PT2Dx	2D contact point	—

3D Contactors (for RBDY3)

Contactor	Description	pylmgc90 Parameter
SPHER	Sphere	<code>r</code> (radius)
PLANx	Semi-infinite plane	normal vector
CYLND	Cylinder	<code>r</code> , length
DNLYC	Half-cylinder	<code>r</code> , length
POLYR	3D convex polyhedron	<code>vertices</code>
PT3Dx	3D contact point	—

Contactors for Deformable Bodies (for MAILx)

Contactor	Description
CLxxx	Contact line on an FE edge (2D)
ALpxx	Polygon line (2D FE interface)

Colors and Matching

The color (`colorCandidat`, `colorAntagoniste`) is the main **filtering criterion**. Only contactors carrying exactly the color declared in the visibility table are tested for contact. This allows you to create selective rules:

- Make only blue particles interact with each other: `colorCandidat='BLUEx'`, `colorAntagoniste='BLUEx'`
- Make red particles interact with a gray wall: `colorCandidat='REDxx'`, `colorAntagoniste='GRAYx'`
- Contact between disks and a fixed plane: `RBDY2/DISKx/BLUEx ↔ RBDY3/PLANx/GRAYx`

Common error: if no interaction is detected during the computation, check that the colors in the visibility table match **exactly** (5 characters, case-sensitive) the colors declared on the avatars' contactors.

Alert Distance (`alert`)

The `alert` distance determines the contact search zone. Two contactors are placed on the candidate list for the computation if the distance between them is less than `alert`.

Situation	Recommended Value
Granular particles ($r \approx 0.05$ to 0.15 m)	<code>0.1</code> to <code>0.3</code> m
Masonry (standard bricks)	<code>0.02</code> to <code>0.05</code> m
Large structures	5 to 10% of the maximum radius
FE deformable bodies	Characteristic size of an element

Too small: some real contacts are not detected → unhandled interpenetration (very minor). **Too large:** too many candidate pairs → computation slowed down unnecessarily.

Managing Rules

Creating a Rule

Fill in the form and click **Create**. The rule will be created as follows:

1. Checks that the referenced contact law exists in `_pylmgc_laws`.
2. Creates the `pylmgc90` `see_table` object via `LMGC90Bridge.create_visibility_rule()`.
3. Adds it to the `_visibility_container` (the `sees` collection).
4. Saves it in `state.visibility_rules`.
5. Emits the `rule_created` signal → `_refresh_all()`.

Editing a Rule

Double-click in the list, or select it and click **Edit**. After confirmation, `update_visibility_rule()`:

1. **Fully** rebuilds the `_visibility_container` (`pylmgc90` limitation — no in-place modification of an existing `see_table`).
2. Reinserts all the rules with the new values.
3. Emits the `rule_updated` signal → `_refresh_all()`.

Deleting a Rule

Select it and click **Delete**. The rule is removed from `state.visibility_rules` via `remove_visibility_rule()`. The `rule_deleted` signal is emitted.

Cascading update: renaming a contact law in the Contact tab automatically updates the `behavior_name` field of all visibility rules that reference it.

Common Rules by Simulation Type

2D Granulometry — Rigid Disks

```
CandidateBody : RBDY2 candidate : DISKx candidateColor : BLUEx
AntagonistBody : RBDY2 antagonist : DISKx antagonistColor : BLUEx
behav : IQS_CLB alert : 0.1
```

2D Masonry — Rigid Bricks

```
CandidateBody : RBDY2 candidate : CLxxx candidateColor : BLUEx
AntagonistBody : RBDY2 antagonist : CLxxx antagonistColor : BLUEx
behav : IQS_CLB alert : 0.02
```

3D Granulometry — Rigid Spheres

```
CandidateBody : RBDY3 candidate : SPHER candidateColor : BLUEx
AntagonistBody : RBDY3 antagonist : SPHER antagonistColor : BLUEx
behav : IQS_CLB alert : 0.1
```

Rigid / Deformable — Disk on an FE Mesh

```
CandidateBody : RBDY2 candidate : DISKx candidateColor : BLUEx
AntagonistBody : MAILx antagonist : CLxxx antagonistColor : VERTx
behav : GAP_SGR_CLB alert : 0.05
```

Different Body Types — Disks and Jongs

```
CandidateBody : RBDY2 candidate : DISKx candidateColor : BLUEx
AntagonistBody : RBDY2 antagonist : JONCx antagonistColor : REDxx
behav : IQS_CLB alert : 0.15
```

Connection with the Project Wizard

The **Project Wizard** (**Ctrl+Shift+N**) offers a **Visibility** page that automatically pre-fills the visibility table based on the choices made in the previous steps:

- **Body:** **RBDY2** (2D) or **RBDY3** (3D) depending on the project's dimension.
- **Contact:** **DISKx** (2D) or **SPHER** (3D).
- **Colors:** automatically synchronized with the color of the avatar created at the Avatar step.
- **Law:** the law created at the Contact step.

For more complex configurations (multiple contactor types, multiple laws), use the Visibility tab directly after finishing the wizard.

Important Notes

The law must exist before the rule: `add_visibility_rule()` checks that the referenced law is present in `_pylmgc_laws`. If the law is deleted from the Contact tab after the rule is created, the rule becomes invalid and script generation will fail. The Contact tab warns if you attempt to delete a law used by a visibility rule.

Full reconstruction upon editing: `pylmgc90` does not allow modifying an existing `see_table`. The slightest modification to a rule triggers a full reconstruction of the `sees` container with all the rules. This behavior is transparent to the user but can be slow if the project contains many rules.

Multiplicity of rules: a project can have as many visibility rules as needed. For an assembly with several populations of particles of different colors (e.g. blue, red, green), create one rule per pair of colors that must interact. Populations of different colors with no common rule will not see each other.

Contact detector vs. visibility table: the visibility table declares *who* can interact. The Computation tab (**Ctrl+8**) configures the *contact detectors* (**DKDKx**, **SPSPx...**) that determine *how* the contact is resolved. The two must be consistent — if a rule declares **DISKx/DISKx**, the **DKDKx** detector must be enabled in the Computation tab.

Masonry Wizard

The **Masonry Wizard** guides you step by step through the creation of masonry structures in LMGC90_GUI. It automatically generates stacks of rigid bricks via the pylmgc90 API (**brick2D**, **brick3D**, **paneresse_simple**, **paneresse_double**), managing the generation functions, joints, transformations, and avatar groups.

Launching the Wizard

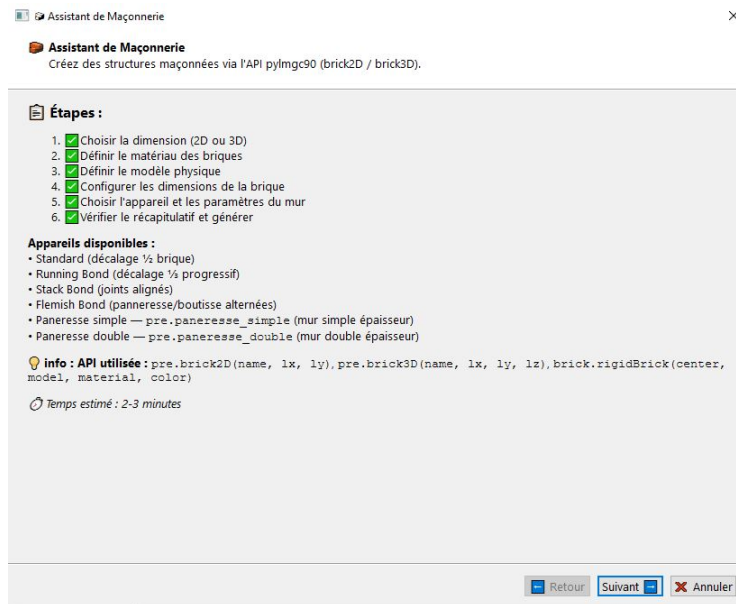
Method	Action
Menu	Wizards → Masonry Wizard...
Keyboard shortcut	Ctrl+Shift+M

Cancellation possible at any time via the **■ Cancel** button, with no changes made to the project.

Overview of Steps

The wizard consists of **8 pages** navigated sequentially.

Page	Title	Description
0	Introduction	Presentation of the bonding patterns and the API used
1	Dimension	2D (brick2D) or 3D (brick3D)
2	Material	Create or reuse a RIGID -type material
3	Model	Create or reuse a Rxx2D / Rxx3D model
4	Brick dimensions	lx , ly , lz , and brick name
5	Masonry bond	Bond type, wall dimensions, position, color, group
6	Transformations	Translation, rotation, offset copy after generation
7	Summary	Verification before generation



captures/assistant_maçon_page1.JPG

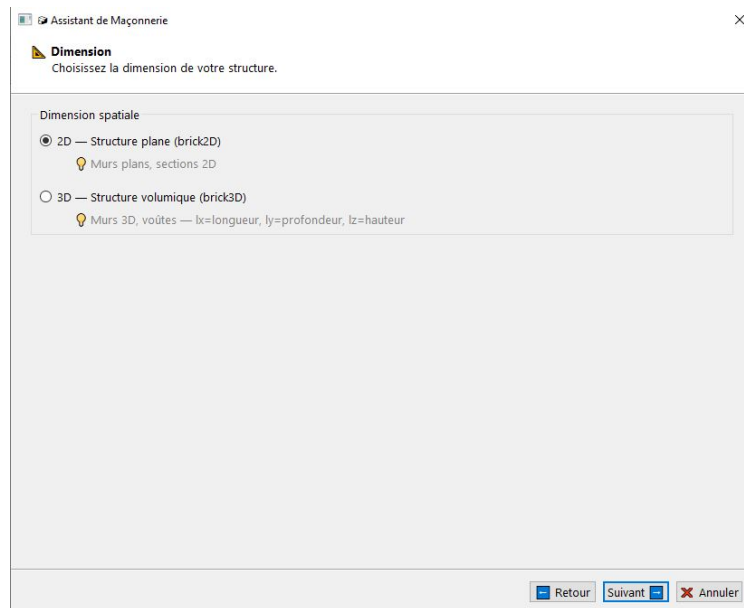
pylmgc90 API used:

- `pre.brick2D(name, lx, ly)` — 2D brick (length × height)
- `pre.brick3D(name, lx, ly, lz)` — 3D brick (length × depth × height)
- `brick.rigidBrick(center, model, material, color)` — pylmgc90 avatar from the brick
- `pre.panerese_simple(brick_ref, disposition)` — single-thickness wall managed by pylmgc90
- `pre.panerese_double(brick_ref, disposition)` — double-thickness wall managed by pylmgc90

Page 1 — Dimension

Choice	pylmgc90 API	Description
2D — Flat structure	<code>pre.brick2D(name, lx, ly)</code>	Flat walls, 2D sections. ly is the height of the brick.
3D — Volumetric structure	<code>pre.brick3D(name, lx, ly, lz)</code>	3D walls, vaults. ly is the depth , lz is the height .

The 2D value is selected by default.



captures/assistant_maçon_page2.JPG

Effect on subsequent steps: the dimension determines the label of **ly** (height in 2D / depth in 3D), the visibility of **lz** and the Z offset, the model element (**Rxx2D** or **Rxx3D**), and the available rotation axes.

Page 2 — Brick Material

Two modes available:

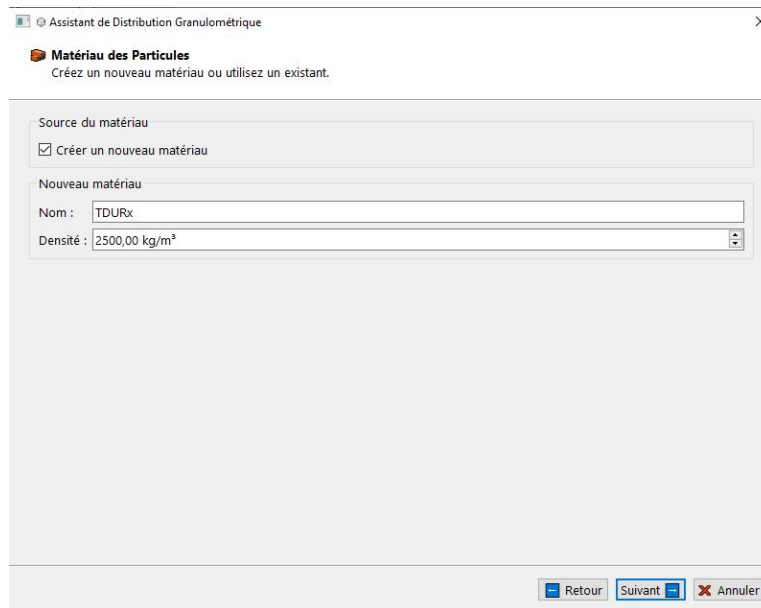
Mode A — Create a New Material _(checked by default)_

Field	Description	Default Value
Name	Material identifier. 5 characters maximum (LMGC90 constraint).	brick
Density	Density (kg/m³). Range: 100 to 20,000.	1800 kg/m³ (typical masonry)

The type is always **RIGID** — bricks are rigid bodies. There is no need to specify elastic properties.

Mode B — Use an Existing Material

Dropdown list of all materials already defined in the Material tab. All types are accepted, but **RIGID** is recommended for bricks.



captures/assistant_granulo_page3.JPG

Validation: the Next button is disabled if the name is empty (mode A) or if no valid material is available (mode B).

Page 3 — Physical Model

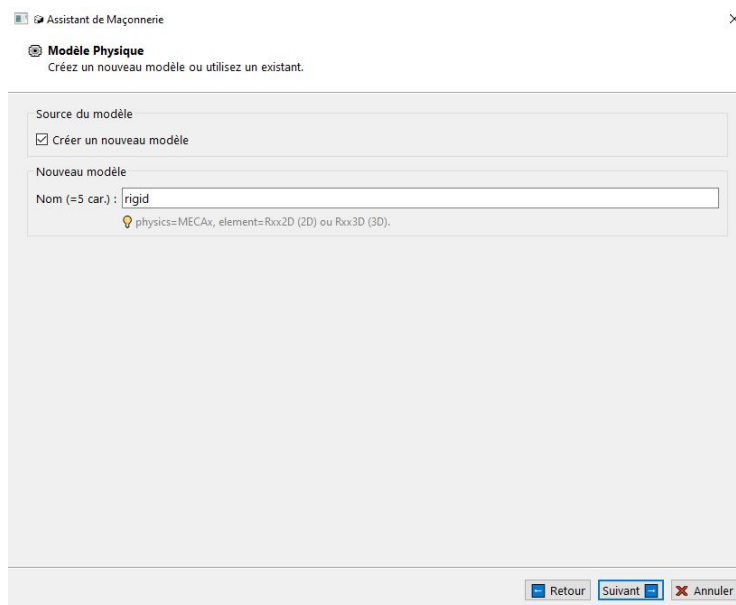
Two modes available:

Mode A — Create a New Model (checked by default)

Field	Description	Default Value
Name	Model identifier. 5 characters maximum.	rigid
Physics	Always MECAx for rigid bricks.	MECAx (automatic)
Element	Adapted automatically to the dimension.	Rxx2D (2D) or Rxx3D (3D)

Mode B — Use an Existing Model

Dropdown list of the models defined in the Model tab.



captures/assistant_maçon_page4.JPG

Validation: the Next button is disabled if the name is empty (mode A) or if no valid model is available (mode B).

Page 4 — Brick Dimensions

Defines the geometry of the reference brick passed to `brick2D()` or `brick3D()`.

Field	Description	2D	3D	Default Value
Brick name	Internal pylmgc90 identifier (8 characters max). Used as the first argument of <code>brick2D</code> / <code>brick3D</code> .	■	■	std
lx — length	Length of the brick along the X direction (m).	■	■	0.200 m
ly — height (2D)	In 2D: height of the brick. <code>ly</code> argument of <code>brick2D</code> .	■	—	0.065 m
ly — depth (3D)	In 3D: depth (wall thickness). <code>ly</code> argument of <code>brick3D</code> .	—	■	0.100 m
lz — height (3D)	3D only: height of the brick. <code>lz</code> argument of <code>brick3D</code> .	—	■	0.065 m

Standard French brick: $lx = 0.20\text{ m} \cdot ly = 0.10\text{ m} \cdot lz = 0.065\text{ m}$ (NF EN 771-1 format). **pylmgc90 convention:** `brick2D(name, lx, ly)` with `ly = height`. `brick3D(name, lx, ly, lz)` with `ly = depth`, `lz = height`.

captures/assistant_maçon_page5.JPG

Page 5 — Masonry Bond

Main configuration page. Defines the stacking type, wall dimensions, position, color, and avatar group.

Available Bond Types

Standard — Half-Brick Offset

Each odd row is offset by half a brick ($lx/2$) relative to the even row. The most common bond, very resistant to vertical loads.

```
Row 2: [=====] [=====] [=====]
Row 1: [=====] [=====] [=====]
Row 0: [=====] [=====] [=====]
```

Center calculation:

```
cx = offset_x + col × (lx + joint) + (lx/2 if odd row) + lx/2
cy = offset_y + row × (ly + joint) + ly/2
```

Running Bond — Progressive Third Offset

The offset increases by a third of a brick ($lx/3$) at each row, then returns to zero every three rows.

```
Row 3: [=====] [=====] [=====]
Row 2: [=====] [=====] [=====]
Row 1: [=====] [=====] [=====]
Row 0: [=====] [=====] [=====]
```

Center calculation:

```
cx = offset_x + col × (lx + joint) + (row % 3) × (lx/3) + lx/2
```

Stack Bond — Aligned Joints

All vertical joints are perfectly aligned. No offset between rows. Aesthetically pleasing, but mechanically less resistant (no interlocking between rows).

```
Row 2: [=====][=====][=====]
Row 1: [=====][=====][=====]
Row 0: [=====][=====][=====]
```

Center calculation:

```
cx = offset_x + col * (lx + joint) + lx/2
cy = offset_y + row * (ly + joint) + ly/2
```

Flemish Bond — Alternating Stretcher / Header

Each row alternates bricks laid as stretchers (length lx) and bricks laid as headers (length $lx/2$). The pattern reverses from one row to the next.

```
Row 1: [1/2][==lx==][1/2][==lx==]
Row 0: [==lx==][1/2][==lx==][1/2]
```

Calculation: if $(row + col) \% 2 == 0 \rightarrow$ stretcher ($brick_lx = lx$), otherwise header ($brick_lx = lx/2$). The X position is calculated using a cumulative cursor.

Functions Available Only in 3D

Single Stretcher Bond — `pre.paneresse_simple`

Single-thickness wall generated by the pylmgc90 API, with automatic handling of half-bricks at the ends, joints, and height.

Options specific to this bond:

Option	Description	Values
Disposition	Orientation of the bricks within the row.	<code>paneresse</code> (stretcher) · <code>boutisse</code> (header) · <code>chant</code> (on edge)
First brick	Brick type at the start of the first row.	<code>1</code> · <code>1/2</code> · <code>1/4</code> · <code>3/4</code>
Sizing	Method for defining the width of the wall.	Number of bricks · Total length (m)
Total length	Visible only if Sizing = "Total length". Calls <code>setFirstRowByLength()</code> .	E.g.: <code>3.0 m</code>
No half-bricks	If checked, calls <code>buildRigidWallWith outHalfBricks()</code> instead of <code>buildRigidWall()</code> .	checkbox

Dispositions:

Disposition	Description
<code>paneresse</code> (stretcher)	Bricks laid lengthwise (visible face = long side).
<code>boutisse</code> (header)	Bricks laid crosswise (visible face = short side, $lx/2$).
<code>chant</code> (on edge)	Bricks laid on edge (height = width of the brick).

Double Stretcher Bond — `pre.paneresse_double`

Double-thickness wall (two rows of bricks side by side). All the options of the single stretcher bond are available. The `pylmgc90` API automatically manages the offset between the two thicknesses.

Wall Dimensions

Field	Description	Default
Number of rows	Height of the wall in number of brick rows. Range: 1 to 200.	10
Number of columns	Width of the wall in number of bricks per row. Range: 1 to 200.	15
Joint thickness	Thickness of the mortar between bricks (m). Range: 0 to 0.05.	0.010 m

Automatic estimate in the summary: $Wall\ width \approx nb_columns \times (lx + joint)$ $Wall\ height \approx nb_rows \times (lz\ or\ ly + joint)$

Warning beyond 5,000 bricks: a confirmation dialog requests validation before continuing, since generation can take several seconds.

Initial Position

Coordinates of the **bottom-left corner** of the wall (in meters). The centers of the bricks are calculated from this point.

Field	Description	Default
Offset X	X coordinate of the bottom-left corner.	0.0 m
Offset Y	Y coordinate of the bottom-left corner.	0.0 m
Offset Z	Z coordinate of the wall (3D only).	0.0 m

Options

Option	Description	Default
LMGC90 Color	5-character color code.	BLUEx
Store in a group	If checked, saves all generated avatars in a named group (<code>state.avatar_groups</code>). The group is also saved in <code>masonry_patterns</code> for script reconstruction.	checked
Group name	Avatar group identifier.	mur_briques

captures/assistant_maçon_page6.JPG

Page 6 — Post-Generation Transformations

Transformations are applied **after** the generation of all the bricks, to the entire set of pylmgc90 bodies in the current session. They correspond to the pylmgc90 calls `bodies.translate()`, `bodies.rotate()`, and `copy.deepcopy(bodies)`.

All transformations are **disabled by default** (unchecked boxes). The fields are grayed out as long as the corresponding box is not checked.

Translation — `bodies.translate(dx, dy, dz)`

Moves the entire wall by a vector (dx, dy, dz).

Field	Description	Default
Enable translation	checkbox	unchecked
dx	Displacement along X (m).	0.0 m
dy	Displacement along Y (m).	0.0 m
dz	Displacement along Z (m). 3D only.	0.0 m

Generated pylmgc90 call:

```
# In 2D
bodies.translate(dx=dx, dy=dy)
# In 3D
bodies.translate(dx=dx, dy=dy, dz=dz)
```

Rotation — `bodies.rotate(description='axis', center, axis, alpha)`

Rotates the entire wall about an axis passing through a given center. The angle is entered in degrees and automatically converted to radians.

Field	Description	Values	Default
Enable rotation	checkbox	—	unchecked
Center x	X coordinate of the center of rotation (m).	—	0.0 m
Center y	Y coordinate of the center of rotation (m).	—	0.0 m
Center z	Z coordinate of the center of rotation (m). 3D only.	—	0.0 m
Axis	Axis of rotation. In 2D, only Z is relevant (set automatically).	Z · X · Y	Z
Angle α	Rotation angle in degrees. Converted to radians before the pylmgc90 call.	−360° to +360°	90.0°

Generated pylmgc90 call:

```
import math, numpy as np
alpha = math.radians(90.0)
axis = [0., 0., 1.] # Z axis
center = np.array([cx, cy, cz])
bodies.rotate(description='axis', center=center, axis=axis, alpha=alpha)
```

In 2D: the axis is always Z (rotation in the XY plane). The Axis field is grayed out and fixed to Z.

Offset Copy — `copy.deepcopy(bodies) + translate`

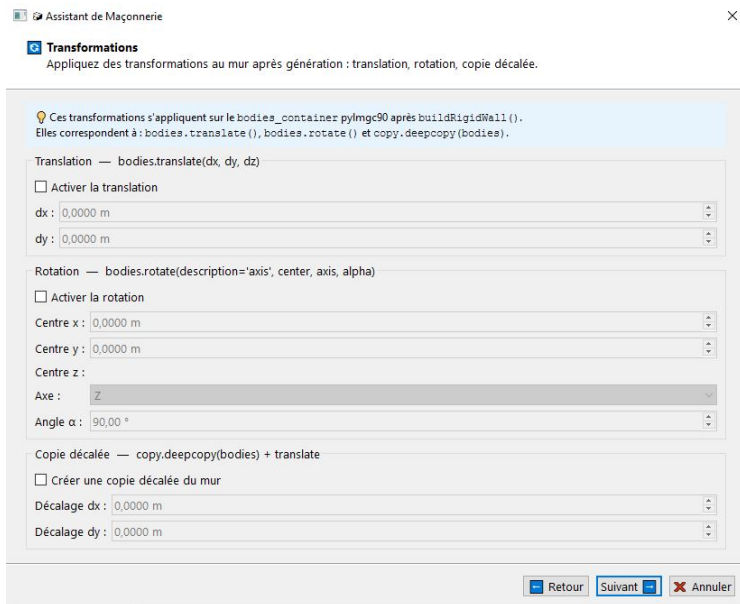
Duplicates the complete wall (after any translation and rotation) and applies an offset to it. This creates a second, independent wall in the project.

Field	Description	Default
Create an offset copy	checkbox	unchecked
Offset dx	X offset of the copy (m).	0.0 m
Offset dy	Y offset of the copy (m).	0.0 m
Offset dz	Z offset of the copy (m). 3D only.	0.0 m

Generated pylmgc90 call:

```
import copy
bodies_copy = copy.deepcopy(bodies)
bodies_copy.translate(dx=dx_copy, dy=dy_copy) # or dz in 3D
```

Typical use: creating two parallel walls at once (dx = room thickness), or, combined with rotation, creating a building corner.



captures/assistant_maçon_page7.JPG

Page 7 — Summary

Displays a complete HTML table before generation, including:

Section	Information
Dimension	2D or 3D
pymgc90 API	<code>brick2D(...)</code> or <code>brick3D(...)</code> call with the exact values
Material	Name and density (new) or existing name
Model	Name (new) or existing name
Brick dimensions	lx, ly, lz
Bond	Name of the selected pattern
Rows × columns	Estimated total number of bricks
Joint thickness	Value in meters
Estimated wall size	Width × Height calculated from the parameters
Position	Offset X, Y (and Z in 3D)
Color	LMGC90 color code
Group	Group name (if enabled)
Translation	dx, dy, dz (if enabled)
Rotation	Axis, angle α , center (if enabled)
Offset copy	dx, dy, dz of the copy (if enabled)
Stretcher bond options	Disposition, first brick, sizing mode, no half-bricks (if a stretcher bond)

Automatic warning: if the total number of bricks exceeds 1,000, an orange message indicates that generation may be slow.

Click **■ Generate** to create the structure. A confirmation message indicates the number of bricks created.

Generation Result

At the end of the wizard, the following elements are created in the project:

Element	Description
Material	Added to the Material tab (if created). Type RIGID , configured density.
Model	Added to the Model tab (if created). MECAx + Rxx2D or Rxx3D .
Brick avatars	One EMPTY_AVATAR avatar per brick, with wall_params stored for reconstruction. Configured color.
Group	Group named in state.avatar_groups , grouping together all the indices of the generated avatars (if the option is enabled).
masonry_patterns	Dictionary saved in the state for reconstruction of the generation script by ScriptGenerator .

Script reconstruction: the complete parameters of the pattern (bond, dimensions, joints, transformations, etc.) are saved in **state.masonry_patterns[group_name]**. The script generator (**script_generator.py**) uses this data to faithfully reproduce the **pylmgc90** generation upon export.

Usage Examples

Simple 2D Standard Wall

```
Dimension : 2D
Material : brick - RIGID - 1800 kg/m³
Model : rigid - MECAX - Rxx2D
Brick : std - lx=0.200 m, ly=0.065 m
Bond : Standard
Rows × cols. : 10 × 15 (150 bricks)
Joint : 0.010 m
Position : (0.0, 0.0)
Color : BLUEx
Group : mur_nord
Transformations: (none)
```

Single Stretcher Bond Wall with Fixed Length

```
Dimension : 3D
Material : stone - RIGID - 2200 kg/m³
Model : rigid - MECAX - Rxx3D
Brick : std - lx=0.200 m, ly=0.100 m, lz=0.065 m
Bond : Single stretcher bond (pylmgc90)
Disposition : paneresse (stretcher)
First brick : 1/2
Sizing : Total length - 4.000 m
No half-bricks : No
Rows : 15
Joint : 0.010 m
Group : mur_facade
```

Two Parallel Walls via Offset Copy

```
Bond : Standard – 8 rows × 12 columns
Translation : disabled
Rotation : disabled
Offset copy : enabled – dx=0.0, dy=3.5 m (room width)
```

Result: two identical walls 3.5 m apart, created in a single run of the wizard.

Important Notes

Brick name limited to 8 characters: the Brick name field accepts up to 8 characters, but LMGC90 may truncate identifiers. Use short names (`std`, `half`, `custom`).

Performance: beyond 5,000 bricks, generation and the interface can slow down significantly. For large assemblies (> 10,000 bricks), prefer direct generation from a Python script.

Default color: if the Color field is left empty, the value `BLUEx` is used automatically.

Transformations and centers: after a translation or rotation, the centers of the avatars in `state.avatars` are updated from the actual coordinates of the pylmgc90 nodes (`body.nodes[1].coord`). The avatars of the offset copy are added afterward with their own calculated center.

Reusability: the wizard can be launched multiple times on the same project. Each run adds a new group of bricks without erasing the previous ones, provided different element and group names are used.

Corps déformables (EF)

Deformable Body Wizard — FE Mesh

The **Deformable Body Wizard** guides you step by step through the creation of a deformable body meshed with finite elements in LMGC90_GUI. It automatically generates the mesh, creates the associated material and model, applies the boundary conditions, then stores the **MESH_DEFORMABLE**-type avatar directly in the project.

The mesh is built via the pylmgc90 functions `pre.buildMesh2D()` or `pre.buildMeshH8()` for structured geometries, and via **gmsh** for curved geometries (disk, sphere, cylinder). In all cases, the raw mesh is converted into a pylmgc90 avatar via `pre.buildMeshedAvatar()`, which produces a body of type **MAILx** (deformable mesh). Complex geometries can also be imported from an external file via `pre.readMesh()`.

Launching the Wizard

Method	Action
Menu	Wizards → Deformable Wizard...
Keyboard shortcut	Ctrl+Shift+D

Cancellation: clicking **Cancel** at any step closes the wizard without modifying the project. The project state is fully restored.

Overview of Steps

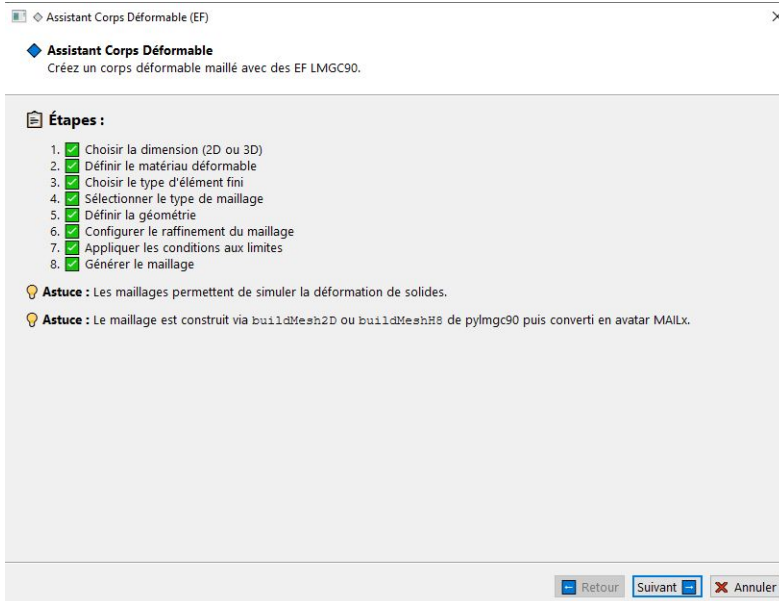
The wizard consists of **8 pages** navigated sequentially. The **Back** and **Next** buttons allow free navigation.

Page	Title	Description
0	Introduction	Presentation of the wizard and the steps
1	Dimension	2D or 3D
2	Material	Create or reuse an elastic material
3	FE Model	Create or reuse a finite element model
4	Geometry	Shape and dimensions of the body
5	Refinement	Mesh fineness
6	Boundary Conditions (DOF)	Dirichlet and loading conditions
7	Summary	Verification before generation

Page 0 — Introduction

Presentation of the 8 upcoming steps. No input required. Click **Next** to begin.

The generated mesh is a **MESH_DEFORMABLE** avatar of color **CYANx**, automatically added to the project's avatar list. Its reconstruction parameters are saved in **mesh_params** to allow the project to be reloaded.



captures/assistant_defor_page1.JPG

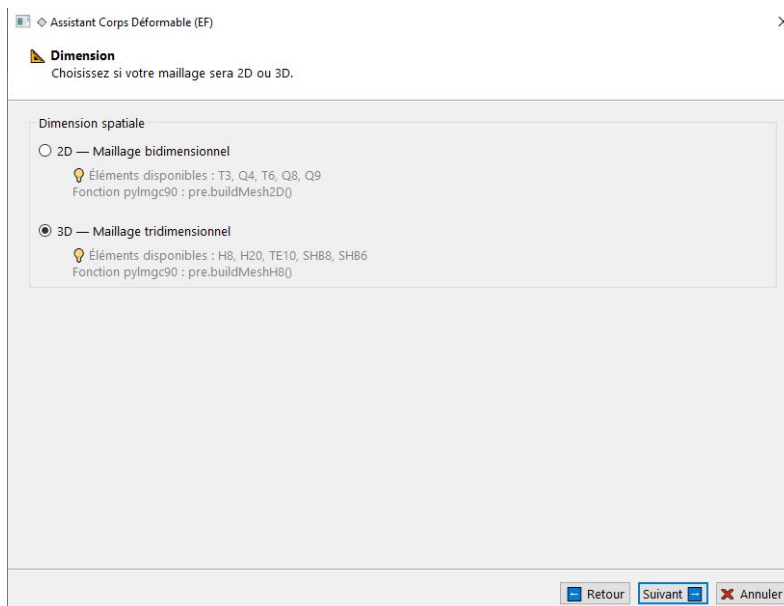
Page 1 — Dimension

Choose between two mutually exclusive options:

Choice	pylmgc90 Function Used	Available Geometries
2D	<code>pre.buildMesh2D()</code> + gmsh	Rectangle, Disk, External file
3D	<code>pre.buildMeshH8()</code> + gmsh	Box (H8), Sphere, Cylinder, External file

The **2D** value is selected by default.

Effect on subsequent steps: the dimension determines the element types available on the Model page, the shapes available on the Geometry page, the refinement parameters, and the surface groups (down/up/left/right in 2D, with front/rear added in 3D).



captures/assistant_defor_page2.JPG

Page 2 — Deformable Material

This page offers two modes:

Mode A — Use an Existing Material (if elastic materials exist in the project)

Only materials of elastic types are listed: **ELAS**, **ELAS_DILA**, **VISCO_ELAS**, **ELAS_PLAS**, **THERMO_ELAS**, **PORO_ELAS**. **RIGID** materials are not offered since they are incompatible with a deformable body.

Mode B — Create a New Material

Check **Create a new material instead** to display the form. Conditional fields appear or are hidden automatically depending on the chosen type.

Fields Common to All Elastic Types

Field	Description	Default Value
Name	Material identifier. 5 characters maximum.	ELAS1
Type	Type of mechanical behavior. See table below.	ELAS
Density	Density (kg/m ³). Range: 10 to 25,000.	2700 kg/m³ (aluminum)
Young's modulus	Stiffness of the material (Pa). Range: 10 ³ to 10 ¹² .	70 × 10⁹ Pa
Poisson's ratio (ν)	Lateral compressibility (dimensionless). Range: 0 to 0.4999.	0.3

Available Material Types and Conditional Fields

Type	Additional Fields	Description
ELAS	<u>(none)</u>	Standard linear elasticity.
ELAS_DILA	Thermal dilation (K ⁻¹ , default 1e-5), T_ref_meca (°C, default 20.0)	Elasticity with one-sided thermal coupling.
VISCO_ELAS	Viscous Young (Pa, default 1.17 × 10⁹), Viscous Poisson (default 0.35)	Kelvin-Voigt visco-elasticity.
ELAS_PLAS	Yield strength iso_hard (Pa, default 4 × 10⁹), Hardening modulus isoh_coeff (Pa, default 10⁹)	J2 elasto-plasticity with linear isotropic hardening, Von-Mises criterion.
THERMO_ELAS	<u>(young and nu set to 0 — defined by the model)</u>	Coupled thermo-elasticity. conductivity='field' and specific_capacity='field' .
PORO_ELAS	<u>(young and nu set to 0 — defined by the model)</u>	Poro-elasticity following Biot's theory. hydro_cpl=0.0 by default.

Automatically generated properties: **elas='standard'** and **anisotropy='isotropic'** are always added. For **ELAS_PLAS**, the criterion is always **Von-Mises**, the hardening is **isoh='linear'**, and the parameters **cinh='none'**, **visc='none'**.

Assistant Corps Déformable (EF)

Matériau Déformable
Sélectionnez ou créez un matériau élastique.

Utiliser un matériau existant
Sélectionner : (Aucun matériau élastique)

☒ Créer un nouveau matériau à la place

Nouveau matériau

Nom (max 5 car.) : ELAS1

Type : ELAS

Densité : 2700,0 kg/m³

Module de Young : 70000000000,000 Pa

Coefficient de Poisson (ν) : 0,3000

Rappel : Les matériaux élastiques requièrent au minimum elas='standard', young, nu et anisotropy='isotropic'.

Retour Suivant Annuler

captures/assistant_defor_page3.JPG

Page 3 — Finite Element Model

Same logic as the Material page: reuse an existing model compatible with the dimension, or create a new one.

Mode A — Use an Existing Model

Only models whose dimension matches the one chosen on page 1 are listed.

Mode B — Create a New Model

Field	Description	Default Value
Name	Model identifier. 5 characters maximum.	femxx
Physics	Physics being solved.	MECAx, THERx, or HYDRx
Finite element	Element type. Adapted automatically to the dimension.	See table below
Anisotropy	iso__ (isotropic) or ortho (orthotropic).	iso__
Kinematics	small (small strains) or large (large strains).	small
Formulation	UpdtL (updated Lagrangian) or Total (total Lagrangian).	UpdtL
Mass storage	lump_ (lumped mass) or coher (coherent mass).	lump_

A description of the selected element is displayed in italics below the dropdown list.

Available Elements by Dimension

In 2D:

Element	Description
T3xxx	3-node linear triangle

Element	Description
Q4xxx	4-node bilinear quadrangle
T6xxx	6-node quadratic triangle
Q8xxx	8-node serendipity quadrangle
Q9xxx	9-node full biquadratic quadrangle

In 3D:

Element	Description
H8xxx	8-node trilinear hexahedron
H20xx	20-node triquadratic hexahedron
TE10x	10-node quadratic tetrahedron
SHB8x	8-node SHB8 hexahedral solid-shell
SHB6x	6-node SHB6 prismatic solid-shell

The options `material='elas_'` and `external_model='no___'` are automatically added to the model options during generation.

Assistant Corps Déformable (EF)

Modèle — Élément Fini
Sélectionnez ou créez un modèle éléments finis.

Utiliser un modèle existant
Sélectionner : (Aucun modèle 3D)

☒ Créer un nouveau modèle à la place

Nouveau modèle

Nom (max 5 car) : femxx

Physique : MECAx

Élément fini : TE4xx

Tétraèdre linéaire à 4 nœuds

Anisotropie : iso_

Cinématique : small

Formulation : UpdtL

Stockage masse : lump_

Retour Suivant Annuler

captures/assistant_defor_page4.JPG

Page 4 — Geometry

Defines the shape and dimensions of the body. The list of available shapes depends on the chosen dimension.

2D Shapes

Rectangle (native pylmgc90 structured mesh)

Generates a rectangular mesh via `pre.buildMesh2D()`.

Field	Description	Default
Center X, Y	Position of the center of the rectangle (m).	0.0, 0.0
Length X (lx)	Horizontal dimension (m).	1.0 m
Length Y (ly)	Vertical dimension (m).	1.0 m

The bottom-left corner is calculated automatically: $x_0 = cx - lx/2$, $y_0 = cy - ly/2$.

Disk (via gmsh)

Generates a mesh of a full circular disk via **gmsh** (`addDisk`), Frontal-Delaunay algorithm, then imports the `.msh` v2.2 file.

Field	Description	Default
Center X, Y	Position of the center of the disk (m).	0.0, 0.0
Radius (r)	Radius of the disk (m).	0.5 m

Requires that **gmsh** be installed and accessible in Python (`import gmsh`).

External File

Imports a mesh from an existing file via `pre.readMesh(filepath, 2)`.

Field	Description
Mesh file	Path to the file. Accepted formats: <code>.msh</code> , <code>.vtk</code> , <code>.mesh</code> . Use the  Browse button to navigate.

3D Shapes

Box (H8) (native pylmgc90 structured mesh)

Generates a hexahedral mesh via `pre.buildMeshH8()`.

Field	Description	Default
Center X, Y, Z	Position of the center of the box (m).	0.0, 0.0, 0.0
Length X (lx)	Dimension along X (m).	1.0 m
Length Y (ly)	Dimension along Y (m).	1.0 m
Length Z (lz)	Dimension along Z (m).	1.0 m

Sphere (via gmsh)

Generates a mesh of a full sphere via **gmsh** (`addSphere`), 3D Frontal algorithm.

Field	Description	Default
Center X, Y, Z	Position of the center of the sphere (m).	0.0, 0.0, 0.0
Radius (r)	Radius of the sphere (m).	0.5 m

Cylinder (via gmsh)

Generates a mesh of a full cylinder via **gmsh** (`addCylinder`), Z axis, centered at (cx, cy, cz).

Field	Description	Default
Center X, Y, Z	Position of the geometric center of the cylinder (m).	0.0, 0.0, 0.0
Radius (r)	Radius of the cylinder (m).	0.5 m
Height (h)	Axial length of the cylinder (m).	1.0 m

External File (3D)

Imports a 3D mesh via `pre.readMesh(filepath, 3)`. Accepted formats: `.msh`, `.vtk`, `.mesh`.

Assistant Corps Déformable (EF)

Géométrie
Définissez la forme et les dimensions de votre corps.

Forme : Fichier externe

Centre X (m) : 0,0000

Centre Y (m) : 0,0000

Centre Z (m) : 0,0000

Fichier maillage : C:/lmgc90_user_2023/examples/Tutorials/deformable/pre/gmsh_files/Donut.msh

Parcourir

Retour Suivant Annuler

captures/assistant_defor_page5.JPG

Page 5 — Mesh Refinement

Defines the fineness of the mesh. The displayed parameters depend on the geometry selected on the previous page.

Structured Mesh Type (Rectangle Only)

Type	Description
Q4	4-node bilinear quadrangles — fast and robust. Recommended by default.
2T3	3-node triangles obtained by cutting each Q4 in 2.
4T3	3-node triangles obtained by cutting each Q4 in 4. More isotropic mesh.
Q8	8-node serendipity quadrangles. More accurate, more costly.

Refinement Parameters by Geometry

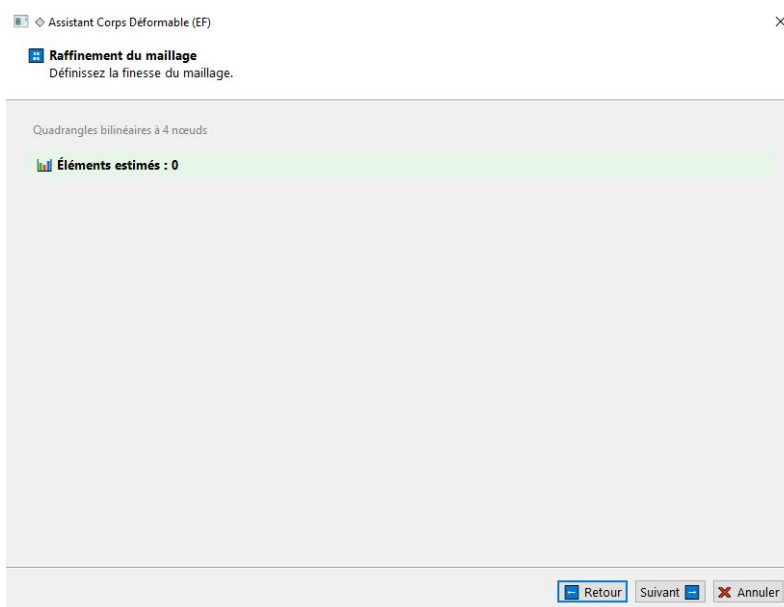
Geometry	Parameters	Estimate
Rectangle	<code>nx</code> × <code>ny</code> (default 10 × 10)	<code>nx</code> × <code>ny</code> elements
Disk	<code>nr</code> (default 5) × <code>ntheta</code> (default 16)	<code>nr</code> × <code>ntheta</code> elements
Box (H8)	<code>nx</code> × <code>ny</code> × <code>nz</code> (default 10 × 10 × 5)	<code>nx</code> × <code>ny</code> × <code>nz</code> elements

Geometry	Parameters	Estimate
Sphere	$nr \times ntheta \times nphi$ (default $5 \times 16 \times 8$)	$nr \times ntheta \times nphi$ elements
Cylinder	$nr \times ntheta \times nz$ (default $5 \times 16 \times 5$)	$nr \times ntheta \times nz$ elements
External file	_(no parameter — mesh defined in the file)_	—

An estimated element counter updates in real time as the parameters are modified.

Meaning of the gmsh Parameters:

Parameter	Meaning
nr	Number of layers of elements in the radial direction.
$ntheta$	Number of elements in the angular direction (circumference).
$nphi$	Number of elements in the polar direction (latitude, sphere only).
nz	Number of elements in the axial Z direction.
nx, ny, nz	Number of elements in the Cartesian X, Y, Z directions.



captures/assistant_defor_page6.JPG

Tip: start with the default values to check the geometry, then increase $nx/ny/nr/ntheta$ to improve the accuracy of the computation.

Page 6 — Boundary Conditions (DOF)

Defines the mechanical boundary conditions applied to the mesh's **surface groups**. These groups are created automatically by `buildMesh2D` and `buildMeshH8`.

Available Surface Groups

Group	Description	Available
$down$	Bottom edge ($y = y_{min}$ or $z = z_{min}$)	2D and 3D

Group	Description	Available
up	Top edge ($y = y_{\max}$ or $z = z_{\max}$)	2D and 3D
left	Left edge ($x = x_{\min}$)	2D and 3D
right	Right edge ($x = x_{\max}$)	2D and 3D
front	Front face ($z = z_{\min}$)	3D only
rear	Rear face ($z = z_{\max}$)	3D only

Note: for curved geometries (disk, sphere, cylinder) imported from gmsh, the surface groups depend on the physical entities defined in the .msh file. The names may differ from those above.

Creating a DOF Condition

Click + **Add a DOF condition** to create a row. Each row contains three columns:

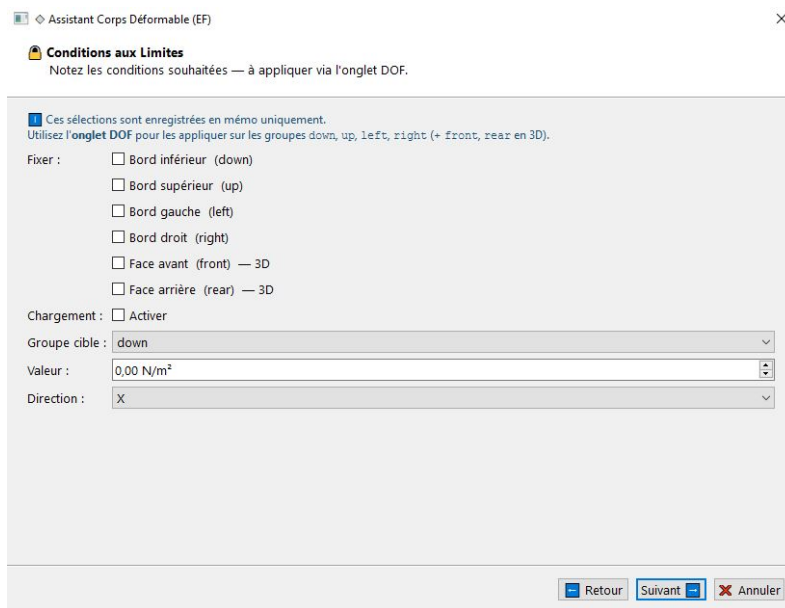
Column	Description	Values
DOF type	Nature of the condition.	<code>imposeDrivenDof</code> or <code>imposeInitValue</code>
Group	Surface group on which to apply the condition.	<code>down</code> , <code>up</code> , <code>left</code> , <code>right</code> , <code>front</code> , <code>rear</code>
Parameters	pylmgc90 arguments in the format <code>key=value</code> , <code>key=value</code> . Accepts Python expressions.	See examples below

Types of DOF Conditions

Type	Description
<code>imposeDrivenDof</code>	Driven degree of freedom: imposes a displacement or velocity for the duration of the computation.
<code>imposeInitValue</code>	Initial value: imposes a position or velocity only at time $t = 0$.

Parameter Examples

Usage	Parameters
Locking translation (2D)	<code>component=[1,2], dofty="vlocy"</code>
Locking translation (3D)	<code>component=[1,2,3], dofty="vlocy"</code>
Imposed displacement in Y	<code>component=[2], ct=0.001</code>
Zero initial value	<code>component=[1,2], value=0.0</code>
Locking X only	<code>component=[1], dofty="vlocy"</code>



captures/assistant_defor_page7.JPG

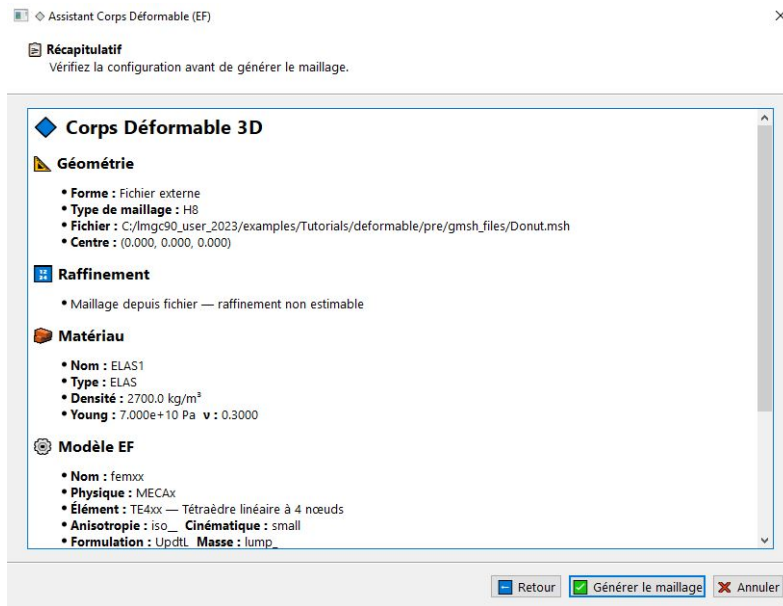
Parameter processing: the *Parameters* field is parsed via `SafeEvaluator.eval_dict()` — simple Python expressions are allowed (`[1, 2]`, `0.001`, `"vlocy"`). Each condition is converted into a `DOFOperation` and passed to `controller.add_dof_operation()`, which applies the condition AND saves it in `state.operations` (visible in the *DOF* tab for later editing).

To delete a row, click the **x** button to the right of the row.

Page 7 — Summary

Displays a complete summary of the configuration before generation.

Section	Information Displayed
Geometry	Shape, structured mesh type (2D), dimensions, center
Refinement	Discretization parameters, estimated number of elements
Material	Name, type, density, Young, ν — or existing material
FE Model	Name, physics, element, anisotropy, kinematics, formulation, mass storage
DOF Conditions	List of conditions: type, group, parameters



captures/assistant_defor_page8.JPG

Click **Generate the mesh** to launch generation. A confirmation message indicates the number of nodes and elements created.

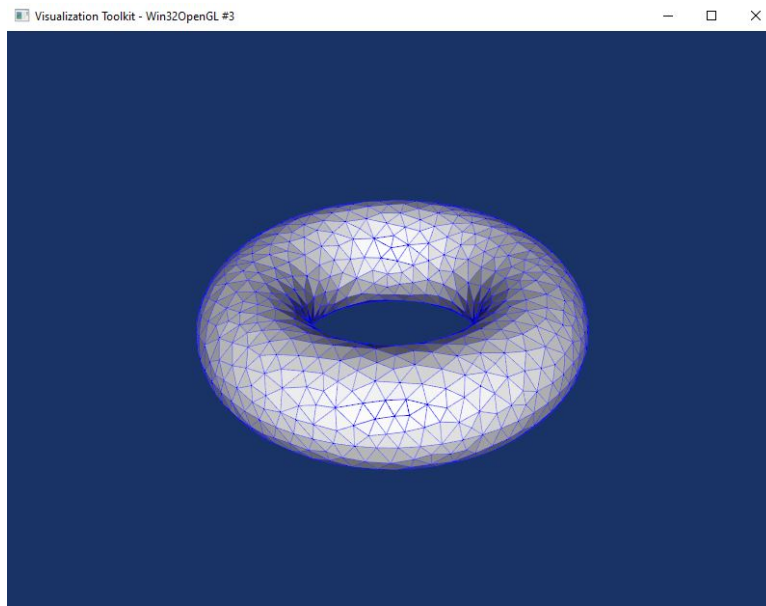
***In case of error:** the project state is fully restored. The error message details the cause (material not found, missing file, invalid parameter, gmsh not available, etc.).*

Generation Result

At the end of the wizard, the following elements are created in the project:

Element	Description
Material	Added to the Material tab (if created).
FE Model	Added to the Model tab (if created).
MESH_DEFORMABLE Avatar	Deformable body of color CYANx , added to the avatar list.
DOF Conditions	Saved in state.operations , visible in the DOF tab.

The deformable body can then be enriched via the **Empty Avatar** tab (mode "Existing deformable body") to add surface contactors enabling interactions with rigid bodies.



captures/assistant_defor_page9.JPG

Important Notes

External files: the `.msh` (gmsh v2), `.vtk`, and `.mesh` formats are accepted by `pre.readMesh()`. The file must be compatible with the chosen dimension (2D or 3D). For gmsh files, make sure the physical entities are defined (surfaces in 2D, volumes in 3D).

DOF conditions: the boundary conditions created in the wizard are saved and visible in the DOF tab. They can be modified or deleted after generation without having to relaunch the wizard.

Estimated number of elements: the counter on the Refinement page is an **estimate** based on the product of the discretization parameters. The actual number may differ slightly for curved geometries generated by gmsh.

Computation

Overview

The **calcul** (Computation) tab manages the configuration and launching of LMGC90 simulations. It automatically generates the DATBOX and the **command.py** script, then executes that script.

Computation Parameters

captures/calcul.JPG

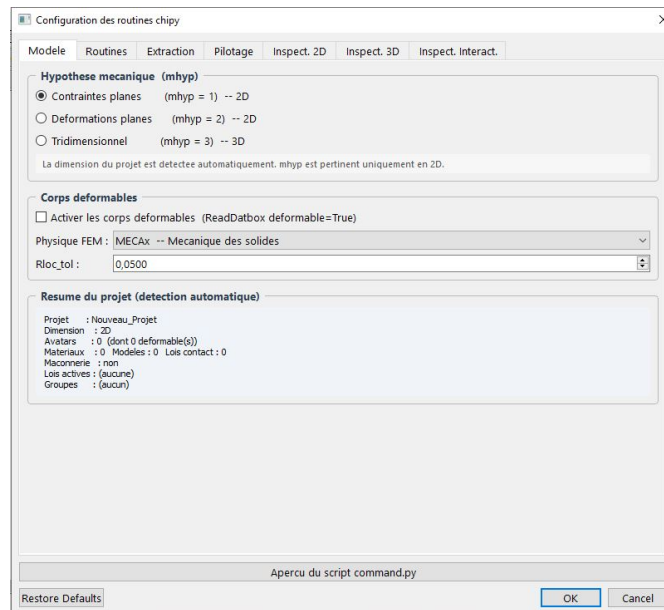
Parameter	UI Field	Default Value	Description
dt	Time step	1e-3	Time increment
nb_steps	Number of iterations	1000	Number of computation steps
theta	Integrator theta	0.5	Integration scheme parameter
tol	Tolerance	1.666e-4	Solver convergence tolerance
relax	Relaxation	1.0	Relaxation factor
norm	Norm	Quad	Convergence norm (Quad , QM , Maxim)
gs_it1	GS1 iterations	50	Number of Gauss-Seidel iterations (outer loop)
gs_it2	GS2 iterations	100	Number of Gauss-Seidel iterations (inner loop)
solver_type	Solver	NLGS	Solver type (Stored_Delassus_Loops , Exchange_Local_Global , Exchange_Local_Global)
freq_write	Write frequency	50	Write results every N steps
freq_display	Display frequency	50	Update display every N steps

Files Generated When Launching a Computation

File	Location	Description
DATBOX/	Project folder	LMGC90 input data
command.py	Project folder	chipy computation script
OUTBOX/	Project folder	Results (created by LMGC90)
Display/	Project folder	Display files
Postpro/	Project folder	Post-processing files

Configuring Your Computations

You can configure your computation scripts via the "Configure chipy routines" button. A dialog box will open in your interface.



captures/config_calculs.JPG

1. Model Tab

The first tab performs automatic detection based on your model's hypothesis, and loads all parameters according to your avatars.

Plane stress (mhyp = 1)	2D computation under plane stress conditions. Relevant for thin structures.
Plane strain (mhyp = 2)	2D computation under plane strain conditions. Relevant for structures that are infinitely long in z.
Three-dimensional (mhyp = 3)	Full 3D computation.

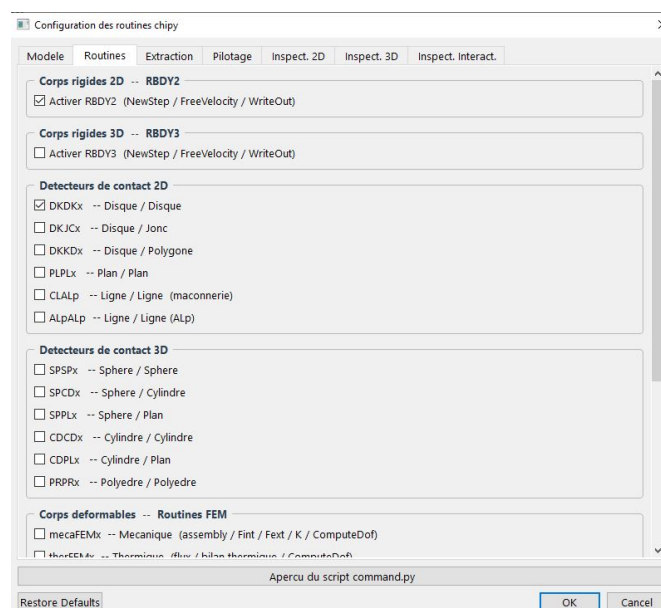
Deformable Bodies

Enable deformable bodies	Generates ReadDatbox(deformable=True) in the script. Automatically enables mecaFEMx if the physics is MECAX.
FEM Physics	Determines the finite element solver used: MECAX (solid mechanics), THERx (thermal), HYDRx (hydraulic), or THMx (coupled thermo-hydro-mechanical).

Enable deformable bodies	Generates ReadDatbox(deformable=True) in the script. Automatically enables mecaFEMx if the physics is MECaX.
Rloc_tol	Tolerance on contact forces for Rloc recovery. Typical value: 5×10^{-2} . Used in chipy.SetRlocTol().

2. Routines

This tab selects the chipy routines to include in the computation loop. Each checkbox corresponds to a call from the NewStep / ComputeStep / WriteOut family in the generated script.



captures/config_calculs_routines.JPG

2D Rigid Bodies — RBDY2

– RBDY2 (NewStep / FreeVelocity / WriteOut) — checked by default. Mandatory routines for any 2D rigid body. Generates RBDY2_NewStep(), RBDY2_FreeVelocity(), RBDY2_WriteOut() in the loop.

3D Rigid Bodies — RBDY3

– RBDY3 (NewStep / FreeVelocity / WriteOut) — triggers the equivalent 3D routines.

2D Contact Detectors

Each checkbox enables contact detection between a pair of contactor types. The script generates the corresponding XXX_SelectProxTactors() and XXX_RecupRloc() / XXX_StockRloc().

Detector	Description
DKDKx	Disk / Disk (checked by default) — 2D granulometry, granular media
DKJCx	Disk / Jonc — particles with ellipses
DKKDX	Disk / Polygon (Corde) — disk-to-polygonal-wall interaction
PLPLx	Plane / Plane — flat walls against each other
CLALp	Masonry line / line — interfaces between bricks (CLALp)
ALpALp	ALp line / line — variant for polygonal contactors

3D Contact Detectors

Detector	Description
SPSPx	Sphere / Sphere — 3D granulometry
SPCDx	Sphere / Cylinder
SPPLx	Sphere / Plane
CDCDx	Cylinder / Cylinder
CDPLx	Cylinder / Plane
PRPRx	Polyhedron / Polyhedron

Deformable Bodies — FEM Routines

- mecaFEMx — Solid mechanics: assembly, computation of internal forces (Fint), external forces (Fext), stiffness matrix (K), and resolution (ComputeDof).
- therFEMx — Thermal: heat flux, thermal energy balance, ComputeDof resolution.
- hydrFEMx — Hydraulic: hydraulic pressure, fluid flux, ComputeDof resolution.

Mixed Contactors — Rigid / Deformable

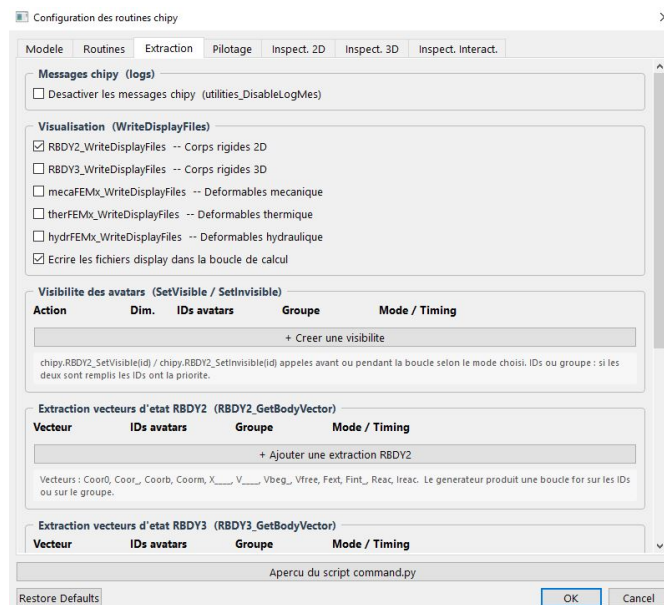
- DKMECAx — Interaction between rigid disks (2D) and mechanical meshes (2D FEM MECAx).
- ALpMECAx — Interaction between masonry interfaces (CLALp) and 2D FEM mechanical meshes. Useful for simulations of masonry structures with deformable blocks.
- SPMECAx — Interaction between rigid spheres (3D) and 3D FEM mechanical meshes.

Special Routines

- PT2Dx — 2D point nodes: point/point interaction for cable elements (ELASTIC_WIRE) and elastic rods (ELASTIC_ROD).
- PT3Dx — 3D equivalent of PT2Dx.
- NODES — Coupled nodes: degree-of-freedom coupling routines (COUPLED_DOF, NORMAL_COUPLED_DOF).
- UpdateBulkBehav — Bulk behavior laws: generates chipy.UpdateBulkBehav() for models with plasticity, damage, or history variables.

3. Extraction

This tab configures all computation outputs: visualization files, avatar visibility, state vector extraction, contact forces, energy, and FEM fields.



captures/config_calculs_extraction.JPG

chipy Messages (logs)

– Disable chipy messages (`utilities_DisableLogMes`) — Generates `chipy.utilities_DisableLogMes()` immediately after `chipy.Initialize()`. Suppresses all progress messages in the console. Recommended for production or long-duration computations.

Visualization (WriteDisplayFiles)

Controls the writing of visualization files to the `DISPLAY/` directory. Each checkbox corresponds to a family of avatars:

- `RBDY2_WriteDisplayFiles` — 2D rigid bodies (checked by default).
- `RBDY3_WriteDisplayFiles` — 3D rigid bodies.
- `mecaFEMx_WriteDisplayFiles` — Mechanical deformable meshes.
- `therFEMx_WriteDisplayFiles` — Thermal deformable meshes.
- `hydrFEMx_WriteDisplayFiles` — Hydraulic deformable meshes.
- Write display files in the loop — If checked, files are written at every step (or at the defined frequency). If unchecked, only a single file is written at the end of the computation.

Avatar Visibility (SetVisible / SetInvisible)

Allows you to show or hide avatars individually or by group at specific moments during the simulation. Each row in the list corresponds to a visibility rule. To create a rule, click "+ Create a visibility". Each row contains:

Action	SetVisible or SetInvisible — makes the avatar visible or invisible in chipy.
Dim.	2D (RBDY2) or 3D (RBDY3) — determines the prefix of the generated function.
Avatar IDs	Comma-separated list of identifiers (e.g.: 1, 3, 5). Takes priority over the group if both are filled in.
Group	Name of an avatar group defined in the project. Resolved into a list of IDs at generation time.
Mode / Timing	Determines when the call is generated (see timing modes table below).

Timing Modes

Timing Modes

Mode	Generated Condition	Typical Use
Every step	No condition (direct call)	Systematic extraction, continuous energy balance

Every N steps |if k % N == 0: |Reduce write frequency to lighten outputs|

At step k =	if k == K:	One-off event: change visibility at a specific step
After the loop	Outside the loop (after for k ...)	Final state only, post-processing at the end of the computation

RBDY2 State Vector Extraction (RBDY2_GetBodyVector)

Generates calls to `chipy.RBDY2_GetBodyVector(vector, id)` in the computation loop. For each row added via "+ Add an RBDY2 extraction", configure:

Vector	Name of the state vector to extract (see full list below).
Avatar IDs	Comma-separated list of IDs. If filled in, generates a for loop over these IDs.
Group	Group of avatars to iterate over. Lower priority than IDs if both are filled in.
Mode / Timing	One of the four timing modes described above.

Available vectors:

Name	Description
Coor0	Reference position (initial configuration)
Coor_	Current position
Coorb	Position at the previous step
Coorm	Average position between two steps
X____	Total accumulated displacement
V____	Current velocity (linear and angular)
Vbeg_	Velocity at the beginning of the step
Vfree	Free velocity (before contact resolution)
Fext_	Applied external forces and moments
Fint_	Internal forces and moments
Reac_	Resultant of contact reactions
Ireac	Contact reaction impulses

Contact Forces and Reactions

– Nodal forces (inter_handler_Rnod) — Extracts nodal forces at contact points and writes them to POSTPRO/. – Local velocities (inter_handler_Vloc) — Extracts relative velocities in the local contact frame. – Local forces (inter_handler_Rloc) — Extracts impulses/forces in the local contact frame.

Energy

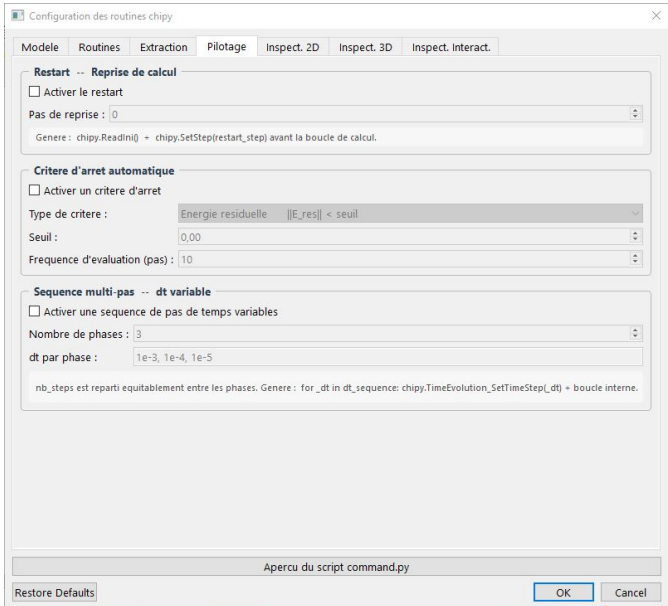
– Global energy balance (ComputeEnergy + WriteEnergy) — Computes and writes the kinetic, potential, and friction-dissipated energy. – RBDY2 kinetic energy (RBDY2_KineticEnergy) — Writes the kinetic energy of each 2D rigid body separately.

FEM Fields (stresses, strains, temperature...)

- Per-element fields (mecaFEMx_WriteBodies) — Writes per-element fields: stresses and strains (MECAx), temperature (THERx), pressure (HYDRx).
- Internal variables (mecaFEMx_WriteInternalVariables) — Writes internal variables at Gauss points: plasticity, damage, history variables.

4. Control

This tab controls advanced functions of the computation workflow: resuming from a saved state, early stopping based on a convergence criterion, and multi-step sequencing.



captures/config_pilotage.JPG

Restart — Resuming a Computation Allows you to resume a computation from a state previously saved in the .dat.last files.

Enable restart	Generates <code>chipy.ReadIni()</code> then <code>chipy.SetStep(restart_step)</code> before the computation loop.
Restart step	Number of the time step from which to resume (integer, from 0 to 9,999,999).

Automatic Stopping Criterion

Interrupts the computation loop before the planned number of steps is reached if a convergence criterion is satisfied.

Enable a stopping criterion	Enables the early stopping mechanism. Generates a break condition in the for k loop.		
Criterion type	Three types available: energy residual (<code> E_res < threshold</code>), maximum displacement (<code>max</code>	u	< threshold), force residual (<code> F_res < threshold</code>).
Threshold	Numerical value of the stopping criterion (from <code>10⁻¹²</code> to 1.0). Default value: <code>10⁻⁶</code> .		
Evaluation frequency	Evaluate the criterion every N steps. Avoids computing the criterion at every iteration, which can be costly.		

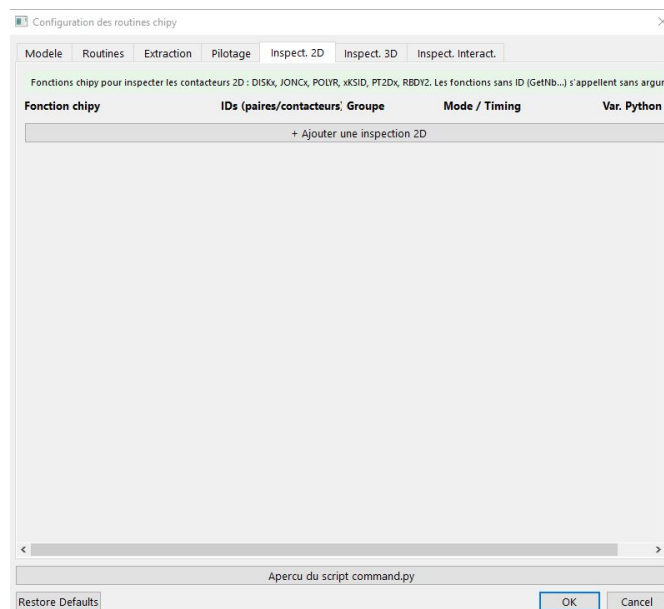
Multi-Step Sequence — Variable dt

Allows you to define several computation phases with different time steps. Useful for computations with progressive loading or to refine the time step as a critical event approaches.

Enable a multi-step sequence	Generates an outer loop over the phases: <code>for _dt in dt_sequence: chipy.TimeEvolution_SetTimeStep(_dt) + inner loop.</code>
Number of phases	Between 2 and 20 phases. The total number of steps (<code>nb_steps</code>) is evenly distributed across the phases.
dt per phase	Comma-separated list of dt values, one per phase (e.g.: 1e-3, 1e-4, 1e-5).

5. 2D Inspection

This tab lets you add inspection calls on the model's 2D contactors. Each row corresponds to a `chipy.XXXX_GetYYYY()` call inserted in the computation loop or after it.



captures/config_inspect2D.JPG

Click "+ Add a 2D inspection" to create a row. Each row contains five columns:

chipy Function	Selection from the dropdown list of functions available for 2D contactors. The description is displayed in the tooltip.
IDs (contactors)	Comma-separated list of chipy identifiers. Left empty for <code>GetNb...</code> type functions that take no argument.
Group	Name of an avatar group. Resolved into IDs at generation time if the IDs are empty.
Mode / Timing	One of the four timing modes (Every step, Every N steps, At step k =, After the loop).
Python var.	Name of the Python variable in which to store the result (e.g.: <code>vel_disk</code>). Left empty if the result is not reused.

Available Functions — 2D Contactors

Functions are grouped by contactor type:

DISKx — 2D Rigid Disks — DISKx_GetNbDISKx — Total number of DISKx contactors (no ID required). — DISKx_GetBodyId(i) — ID of the RBDY2 body to which contactor i belongs. — DISKx_GetPtrDISKx2BDYTY(i) — Local index of the contactor within its RBDY2 body. — DISKx_GetPtrTactBehav(i) — Contact behavior law associated with contactor i. — DISKx_GetRadius(i) — Radius of disk i. — DISKx_GetCoor(i) — Coordinates of the center of disk i. — DISKx_GetVelocity(i) — Velocity of the center of disk i.

JONCx — 2D Joncs / Ellipses — JONCx_GetNbJONCx — Total number of JONCx contactors. — JONCx_GetBodyId(i), JONCx_GetPtrJONCx2BDYTY(i), JONCx_GetPtrTactBehav(i) — Identification. — JONCx_GetAxes(i) — Half-axes (a, b) of jonc i. — JONCx_GetCoor(i) — Coordinates of the center of jonc i.

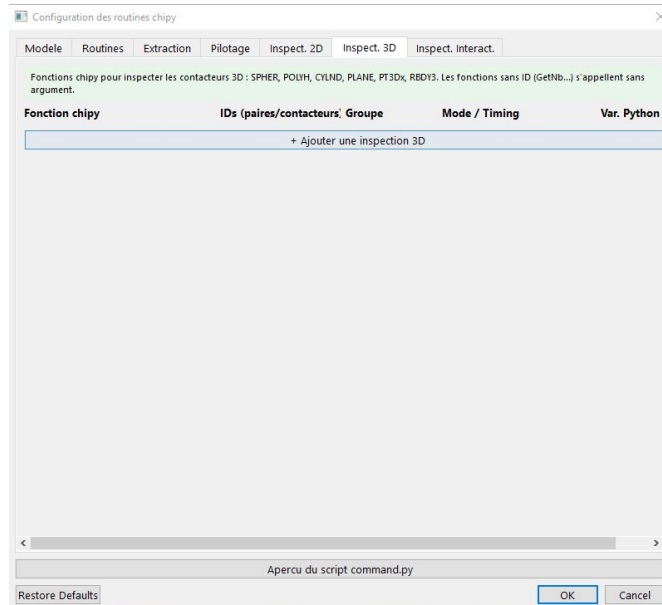
POLYR — 2D Rigid Polygons — POLYR_GetNbPOLYR — Total number of POLYR contactors. — POLYR_GetBodyId(i), POLYR_GetPtrPOLYR2BDYTY(i), POLYR_GetPtrTactBehav(i) — Identification. — POLYR_GetNbVerti(i) — Number of vertices of polygon i. — POLYR_GetVerti(i) — Coordinates of all the vertices of polygon i. — POLYR_GetCoor(i) — Coordinates of the reference center of polygon i.

xKSID — 2D Discrete Disk Clusters — xKSID_GetNbKSID, xKSID_GetBodyId(i), xKSID_GetPtrxKSID2BDYTY(i), xKSID_GetRadius(i).

RBDY2 — 2D Rigid Bodies (summary) — RBDY2_GetNbRBDY2 — Total number of 2D rigid bodies. — RBDY2_KineticEnergy — Total kinetic energy of all RBDY2 bodies.

PT2Dx — 2D FEM Contactor Nodes — PT2Dx_GetNbPT2Dx — Number of 2D FEM contactor nodes. — PT2Dx_GetBodyId(i) — ID of the parent FEM body. — PT2Dx_GetCoor(i) — Coordinates of contactor node i.

6. 3D Inspection



captures/config_inspect3D.JPG

Works the same way as the 2D Inspection tab, but for 3D contactors. The available families are:

SPHER — 3D Rigid Spheres — SPHER_GetNbSPHER, SPHER_GetBodyId(i), SPHER_GetPtrSPHER2BDYTY(i), SPHER_GetPtrTactBehav(i). — SPHER_GetRadius(i) — Radius of sphere i. — SPHER_GetCoor(i), SPHER_GetVelocity(i) — Position and velocity.

POLYH — 3D Rigid Polyhedra — POLYH_GetNbPOLYH, POLYH_GetBodyId(i), POLYH_GetPtrPOLYH2BDYTY(i), POLYH_GetPtrTactBehav(i). — POLYH_GetNbFaces(i), POLYH_GetNbVerti(i), POLYH_GetVerti(i) — Geometry. — POLYH_GetCoor(i) — Coordinates of the reference center.

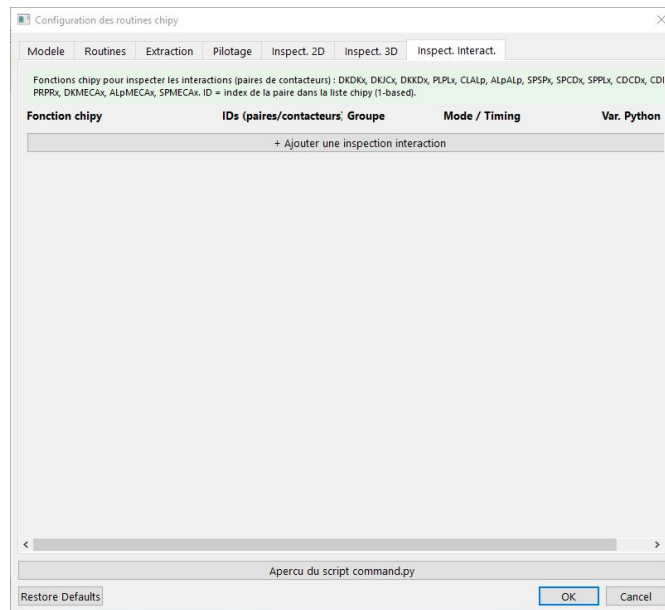
CYLND – 3D Rigid Cylinders – CYLND_GetNbCYLND, CYLND_GetBodyId(i), CYLND_GetPtrCYLND2BDYTY(i), CYLND_GetPtrTactBehav(i). – CYLND_GetRadius(i), CYLND_GetLength(i), CYLND_GetCoor(i).

PLANE – 3D Rigid Planes – PLANE_GetNbPLANE, PLANE_GetBodyId(i), PLANE_GetNormal(i), PLANE_GetCoor(i).

RBDY3 and PT3Dx – RBDY3_GetNbRBDY3 — Total number of 3D rigid bodies. – PT3Dx_GetNbPT3Dx, PT3Dx_GetBodyId(i), PT3Dx_GetCoor(i) — 3D FEM contactor nodes.

7. Interaction Inspection

This tab inspects the active contactor pairs (interactions currently occurring in the simulation). The ID used is the index of the pair in the chipy list (1-based numbering).



captures/config_interac.JPG

Functions are grouped by interaction type:

`DKDKx – Disk / Disk` – DKDKx_GetNbDKDKx — Number of active pairs. – DKDKx_GetBodyIds(i) — RBDY2 IDs of the two bodies in pair i. – DKDKx_GetTactors(i) — IDs of the two DISKx contactors in pair i. – DKDKx_GetGapTT(i) — Gap of pair i. – DKDKx_GetStatusTT(i) — Contact status: 0 = no contact, 1 = active contact. – DKDKx_GetRlocTT(i) — Local reaction (Rn, Rt) in the local contact frame. – DKDKx_GetVlocTT(i) — Relative local velocity (Vn, Vt).

`DKJCx – Disk / Jonc` – DKJCx_GetNbDKJCx, DKJCx_GetBodyIds(i), DKJCx_GetTactors(i), DKJCx_GetGapTT(i), DKJCx_GetStatusTT(i), DKJCx_GetRlocTT(i).

`DKK Dx – Disk / Corde (Polygon)` – DKK Dx_GetNbDKK Dx, DKK Dx_GetBodyIds(i), DKK Dx_GetGapTT(i), DKK Dx_GetRlocTT(i).

`PLPLx – Polygon / Polygon` – PLPLx_GetNbPLPLx, PLPLx_GetBodyIds(i), PLPLx_GetTactors(i), PLPLx_GetGapTT(i), PLPLx_GetStatusTT(i), PLPLx_GetRlocTT(i), PLPLx_GetVlocTT(i).

`CLALp – Brick / Brick (masonry)` – CLALp_GetNbCLALp, CLALp_GetBodyIds(i), CLALp_GetGapTT(i), CLALp_GetStatusTT(i), CLALp_GetRlocTT(i). ● Use CLALp with "Every step" mode to track the evolution of contact forces in masonry joints throughout the simulation.

`ALpALp – ALp / ALp` – ALpALp_GetNbALpALp, ALpALp_GetBodyIds(i), ALpALp_GetGapTT(i), ALpALp_GetRlocTT(i).

`SPSPx — Sphere / Sphere (3D)` — SPSPx_GetNbSPSPx, SPSPx_GetBodyIds(i), SPSPx_GetTactors(i), SPSPx_GetGapTT(i), SPSPx_GetStatusTT(i), SPSPx_GetRlocTT(i) (Rn, Rt, Rs), SPSPx_GetVlocTT(i) (Vn, Vt, Vs).

`SPCDx — Sphere / Cylinder (3D)` — SPCDx_GetNbSPCDx, SPCDx_GetBodyIds(i), SPCDx_GetTactors(i), SPCDx_GetGapTT(i), SPCDx_GetRlocTT(i).

`SPPLx — Sphere / Plane (3D)` — SPPLx_GetNbSPPLx, SPPLx_GetBodyIds(i), SPPLx_GetGapTT(i), SPPLx_GetRlocTT(i).

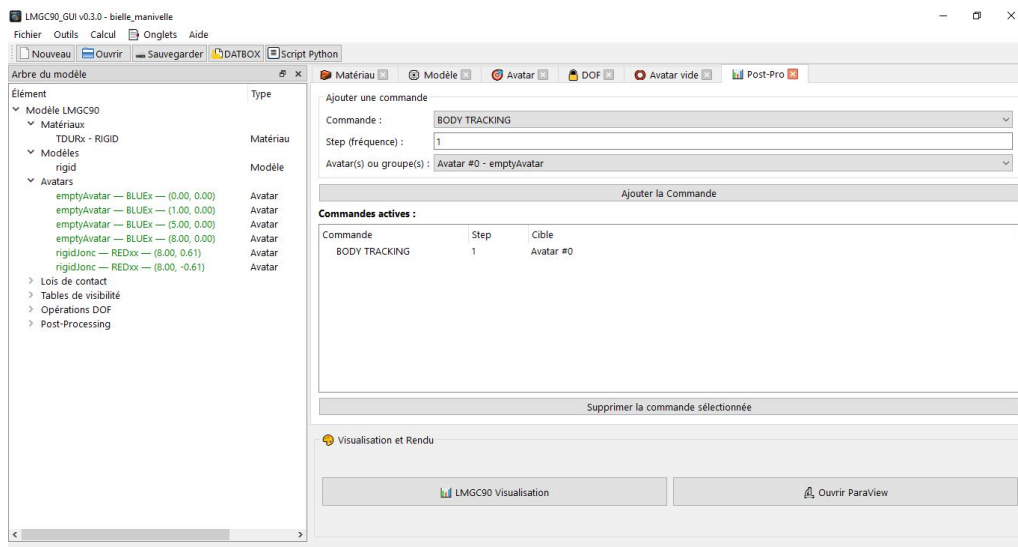
`CDCDx — Cylinder / Cylinder (3D)` — CDCDx_GetNbCDCDx, CDCDx_GetBodyIds(i), CDCDx_GetGapTT(i), CDCDx_GetRlocTT(i). CDPLx — Cylinder / Plane (3D) — CDPLx_GetNbCDPLx, CDPLx_GetBodyIds(i), CDPLx_GetGapTT(i), CDPLx_GetRlocTT(i).

`PRPRx — Polyhedron / Polyhedron (3D)` — PRPRx_GetNbPRPRx, PRPRx_GetBodyIds(i), PRPRx_GetTactors(i), PRPRx_GetGapTT(i), PRPRx_GetStatusTT(i), PRPRx_GetRlocTT(i) (Rn, Rt, Rs).

`Mixed Contactors — Rigid / Deformable` — DKMECAx_GetNbDKMECAx, DKMECAx_GetBodyIds(i), DKMECAx_GetGapTT(i), DKMECAx_GetRlocTT(i) — Disk / 2D FEM MECAx. — ALpMECAx_GetNbALpMECAx, ALpMECAx_GetBodyIds(i), ALpMECAx_GetRlocTT(i) — ALp / 2D FEM MECAx. — SPMECAx_GetNbSPMECAx, SPMECAx_GetBodyIds(i), SPMECAx_GetRlocTT(i) — Sphere / 3D FEM MECAx.

Post-Processing (PostPro)

The **Post-Pro** tab defines the **post-processing commands** that will be included in the DATBOX and executed by LMGC90 during the computation. Each command asks LMGC90 to write output files at a given frequency, allowing the results to subsequently be analyzed and visualized.



captures/postpro.JPG

Operating Principle

A **PostPro command** (`PostProCommand`) associates a pylmgc90 **command name** with a **write frequency** (`step`) and, optionally, a **target** (single avatar or group). The corresponding pylmgc90 call is:

```
# Without a target (global command):
posts.addCommand(pre.postpro_command(name='SOLVER INFORMATIONS', step=1))

# With an avatar target:
posts.addCommand(pre.postpro_command(
    name='BODY TRACKING',
    step=10,
    rigid_set=[bodies[3]]
))

# With a group target:
posts.addCommand(pre.postpro_command(
    name='TORQUE EVOLUTION',
    step=5,
    rigid_set=[bodies[i] for i in group_granulo_box2d]
))
```

The `PostProCommand` structure in the project stores three fields: `name`, `step`, and optionally `target_type` / `target_value`.

Tab Interface

The tab is organized into two areas:

- **List of commands** (top): a table of all recorded commands. Each row displays the command name, the frequency (**step=N**), and the target (**Global**, **Avatar #N**, or **Group: name**). Double-click to edit.
- **Creation / editing form** (bottom): three simple fields.

Form Fields

Field	Description	Default Value
Command	Name of the pylmgc90 post-processing command. Select from the dropdown list.	SOLVER INFORMATIONS
Frequency (step)	Writes every step computation steps. step=1 = writes at every step, step=100 = every 100 steps.	1
Target	Global (entire simulation), Avatar (numeric index), or Group (group name).	Global

Available Commands

Numerical Quality — No Target Required

These commands are **global** and do not require a **rigid_set**. They apply to the entire simulation.

Command	Description	Output Files
SOLVER INFORMATIONS	Contact solver information at each step: number of iterations, residual, computation time. Essential for checking the convergence of the contact scheme.	OUTBOX/solver_informations.dat
VIOLATION EVOLUTION	Average and maximum violation (residual interpenetration) evolution between bodies. Measures the numerical error of non-penetration.	OUTBOX/violation_evolution.dat
KINETIC ENERGY	Total kinetic energy of the system at each step. Useful for tracking energy dissipation and detecting instabilities.	OUTBOX/kinetic_energy.dat
CONTACT ENERGY	Energy dissipated by contacts (friction + restitution).	OUTBOX/contact_energy.dat
STRAIN ENERGY	Strain energy stored in the system. For deformable FE bodies.	OUTBOX/strain_energy.dat

etc.,

Body Tracking — Target Required (**rigid_set**)

These commands write information about a **specific body** or a **group of avatars**. A target (**target_type** + **target_value**) is mandatory.

Command	Description	Extracted Data	Output Files
BODY TRACKING	Complete tracking of the position, velocity, and acceleration of a body over time. The most commonly used command for analyzing an avatar's trajectory.	Position (x, y, z), velocity (vx, vy, vz), acceleration, angle, and angular velocity	OUTBOX/body_tracking.dat
TORQUE EVOLUTION	Evolution of the torque applied to the body or group.	Torque components along X, Y, Z	OUTBOX/torque_evolution.dat
MOMENTUM EVOLUTION	Evolution of the momentum of the body or group.	Momentum (px, py, pz)	OUTBOX/momentum_evolution.dat

Additional Commands

Command	Description
WORK EVOLUTION	Work done by the external forces applied over time.
DISSIPATED ENERGY	Total dissipated energy (contact + damping).

Command Target

Global **__(no rigid_set)__**

The command applies to the entire system. Use for **SOLVER INFORMATIONS**, **VIOLATION EVOLUTION**, **KINETIC ENERGY**.

```
pre.postpro_command(name='SOLVER INFORMATIONS', step=1)
```

Avatar (single index)

The command monitors a specific body identified by its index in the avatar list (0-based).

```
pre.postpro_command(name='BODY TRACKING', step=10, rigid_set=[bodies[3]])
```

In the interface: select **Avatar** as the target type, then enter the index (e.g.: 3).

Group (set of avatars)

The command monitors all the bodies of a named group (loop, granulometry, masonry...).

```
pre.postpro_command(
name='TORQUE EVOLUTION',
step=5,
rigid_set=[bodies[i] for i in group_granulo_box2d]
)
```

In the interface: select **Group** as the target type, then choose the group from the dropdown list.

All groups defined in the project (loops, granulometry, masonry) automatically appear in the list.

Write Frequency **(step)**

The `step` parameter controls the write frequency of the results files.

step Value	Behavior	Use
1	Writes at every computation step	Detailed analysis, debugging, small models
10	Writes every 10 steps	Good accuracy / file size compromise
100	Writes every 100 steps	Large, long-duration simulations
step_total / 1000	~1000 points in the file	Rule of thumb for smooth curves

Performance impact: a `step=1` with `BODY TRACKING` on a large group can generate files several gigabytes in size and slow down the computation. Adapt the frequency to the required duration and accuracy.

Managing Commands

Creating a Command

Fill in the form and click **Add the Command**. The command is created via `add_postpro_command()`, which:

1. Resolves the target (`rigid_set`) into lists of `pymgc90` objects.
2. Creates the `pre.postpro_command(name, step, rigid_set)` object.
3. Adds it to the `_postpro_container` via `addCommand()`.
4. Saves it in `state.postpro_commands`.
5. Emits the `command_added` signal → `_refresh_all()`.

Editing a Command

Double-click in the list to load the values into the form. Edit them and click **Update**. `update_postpro_command()` rebuilds the entire postpro container (same behavior as for visibility tables — a `pymgc90` limitation).

Deleting a Command

Select it and click **Delete**. The command is removed from `state.postpro_commands` via `remove_postpro_command()`. The `command_deleted` signal is emitted.

Display in the Model Tree

PostPro commands are displayed in the model tree (left panel) under the **Post-Processing** node, with for each command:

- Its name (e.g. `BODY TRACKING`)
- Its frequency (`step=10`)
- Its target (`Global`, `Avatar #3`, or `Group: granulo_box2d`)

Double-clicking on a command in the tree directly opens the PostPro tab in edit mode (`load_for_edit(postpro)`).

Generated Python Script

```
# Post-processing
```



```

# Global command
post_cmd_0 = pre.postpro_command(
name='SOLVER INFORMATIONS',
step=1
)
posts.addCommand(post_cmd_0)

# Command with an avatar target
post_cmd_1 = pre.postpro_command(
name='BODY TRACKING',
step=10,
rigid_set=[bodies[3]]
)
posts.addCommand(post_cmd_1)

# Command with a group target
post_cmd_2 = pre.postpro_command(
name='TORQUE EVOLUTION',
step=5,
rigid_set=[bodies[i] for i in group_granulo_box2d]
)
posts.addCommand(post_cmd_2)

```

The `posts` container is passed to `pre.writeDatbox(post=posts, ...)` when generating the DATBOX.

Reading the Results

The output files are written to the LMGC90 project's `OUTBOX/` directory during the computation. Each file is a text file (space-separated columns) whose format depends on the command:

Command	Typical Format	Columns
BODY TRACKING	Text, N columns	<code>t, x, y, z, vx, vy, vz, theta, omega</code>
SOLVER INFORMATIONS	Text	<code>t, iter, residual, cpu_time</code>
VIOLATION EVOLUTION	Text	<code>t, mean_violation, max_violation</code>
KINETIC ENERGY	Text	<code>t, Ec</code>
TORQUE EVOLUTION	Text	<code>t, Mx, My, Mz</code>

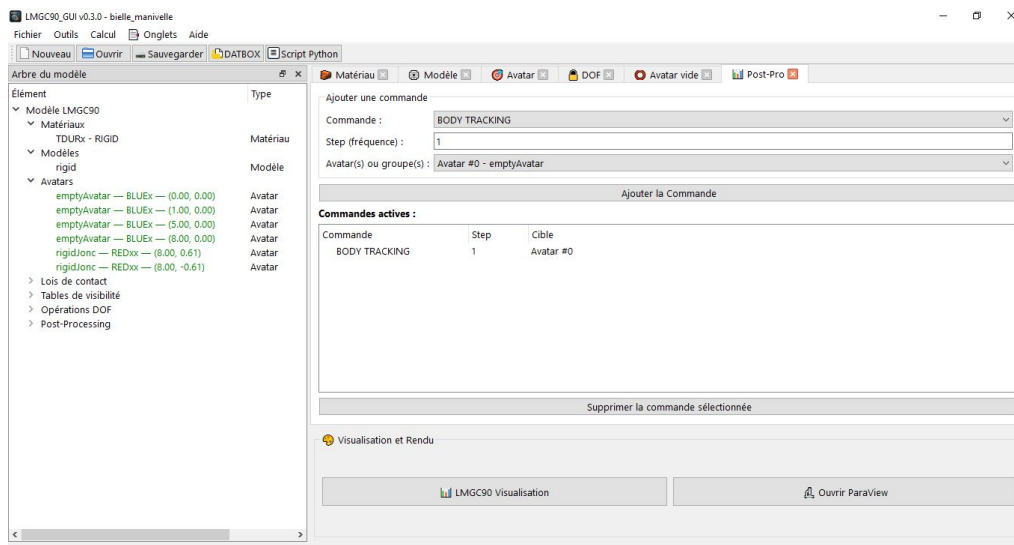
These files can be read and plotted directly with Python (numpy, matplotlib) or with LMGC90's built-in visualization tool.

Usage Example — Slider-Crank

For a slider-crank simulation, configure the following commands:

#	Command	Step	Target	Objective
0	SOLVER INFORMATIONS	1	Global	Check convergence
1	VIOLATION EVOLUTION	1	Global	Monitor interpenetration
2	BODY TRACKING	10	Avatar #0 (crank)	Angular trajectory
3	BODY TRACKING	10	Avatar #2 (slider)	Linear displacement

#	Command	Step	Target	Objective
4	KINETIC ENERGY	1	Global	Energy balance



PostPro tab example

Important Notes

Reconstruction upon editing: as with visibility tables, any modification to a command triggers a full reconstruction of the `_postpro_container`. This is transparent to the user.

Output files are only written during the computation. Generating the DATBOX and the Python script does not write any results — the computation must be launched from the Computation tab (F5) for the OUTBOX/ files to be created.

Step and simulation duration: make sure `step` is smaller than the total number of computation steps. A command with `step=100` in a 50-step simulation will produce no output.

Consistency of avatar indices: the indices in `target_value` reference the position in `state.avatars` at the time of creation. If avatars are deleted or reordered after a command is created, the indices may become incorrect. Prefer named groups for commands covering multiple bodies.

Post-Processing

Definition of the outputs for LMGC90 used to extract and analyze your computations.

Available Commands

1. Checking Numerical Quality

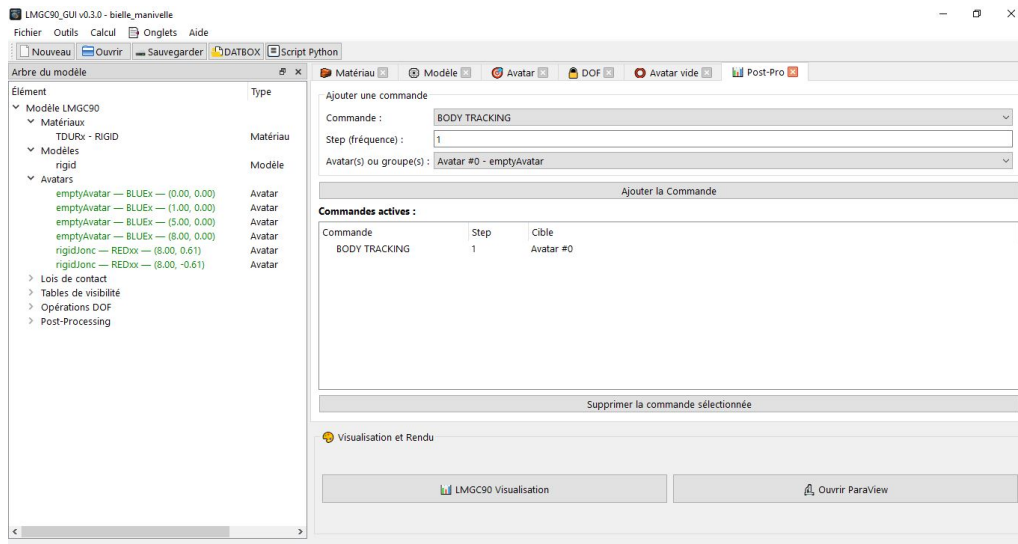
- SOLVER INFORMATIONS: to ensure convergence
- VIOLATION EVOLUTION: measures the average interpenetration "error"
- TORQUE EVOLUTION (on an avatar/group):
- BODY TRACKING (body tracking)
- KINETIC ENERGY, etc.

Features

- step: step
- rigid_set: avatar or group of avatars

Example:

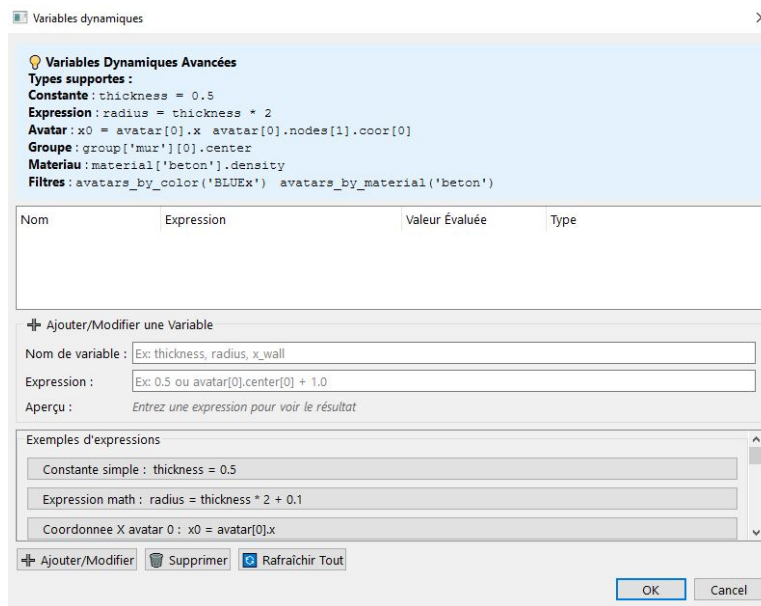
To add a postpro command, go to the "PostPro" tab, then choose the desired command — in this case "BODY TRACKING" — then enter the step, and click the **"Add the Command"** button



captures/postpro.JPG

Dynamic Variables

LMGC90_GUI's **dynamic variables** allow you to write Python expressions directly into the interface's input fields, using the project's data (avatars, materials, models, groups) as variables. They replace fixed numeric values with expressions computed on the fly, and propagate automatically across all tabs.



captures/variables.JPG

Accessing the Variable Manager

Method	Action
Tools → Dynamic Variables menu	Opens the manager
Shortcut	Ctrl+V

The Dynamic Variables Manager

The manager (`DynamicVarsDialog`) uses `SafeEvaluator` with `build_eval_context()` to evaluate expressions safely. It displays a 4-column table: **Name**, **Expression**, **Evaluated Value**, **Type**. Each variable is a Python expression stored as a string and evaluated at the time it is used.

Adding a Variable

1. Enter a **Name** — a valid Python identifier (`thickness`, `r_min`, `x_wall`).
2. Enter an **Expression** — see the possibilities below.
3. The **Preview** updates in real time: result shown in green if valid, error in red.

4. Click **Add / Update**.

Editing a Variable

Click on the row in the table → the Name and Expression fields are filled in → edit → **Add / Update**.

Deleting a Variable

Select the row and click **Delete**.

Refreshing

Click **Refresh** to re-evaluate all variables against the current project state (useful after adding avatars).

Persistence: variables are saved in the project's `.lmgc90` file (`state.dynamic_vars`). They are automatically reloaded when the project is opened.

Supported Expression Types

Numeric Constants

```
thickness = 0.5
radius = 0.075
n_cols = 15
pi_val = 3.14159
```

Mathematical Expressions

All numeric Python expressions are supported:

```
diameter = 2 * radius
area = pi * radius**2
step = lx + joint
half = thickness / 2
diag = sqrt(lx**2 + ly**2)
clamp = max(0.0, min(1.0, ratio))
```

Available mathematical functions:

Symbol	Description
<code>pi</code>	$\pi \approx 3.14159\dots$
<code>e</code>	$e \approx 2.71828\dots$
<code>sqrt(x)</code>	Square root
<code>abs(x)</code>	Absolute value
<code>min(a, b)</code>	Minimum
<code>max(a, b)</code>	Maximum
<code>round(x, n)</code>	Rounded to n decimal places
<code>sum(list)</code>	Sum of a list
<code>len(object)</code>	Length
<code>math.sin(x)</code>	Sine (x in radians)
<code>math.cos(x)</code>	Cosine

Symbol	Description
<code>math.tan(x)</code>	Tangent
<code>math.log(x)</code>	Natural logarithm
<code>math.log10(x)</code>	Base-10 logarithm
<code>math.exp(x)</code>	Exponential
<code>math.floor(x)</code>	Floor (round down)
<code>math.ceil(x)</code>	Ceiling (round up)
<code>math.radians(deg)</code>	Degrees → radians
<code>math.degrees(rad)</code>	Radians → degrees
<code>np.array(list)</code>	numpy array
<code>np.linspace(a, b, n)</code>	Evenly spaced array

etc.

Variables Referencing Other Variables

Variables are evaluated in the order in which they are defined — a variable can reference previous ones:

```
thickness = 0.05
radius = thickness * 3
step = radius + thickness
grid_size = step * 10
```

Accessing Project Data

Avatars — `avatar[i]`

Access to an avatar by its index (0-based). All properties are read-only.

Expression	Description	Returned Type
<code>avatar[i].center</code>	Full center <code>[x, y]</code> or <code>[x, y, z]</code>	<code>list</code>
<code>avatar[i].x</code>	X coordinate of the center	<code>float</code>
<code>avatar[i].y</code>	Y coordinate of the center	<code>float</code>
<code>avatar[i].z</code>	Z coordinate of the center (3D)	<code>float</code> or <code>None</code>
<code>avatar[i].radius</code>	Radius of the avatar	<code>float</code> or <code>None</code>
<code>avatar[i].color</code>	LMGC90 color ('BLUEx')	<code>str</code>
<code>avatar[i].material_name</code>	Material name	<code>str</code>
<code>avatar[i].model_name</code>	Model name	<code>str</code>
<code>avatar[i].avatar_type</code>	pylmgc90 type ('rigidDisk')	<code>str</code>
<code>avatar[i].origin</code>	Origin: 'manual', 'loop', 'granulo'	<code>str</code>
<code>avatar[i].generation_type</code>	'regular', 'full', 'bevel' or <code>None</code>	<code>str</code> or <code>None</code>
<code>avatar[i].is_hollow</code>	Hollow disk	<code>bool</code>
<code>avatar[i].nb_vertices</code>	Number of vertices (polygon)	<code>int</code> or <code>None</code>
<code>avatar[i].vertices</code>	List of vertices	<code>list</code> or <code>None</code>
<code>avatar[i].axis</code>	Axes of the jonc {axe1, axe2, axe3}	<code>dict</code> or <code>None</code>

Expression	Description	Returned Type
<code>avatar[i].contactors</code>	List of contactors	<code>list</code>
<code>avatar[i].wall_params</code>	Raw wall parameters	<code>dict</code>
<code>avatar[i].brick_lx</code>	Masonry brick length (<code>wall_params['l']</code>)	<code>float</code> or <code>None</code>
<code>avatar[i].brick_ly</code>	Brick height/depth (<code>wall_params['h']</code>)	<code>float</code> or <code>None</code>
<code>avatar[i].brick_lz</code>	3D brick height (<code>wall_params['lz']</code>)	<code>float</code> or <code>None</code>
<code>avatar[i].mesh_params</code>	FE mesh parameters	<code>dict</code> or <code>None</code>
<code>avatar[i].index</code>	Index of the avatar in the list	<code>int</code>

pylmgc90 Nodes — `avatar[i].nodes`

The `nodes` property simulates access to pylmgc90 nodes. The pylmgc90 convention numbers nodes starting from 1 — `nodes[1]` is the main node, identical to `nodes[0]` in Python.

Expression	Description
<code>avatar[i].nodes[1].coor</code>	Coordinates of the main node <code>[x, y]</code> or <code>[x, y, z]</code>
<code>avatar[i].nodes[1].coor[0]</code>	X coordinate of the node
<code>avatar[i].nodes[1].coor[1]</code>	Y coordinate of the node
<code>avatar[i].nodes[1].coor[2]</code>	Z coordinate of the node (3D)
<code>avatar[i].nodes[0].coor</code>	Identical to <code>nodes[1]</code> (Python convention)
<code>avatar[i].nodes[k].coor</code>	k-th vertex for polygons ($k \geq 1$)

Why `nodes[1]`? pylmgc90 numbers nodes starting from 1. LMGC90_GUI supports both conventions: `nodes[0]` and `nodes[1]` both point to the main node.

Iterating Over Avatars

```
nb = len(avatar) # Total number of avatars
liste = list(avatar) # All avatars as a list of proxies
centres = [av.center for av in avatar] # List of all centers
rayons = [av.radius for av in avatar if av.radius is not None]
```

Groups — `group['name']`

Access to a group of avatars by its name. Returns a list of `AvatarProxy`.

Expression	Description
<code>group['mur_briques']</code>	List of avatars in the group
<code>group['mur_briques'][0].center</code>	Center of the first avatar in the group
<code>group['mur_briques'][0].x</code>	X coordinate of the first avatar
<code>len(group['mur_briques'])</code>	Number of avatars in the group
<code>'mur_briques' in group</code>	Test for the group's existence (bool)
<code>list(group)</code>	List of all group names

Examples:

```
nb_briques = len(group['mur_facade'])
premier_x = group['mur_facade'][0].x
dernier_y = group['mur_facade'][-1].y
all_centers = [av.center for av in group['granulo_box2d']]
```

Materials — `material['name']`

Expression	Description	Type
<code>material['beton'].name</code>	Material name	<code>str</code>
<code>material['beton'].density</code>	Density (kg/m³)	<code>float</code>
<code>material['beton'].material_type</code>	Type ('RIGID', 'ELAS'...)	<code>str</code>
<code>material['beton']['young']</code>	Custom property (Young's modulus)	<code>float</code>
<code>material['beton']['nu']</code>	Custom property (Poisson's ratio)	<code>float</code>
<code>material['beton'].young</code>	Same, via attribute	<code>float</code>

Examples:

```
rho = material['granite'].density
E = material['acier']['young']
nu = material['acier'].nu
```

Models — `model['name']`

Expression	Description	Type
<code>model['rigid'].name</code>	Model name	<code>str</code>
<code>model['rigid'].physics</code>	Physics ('MECAx', 'THERx'...)	<code>str</code>
<code>model['rigid'].element</code>	Finite element ('Rxx2D', 'Q4xxx'...)	<code>str</code>
<code>model['rigid'].dimension</code>	Dimension (2 or 3)	<code>int</code>
<code>model['femxx']['kinematic']</code>	Numeric option	depends on the option

Example:

```
dim = model['femxx'].dimension
phys = model['rigid'].physics
```

Filtering Functions

Return a list of `AvatarProxy` filtered according to a criterion.

Function	Description	Example
<code>avatars_by_color('BLUEx')</code>	All avatars of color 'BLUEx'	<code>bleus = avatars_by_color('BLUEx')</code>
<code>avatars_by_material('beton')</code>	All avatars using this material	<code>corps = avatars_by_material('TDURx')</code>
<code>avatars_by_type('rigidDisk')</code>	All avatars of this pylmgc90 type	<code>disques = avatars_by_type('rigidDisk')</code>
<code>avatars_by_origin('manual')</code>	Manually created avatars	<code>manuels = avatars_by_origin('manual')</code>

Function	Description	Example
<code>avatars_by_origin('loop')</code>	Avatars generated by a loop	
<code>avatars_by_origin('granulo')</code>	Granular avatars	

Advanced examples:

```
nb_bleus = len(avatars_by_color('BLUEx'))
nb_disques_rouges = len([av for av in avatars_by_color('REDxx') if av.avatar_type == 'rigidDisk'])
rayon_moyen = sum(av.radius for av in avatars_by_type('rigidDisk')) /
len(avatars_by_type('rigidDisk'))
x_min = min(av.x for av in avatars_by_color('BLUEx'))
x_max = max(av.x for av in avatars_by_color('BLUEx'))
```

Usage by Tab

Dynamic variables can be used in **every field** of LMGC90_GUI's tabs. Simply write the variable name (or a full expression) in place of the numeric value.

Material Tab

Field	Accepts Expressions
Density	■ — e.g.: <code>material['granite'].density * 0.9</code>
Properties (<code>key=val</code> , <code>key=val</code>)	■ — e.g.: <code>young=E</code> , <code>nu=nu_beton</code>

Examples:

```
Density : rho
Properties : young=E, nu=nu, elas='standard', anisotropy='isotropic'
```

Empty Avatar Tab

Field	Accepts Expressions
Center	■ — e.g.: <code>avatar[0].x + spacing</code> , <code>avatar[0].y</code>
Contacteur parameters	■ — e.g.: <code>r=radius</code> , <code>axe1=lx/2</code> , <code>axe2=ly/2</code>

Examples:

```
Center : avatar[0].x + spacing, 0.0
Parameters (DISKx) : r=radius
Parameters (JONCx) : axe1=brick_lx/2, axe2=brick_ly/2
Parameters (POLYG) : nb_vertices=6, vertices=sommets
```

Contact Tab

All numeric parameters of all contact laws accept expressions:

Parameter	Example Expression
<code>fric</code>	<code>mu</code> or <code>0.5 * material['beton'].density / 2500</code>
<code>stfr</code> , <code>dyfr</code>	<code>E * 1e3</code>
<code>cohn</code> , <code>coht</code>	<code>cohesion_sol</code>
<code>cn</code> , <code>ct</code>	<code>resistance_traction</code>
<code>stiffness</code>	<code>EA_cable</code>

Examples:

```
fric : mu
stiffness : EA_cable
cn : sigma_c
```

Loops Tab — Generic for Loops

In generic **for** loops, **every expression in the template** has access to the dynamic variables. The bounds (**start**, **end**, **step**) and all template fields are evaluated with the full context:

```
Start : 0
End : nb_cols
Step : 1
center (avatar template) : [i * (lx + joint) + lx/2, offset_y]
radius (avatar template) : r_min + i * (r_max - r_min) / nb_cols
name (material template): 'mat' + str(i)
```

Loops Tab — Geometric Loops

Geometric loop parameters (Circle, Grid, Line, Spiral) can use dynamic variables via the **for** loops form:

```
radius (Circle) : rayon_anneau
step (Grid) : pas_grille
```

Wizards — Deformable Wizard (DOF page)

In the Boundary Conditions page of the deformable wizard, the Parameters field uses `SafeEvaluator.eval_dict()`:

```
component=[1,2], dofty="vlocy", ct=vitesse_imposee
component=[2], ct=deplacement / duree_rampe
```

Supported Advanced Expressions

Operators

Operator	Example
<code>+, -, *, /</code>	<code>lx + joint</code>
<code>//</code>	Integer division
<code>%</code>	Modulo
<code>**</code>	Power: <code>radius**2</code>
<code>==, !=, <, <=, >, >=</code>	Comparisons
<code>and, or, not</code>	Logic

Ternary Expressions

```
r = r_grand if grand else r_petit
val = x if x > 0 else 0.0
```

List Comprehensions

```
centres = [av.center for av in group['mur']]
rayons = [av.radius for av in avatars_by_type('rigidDisk')]
xs = [av.x for av in avatar]
```

Lists and Tuples

```
center_2d = [avatar[0].x, avatar[0].y]
center_3d = [0.0, 0.0, avatar[0].z + 0.5]
```

numpy Expressions

```
moyenne = np.array([av.x for av in avatar]).mean()
coords = np.array([av.center for av in avatar])
```

Comparison with an Equivalent Python Script

The following table shows the equivalence between a dynamic variable in the interface and its Python code in a pre-processing script.

Dynamic Variable (interface)	Python Script Equivalent
<code>r = avatar[0].radius</code>	<code>r = bodies[0].nodes[1].coor[...]</code>
<code>cx = avatar[0].x</code>	<code>cx = bodies[0].nodes[1].coor[0]</code>
<code>cy = avatar[1].center[1]</code>	<code>cy = bodies[1].nodes[1].coor[1]</code>
<code>cx = avatar[0].nodes[1].coor[0]</code>	<code>cx = bodies[0].nodes[1].coor[0]</code>
<code>rho = material['beton'].density</code>	<code>rho = mats['beton'].density</code>
<code>E = material['acier']['young']</code>	<code>E = mats['acier'].young</code>
<code>phys = model['rigid'].physics</code>	<code>phys = mods['rigid'].physics</code>
<code>nb = len(avatar)</code>	<code>nb = len(bodies)</code>
<code>nb_g = len(group['mur'])</code>	<code>nb_g = len(group_mur)</code>
<code>x0 = group['mur'][0].x</code>	<code>x0 = bodies[group_mur[0]].nodes[1].coor[0]</code>
<code>bleus = avatars_by_color('BLUEx')</code>	<code>bleus = [b for b in bodies_list if b.color == 'BLUEx']</code>

Complete Example

Interface — Dynamic Variables:

```
lx = 0.20
ly = 0.065
joint = 0.010
nb_cols = 15
nb_rows = 10
spacing = lx + joint
offset_x = 0.0
wall_width = nb_cols * spacing
```

Equivalent pre-processing Python script:

```
lx = 0.20
ly = 0.065
joint = 0.010
nb_cols = 15
```

```

nb_rows = 10
spacing = lx + joint
offset_x = 0.0
wall_width = nb_cols * spacing

for row in range(nb_rows):
    for col in range(nb_cols):
        cx = offset_x + col * spacing + lx / 2.0
        cy = row * (ly + joint) + ly / 2.0
        body = pre.rigidDisk(center=[cx, cy], ...)
        bodies.addAvatar(body)

```

AvatarProxy Object Properties — Complete Reference

The following table lists **all the properties** accessible via `avatar[i]` or within a group list.

Property	Type	Notes
<code>.center</code>	<code>list[float]</code>	<code>[x, y]</code> in 2D, <code>[x, y, z]</code> in 3D
<code>.x</code>	<code>float</code>	<code>center[0]</code>
<code>.y</code>	<code>float</code>	<code>center[1]</code>
<code>.z</code>	<code>float</code> or <code>None</code>	<code>center[2]</code> or <code>None</code> if 2D
<code>.radius</code>	<code>float</code> or <code>None</code>	Radius of the disk/sphere
<code>.color</code>	<code>str</code>	5-character color code ('BLUEX')
<code>.material_name</code>	<code>str</code>	Name of the associated material
<code>.model_name</code>	<code>str</code>	Name of the associated model
<code>.avatar_type</code>	<code>str</code>	Enum value: 'rigidDisk', 'rigidSphere', 'rigidJonc', 'emptyAvatar'...
<code>.origin</code>	<code>str</code>	'manual', 'loop', 'granulo'
<code>.generation_type</code>	<code>str</code> or <code>None</code>	'regular', 'full', 'bevel'
<code>.is_hollow</code>	<code>bool</code>	Hollow disk if <code>True</code>
<code>.nb_vertices</code>	<code>int</code> or <code>None</code>	Number of vertices of the polygon
<code>.vertices</code>	<code>list</code> or <code>None</code>	Vertex coordinates <code>[[x1,y1], ...]</code>
<code>.axis</code>	<code>dict</code> or <code>None</code>	<code>{'axe1': v, 'axe2': v, 'axe3': v}</code> for joncs/planes
<code>.contactors</code>	<code>list[dict]</code>	Each dict: <code>{'shape', 'color', 'params'}</code>
<code>.wall_params</code>	<code>dict</code>	Masonry wall parameters
<code>.brick_lx</code>	<code>float</code> or <code>None</code>	Brick length (<code>wall_params['l']</code>)
<code>.brick_ly</code>	<code>float</code> or <code>None</code>	Brick height/depth (<code>wall_params['h']</code>)
<code>.brick_lz</code>	<code>float</code> or <code>None</code>	3D brick height (<code>wall_params['lz']</code>)
<code>.mesh_params</code>	<code>dict</code> or <code>None</code>	FE mesh parameters
<code>.index</code>	<code>int</code>	Index in <code>state.avatars</code>
<code>.nodes[1].coor</code>	<code>list[float]</code>	Main node (pylmgc90 convention)
<code>.nodes[0].coor</code>	<code>list[float]</code>	Same (Python convention)

Property	Type	Notes
<code>.nodes[k].coor</code>	<code>list[float]</code>	k-th vertex (polygons)

Security — What Is Allowed and Forbidden

Expressions are parsed via `SafeEvaluator`, which inspects the Python AST before execution. No direct `eval()` is used.

Allowed

- Arithmetic operations, comparisons, logic
- Calls to functions from the context (`sqrt`, `math.sin`, `len`, `list`...)
- List, tuple, and dict comprehensions
- Ternary expressions (`a if cond else b`)
- Attribute and index access
- Conversions: `int(x)`, `float(x)`, `str(x)`, `bool(x)`

Forbidden (blocked by SafeEvaluator)

- `import`, `exec`, `eval`, `open`, `__import__`
- Any statement (assignment, `for/while` loop, `if` — only expressions are allowed)
- Calls to functions not declared in the context
- Access to `__builtins__`, `__class__`, `__dict__`

If an expression attempts a forbidden operation, `SafeEvaluator` raises a `ValueError` with the message "Opération non autorisée : <ASTNodeName>" ("Operation not allowed").

Error Messages and Debugging

When an expression fails in a tab, a detailed dialog box lists:

- The faulty expression
- The Python error message
- The dynamic variables currently defined
- The available references (`avatar[i].x`, `group['name']`, etc.)

Common causes:

Error	Cause	Solution
<code>NameError: 'thickness'</code>	Variable not yet defined	Create <code>thickness</code> in the manager
<code>IndexError: Avatar index 5 invalide</code>	There are fewer than 6 avatars	Check the index
<code>KeyError: Groupe 'mur' introuvable</code>	The group does not exist	Check the name in the Loops/Masonry tab
<code>Opération non autorisée : Import</code>	Attempted import	Use only the available functions
<code>Syntaxe invalide</code>	Unclosed parentheses	Check the Python syntax

Ready-to-Use Dynamic Variable Examples

Masonry Wall Geometry

```
lx = 0.20 # Brick length
ly = 0.065 # Brick height
lz = 0.10 # Brick depth (3D)
joint = 0.010 # Joint thickness
nb_cols = 15 # Number of columns
nb_rows = 10 # Number of rows
spacing_x = lx + joint
spacing_y = ly + joint
wall_width = nb_cols * spacing_x
wall_height = nb_rows * spacing_y
```

Granular Parameters

```
r_min = 0.05
r_max = 0.15
ratio = r_max / r_min
lx_box = 4.0
ly_box = 4.0
nb_particules = 200
```

Coordinates Relative to an Existing Avatar

```
x_ref = avatar[0].x
y_ref = avatar[0].y
x_cible = x_ref + 1.0
y_cible = y_ref + 0.5
dist = sqrt((avatar[1].x - avatar[0].x)**2 + (avatar[1].y - avatar[0].y)**2)
```

Material Parameters

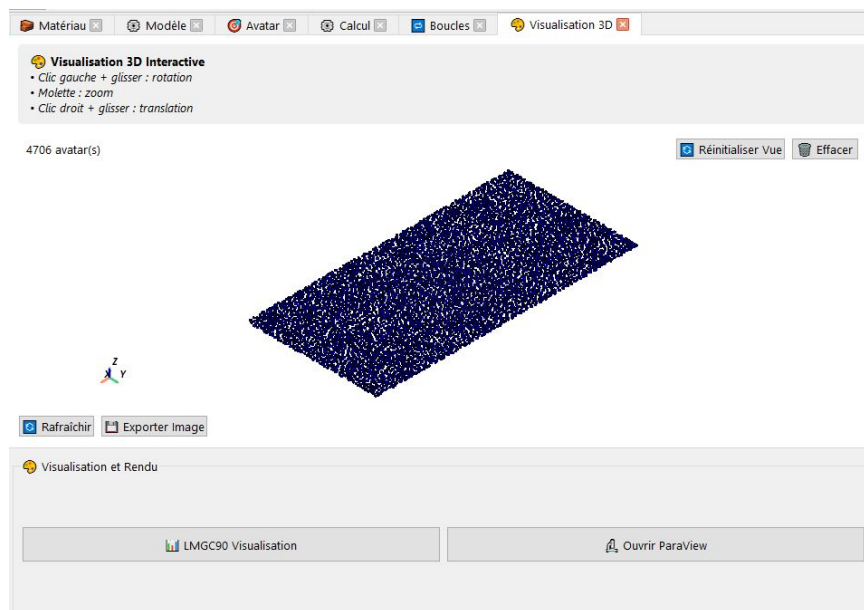
```
rho = material['granite'].density
E = material['acier']['young']
nu = material['acier']['nu']
mu = 0.3 # Coulomb coefficient
K = E / (3 * (1 - 2 * nu)) # Bulk modulus
G = E / (2 * (1 + nu)) # Shear modulus
```

Avatar Statistics

```
nb_total = len(avatar)
nb_granulo = len(avatars_by_origin('granulo'))
nb_manuels = len(avatars_by_origin('manual'))
rayon_moyen = sum(av.radius for av in avatars_by_type('rigidDisk')) / max(1, nb_total)
x_centre_masse = sum(av.x for av in avatar) / max(1, nb_total)
```

Visualization

In addition to the visualization tool built into LMGC90, which is none other than the preprocessor function `pre.visuAvatars()`, LMGC90_GUI allows you to visualize your avatars in a 3D space completely independent of the solver. However, it is currently in its *beta* version and is therefore very resource-intensive.



captures/visual.JPG

LMGC90_GUI — Architecture & Contributor Guide

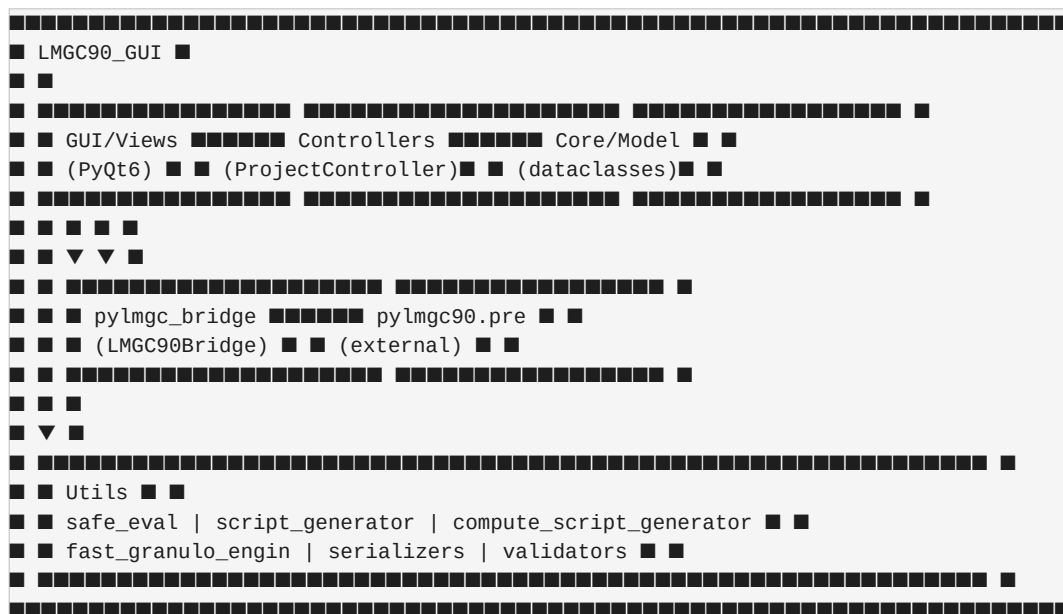
Version 0.4.0 — Graphical interface for the LMG90 mechanical simulation code. The current architecture will be replaced in version 0.5.0.

Table of Contents

1. Overview
2. File Structure
3. Core Layer (Model)
4. Controllers Layer
5. GUI / Views Layer
6. Utils Layer
7. Data Flow
8. Key Systems Explained
9. Project Life Cycle
10. Conventions and Patterns
11. Contribution Guide

1. Overview

LMGC90_GUI is a **PyQt6** desktop application following the **MVC** (Model-View-Controller) pattern. It allows you to create LMG90 mechanical simulations through a graphical interface without manually writing Python code.



Main Dependencies

Dependency	Role
PyQt6	UI framework (widgets, signals/slots, threads)
pymgc90	LMGC90 library (pre-processing, simulation)
numpy	Vectorized computations (granulometry, positions)
pyvista / pyvistaqt	3D visualization of avatars
gmsh (optional)	Meshing of geometries for deformable bodies

2. File Structure

```
lmgc90_gui/
├── main.py # Entry point, QApplication
├── src/
│   ├── core/ # Model Layer (MVC)
│   │   ├── models.py # Dataclasses: Material, Model, Avatar, ...
│   │   ├── validators.py # Data validation
│   │   ├── generators.py # LoopGenerator, GranuloGenerator
│   │   ├── serializers.py # JSON save/load (.lmgc90)
│   │   ├── pymgc_bridge.py # Conversion of models → pymgc90 objects
│   │   ├── particle_factory.py # Progressive generation engine
│   │   ├── avatar_factory.py # Predefined avatar templates
│   │   ├── app_logger.py # Application logger
│   │   └── workers/
│   │       └── granulo_worker.py # QThread for granulo generation
│   ├── controllers/
│   │   └── project_controller.py # Central controller (business logic)
│   └── views/ # View Layer (MVC)
│       ├── main_window.py # Main window (QMainWindow)
│       ├── tree_view.py # Model tree (QTreeWidget)
│       └── tabs/ # Working tabs
│           ├── base_tab.py # Base class with safe_eval
│           ├── material_tab.py # Material management
│           ├── model_tab.py # FE model management
│           ├── avatar_tab.py # Standard avatar management
│           ├── empty_avatar_tab.py # Empty avatars (manual contactors)
│           ├── loop_tab.py # Generation loops
│           ├── granulo_tab.py # Granulometric generation
│           ├── dof_tab.py # DOF boundary conditions
│           ├── contact_tab.py # Contact laws
│           ├── visibility_tab.py # Visibility tables
│           ├── postpro_tab.py # Post-processing
│           └── viewer_tab.py # 3D visualization tab wrapper
│           ...
└── gui/
    ├── dialogs/ # Dialogs and wizards
    ├── dialogs.py # DynamicVarsDialog, PreferencesDialog, DuplicateDialog
    ├── setup_wizard.py # Project wizard
    ├── factory_wizard.py # Particle Factory wizard (+ FactoryTab)
    ├── granulo_wizard.py # Granulometry wizard
    ├── mesh_wiz_def.py # Deformable body wizard (FEM)
    ├── masonry_wizard.py # Masonry wizard
    └── fast_granulo_dialog.py # Fast numpy generation dialog
```

```

■ ■ ■■ viewer_3d.py # PyVista widget (3D visualization)
■ ■ ■■ chipy_routines_dialog.py # chipy routine config
■ ■ ■■ app_log_dialog.py # Log visualization
■ ■ ■■ convert_dialog.py # pylmgc90 script conversion
■ ■
■ ■■ ■■ utils/
■ ■■ ■■ safe_eval.py # Secure evaluator + project context proxies
■ ■■ ■■ script_generator.py # pre.py script generation
■ ■■ ■■ compute_script_generator.py # chipy script generation (command.py)
■ ■■ ■■ fast_granulo_engin.py # High-perf numpy granulo engine
■ ■■ ■■ convert.py # pylmgc90 script → .lmgc90 converter

```

3. Core Layer (Model)

3.1 models.py — The Dataclasses

All project entities are **Python dataclasses**. They are the source of truth.

```

# Model hierarchy
ProjectState
■■■ List[Material] # Materials (RIGID, ELAS, ...)
■■■ List[Model] # FE models (Rxx2D, T3xxx, ...)
■■■ List[Avatar] # Rigid/deformable bodies
■■■ List[ContactLaw] # Contact laws (IQS_CLB, ...)
■■■ List[VisibilityRule] # Visibility tables
■■■ List[DOFOperation] # Boundary conditions
■■■ List[Loop] # Geometric loops (circle, grid...)
■■■ List[ForLoop] # Generic for loops (JSON template)
■■■ List[GranuloGeneration] # Granulometric deposits
■■■ List[PostProCommand] # Post-processing commands
■■■ Dict[str, List[int]] # Avatar groups (name → indices)
■■■ Dict[str, Any] # Dynamic variables (expressions)
■■■ List[dict] # Factories (serialized FactoryConfig)
■■■ ProjectPreferences # User preferences

```

Each **dataclass** exposes `to_dict()` and `from_dict()` for JSON serialization. Example:

```

@dataclass
class Avatar:
    avatar_type: AvatarType # Enum: rigidDisk, roughWall, ...
    center: List[float]
    material_name: str
    model_name: str
    color: str = "BLUEx"
    origin: AvatarOrigin = AvatarOrigin.MANUAL
    radius: Optional[float] = None
    # ... other type-specific fields

```

Important Enums:

Enum	Notable Values
AvatarType	RIGID_DISK, ROUGH_WALL, MESH_DEFORMABLE, EMPTY_AVATAR, ...
AvatarOrigin	MANUAL, LOOP, GRANULO
MaterialType	RIGID, ELAS, ELAS_PLAS, ...
ContactLawType	IQS_CLB, MAC_CZM, ELASTIC_WIRE, ...

Enum	Notable Values
UnitSystem	SI, CGS

3.2 validators.py — Validation

Each entity has its own validator:

```
MaterialValidator.validate_or_raise(material) # Name ≤ 5 chars, density > 0
ModelValidator.validate_or_raise(model) # Element compatible with physics+dim
AvatarValidator.validate_or_raise/avatar, model) # Parameters based on type
ContactLawValidator.validate_or_raise(law) # Required properties
```

The validators raise `ValidationError` (inherits from `Exception`), caught by the View to display a `QMessageBox`.

3.3 pylmgc_bridge.py — Bridge to pylmgc90

`LMGC90Bridge` is a **static** class that converts the dataclasses into `pylmgc90.pre` objects:

```
LMGC90Bridge.create_material(material) → pre.material(...)
LMGC90Bridge.create_model(model) → pre.model(...)
LMGC90Bridge.create_avatar/avatar, mod, mat) → pre.rigidDisk(...) / pre.avatar(...)
LMGC90Bridge.create_contact_law(law) → pre.tact_behav(...)
LMGC90Bridge.create_visibility_rule(rule, b) → pre.see_table(...)
LMGC90Bridge.apply_dof_operation(op, body) → body.translate(...) / body.imposeDrivenDof(...)
```

Complex Cases Handled:

- Deformable bodies: `pre.buildMesh2D`, `pre.buildMeshH8`, `pre.readMesh` + `pre.buildMeshedAvatar`
- Masonry bricks: `pre.brick2D/3D` + `brick.rigidBrick(...)`
- Empty avatars with contactors: step-by-step creation via `pre.avatar()` → `addBulk` → `addNode` → `addContactors`

3.4 generators.py — Position Generation

```
LoopGenerator.generate_positions(loop: Loop) → List[[x, y]]
# Dispatches to: generate_circle / generate_grid / generate_line / generate_spiral

GranuloGenerator.generate(config) → (nb_particles, coordinates_array, radii_array)
# Calls pre.granulo_Random then pre.depositInBox2D / depositInDisk2D / ...
```

3.5 particle_factory.py — Particle Factory

A **progressive** particle generation system (inspired by EDEM):

- `FactoryConfig`: complete dataclass (type, zone, schedule, container)
- `ParticleFactory`: engine (validates, assigns body indices, generates code)
- `PreCodeGenerator`: generates the `pre.py` block (invisible creation + schedule)
- `ChipyCodeGenerator`: generates the `chipy.py` block (wave activation)

3.6 serializers.py

```
ProjectSerializer.save(state, filepath) # JSON → .lmgc90 file
ProjectSerializer.load(filepath) # .lmgc90 file → ProjectState
```

The `.lmgc90` format is **pure JSON**. Only avatars with `origin == MANUAL` are serialized; generated avatars (loops, granulo) are regenerated on loading.

4. Controllers Layer

4.1 `project_controller.py` — The Central Controller

`ProjectController(QObject)` is the **heart of the application**. It:

1. Maintains the project state (`self.state: ProjectState`)
2. Maintains the `pylmgc90` objects in memory:

```
self._materials_container # pre.materials()
self._models_container # pre.models()
self._bodies_container # pre.avatars()
self._contact_laws_container # pre.tact_behavs()
self._visibility_container # pre.see_tables()
self._postpro_container # pre.postpro_commands()

self._pylmgc_materials: Dict[str, Any] # name → pylmgc90 object
self._pylmgc_models: Dict[str, Any]
self._pylmgc_bodies: List[Any] # indexed like state.avatars
self._pylmgc_laws: Dict[str, Any]
```

3. Emits `state_changed = pyqtSignal()` on every modification

Fundamental invariant: `self._pylmgc_bodies[i]` always corresponds to `self.state.avatars[i]`.

Controller API (main methods)

```
# Project
controller.new_project(name)
controller.save_project(filepath?)
controller.load_project(filepath) # → rebuilds everything via _rebuild_pylmgc_objects()

# Material CRUD
controller.add_material(material) # validates + creates pylmgc object + state
controller.update_material(old_name, m) # updates refs in avatars
controller.remove_material(name)

# Model CRUD (identical)
# Avatar CRUD
controller.add_avatar/avatar, create_pylmgc=True) # create_pylmgc=False for performance
controller.update_avatar(index, avatar)
controller.remove_avatar(index)
controller.duplicate_avatar(index, n, offset, group?)
controller.duplicate_group(group_name, n, offset, prefix?)

# Generation
controller.generate_loop(loop) # → creates avatars + adds to the group
controller.generate_granulo(config) # → via GranuloGenerator
controller.generate_for_loop(for_loop) # → generic for loop

# DATBOX
controller.generate_datbox(output_path) # → pre.writeDatbox(...)
```

`_rebuild_pylmgc_objects()` — Reconstruction on Loading

When loading a project, the order is **strict**:

1. Materials → models → MANUAL avatars → loops → granulo → For loops
2. Contact laws → visibility → DOF

If an error occurs (e.g.: missing material), it is stored in `state.load_warnings` and displayed in the UI.

Batch Mode

```
self._batch_mode = True # Disables state_changed.emit() during creation
# ... create N avatars ...
self._batch_mode = False
self.state_changed.emit() # A single signal at the end
```

5. GUI / Views Layer

5.1 `main_window.py` — Main Window

`MainWindow(QMainWindow)` orchestrates everything:

```
MainWindow
■■■ MenuBar # File, Wizards, Tools, Computation, Tabs, Help
■■■ ToolBar # New, Open, Save, DATBOX, Script
■■■ DockWidget (left)
■ ■■■ ModelTreeView # QTreeWidget tree
■■■ Central (vertical QSplitter)
■■■ QTabWidget # Working tabs (70%)
■ ■■■ MaterialTab
■ ■■■ ModelTab
■ ■■■ AvatarTab
■ ■■■ ...
■■■ QWidget (bottom 30%) # LMGC90 Viz + ParaView buttons
```

Tab management: Tabs can be opened/closed dynamically. `material_tab` and `model_tab` are **essential** (cannot be closed). Each tab is instantiated only once and hidden/shown.

Signal connections:

```
# Each tab emits signals → MainWindow._refresh_all()
self.material_tab.material_created.connect(self._refresh_all)
# _refresh_all() calls tree_view.refresh() + tab.refresh() on all tabs
```

5.2 `tree_view.py` — Model Tree

`ModelTreeView(QObject)` manages a `QTreeWidget` displaying the complete structure:

```
LMGC90 Model
■■■ Materials (N)
■■■ Models (N)
■■■ Avatars (N) [filtered according to the show_granulo_individually preference]
■■■ Avatar Groups
■■■ Contact Laws
■■■ Visibility Tables
■■■ DOF Operations
■■■ Loops
■■■ Granulo Deposits
■■■ Post-Processing
```

Signal emitted: `item_selected = pyqtSignal(str, object)` → element type + data. `MainWindow` receives this signal and loads the element into the appropriate tab.

Context menu: Right-click on Avatar → `DuplicateDialog`. Right-click on Group → duplication of the entire group.

5.3 `base_tab.py` — Base Tab Class

All tabs inherit from `BaseTab(QWidget)`. It provides:

```
# Secure evaluation of expressions (uses SafeEvaluator)
self.eval_float(text, default, field_name) # "0.5 * pi" → 1.5707...
self.eval_int(text, default, field_name)
self.eval_list(text, expected_length, field_name) # "1.0, 2.0" → [1.0, 2.0]
self.eval_dict(text, field_name) # "k=1, nu=0.3" → {"k": 1, "nu": 0.3}

# Contextual help label
self.add_expression_help_label(layout)
```

The evaluator gives access to the **full project context** in the form fields: `avatar[0].x`, `group['mur'][0].radius`, `material['acier'].density`, etc.

5.4 The Tabs (tabs/)

Each tab follows the same **CRUD pattern**:

```
Tab
■■■ QTreeWidget # List of existing elements
■■■ Buttons (=■ Edit, ■■ Delete)
■■■ QFormLayout # Creation/editing form
■■■ Buttons (■ Create, ■ Save, ■ Cancel, ■ Reset)
■■■ Signals emitted # element_created, element_updated, element_deleted
```

Methods to implement in each tab:

```
def _setup_ui(self) # Interface construction
def _connect_signals(self) # Signal/slot connection
def _on_create(self) # Create an element
def _on_edit_from_tree(self) # Load for editing from the tree
def _on_update(self) # Save the modifications
def _on_delete(self) # Delete
def load_for_edit(self, ...) # Fill the form from an object
def refresh(self) # Refresh the display
```

`avatar_tab.py` — Avatar Tab

Manages 18+ types of 2D/3D avatars. The `_on_type_changed()` method dynamically shows/hides fields based on the selected type. `_build_avatar_from_form()` builds the `Avatar` object from the visible fields.

`model_tab.py` — Model Tab

Complex management of options depending on the physics (MECAx/THERx/POROx/MULTI) and the element. `_on_element_changed()` dynamically rebuilds the option combo boxes.

`granulo_tab.py` — Granulometric Generation

Uses a `QThread` (`GranuloWorker`) for the computations. Avatar creation is done in **progressive batches** via a `QTimer` so as not to block the UI:

```

_on_generate() → GranuloWorker.run() → data_ready signal
↓
_on_data_ready() → QTimer(0ms)
↓
_create_next_avatar() × N [batches of 50-100]
↓
_on_creation_completed()

```

5.5 viewer_3d.py — 3D Visualization

Viewer3D(QWidget) wrapping `pyvistaqt.QtInteractor`. **Never** refreshes automatically, to avoid freezes. The user clicks "■ Refresh the scene".

Mesh construction:

```

build_avatar_mesh/avatar) → pv.PolyData
# Dispatches to _MESH_BUILDERS[avatar.avatar_type]
# E.g.: _mesh_rigid_disk → pv.Circle().extrude(h)
# _mesh_rigid_polygon → pv.PolyData(vertices)
# _mesh_deformable → rebuilt from mesh_params (geom)

```

Color modes: LMGC90 (by color code) | By type | By material | By origin **Interaction modes:** Navigation | Selection (avatar_clicked signal) | Ruler (distance measurement)

5.6 The Wizards (dialogs/)

Multi-page **QWizard** assistants for complex tasks:

Wizard	Pages	Result
ProjectSetupWizard	Project → Dim → Mat → Mod → Avatar → Contact → Visibility → Summary	Fully initialized project
GranuloWizard	Distribution → Container → Properties → Summary	Generated granulo deposit
MeshWizard	Intro → Dim → Mat → Mod → Geom → Refinement → Boundary → Summary	FEM deformable body
MasonryWizard	Config → Model → Summary	Masonry wall
FactoryWizard	Intro → Zone → Particles → Container → Schedule → Summary	Configured factory

6. Utils Layer

6.1 safe_eval.py — Secure Evaluation

Problem solved: Allow users to enter Python expressions (`avatar[0].x + 0.1, sqrt(2) * radius`) in the forms without security risk.

Architecture:

```

# 1. SafeEvaluator – AST checking + isolated eval
ev = SafeEvaluator(allowed_names=context_dict)
result = ev.eval_expression("avatar[0].x * 2")

# 2. Proxies – access to project data

```

```

AvatarCollectionProxy # avatar[i], len(avatar), list(avatar)
AvatarProxy # .center, .x, .y, .z, .radius, .nodes[1].coor, ...
GroupProxy # group['name'][0].center
MaterialProxy # material['acier'].density
ModelProxy # model['rigid'].physics

# 3. build_eval_context() – builds the full context
ctx = build_eval_context(controller)
# ctx contains: math, np, avatar, group, material, model,
# avatars_by_color(), avatars_by_material(), ...
# + all the project's dynamic variables

```

Security: `SafeEvaluator._check_safe()` walks the AST and rejects any disallowed node (import, exec, dangerous attributes, etc.).

6.2 `script_generator.py` — `pre.py` Script Generation

`ScriptGenerator(controller)` generates a Python script reproducing the project:

```

generate(output_path)
■■■ _write_header() # Header comment
■■■ _write_imports() # from pylmgc90 import pre, numpy, math
■■■ _write_dynamic_vars() # Evaluation and injection of dynamic vars
■■■ _write_containers() # mats, mods, bodies, tacts, sees, posts
■■■ _write_materials() # mat_NAME = pre.material(...)
■■■ _write_models() # mod_NAME = pre.model(...)
■■■ _write_avatars_manual() # MANUAL avatars (with loop or individual option)
■ ■■■ _write_avatars_manual_loop() # Loop mode (by groups)
■ ■■■ _write_masonry_group_loop() # Masonry brick loops
■ ■■■ _write_standard_group_loop() # for loop + list of centers
■ ■■■ _write_single_avatar() # Individual avatar
■■■ _write_for_loops() # Generic For loops
■■■ _write_loops() # Geometric loops (circle, grid...)
■■■ _write_granulo() # pre.granulo_Random + depositIn*
■■■ _write_contact_laws() # pre.tact_behav(...)
■■■ _write_visibility() # pre.see_table(...)
■■■ _write_dof_operations() # body.translate / imposeDrivenDof
■■■ _write_postpro() # pre.postpro_command(...)
■■■ _write_factories() # Particle Factories (pre-computed code)
■■■ _write_datbox() # pre.writeDatbox(...)

```

`script_use_loop` preference: If enabled, avatars from the same homogeneous group are grouped into a `for _c in _centers_<group>`: loop for a more compact script.

6.3 `compute_script_generator.py` — `chipy` Script Generation

`ComputeScriptGenerator(controller)` generates `command.py` (simulation loop) according to the parameters configured in `ChipyRoutinesDialog`.

Generated sections:

1. `chipy` configuration (`SetDimension`, `ReadDatbox`, ...)
2. Initialization of the Particle Factories (invisibility + schedule)
3. Main loop `for k in range(nb_steps)`:
 - FreeVelocity RBDY2/RBDY3/FEM
 - Contact detection (selected detectors)
 - NLGS resolution

- Factory wave activation
 - ComputeDof + UpdateStep
 - Extraction (energy, GBV, inspection)
 - WriteOut + WriteDisplayFiles
4. Finalization

6.4 fast_granulo_engin.py — Numpy Granulometry

A **pylmgc90-free**, fully numpy engine, for generating thousands of particles without blocking the UI:

```
GranuloFastEngine.generate(nb, rmin, rmax, container_type, ...)
# → Placement in batches (candidates → bbox filtering → vectorized collision filtering)
# → FastGranuloResult with List[FastParticle]

GranuloFileWriter.write(result, output_dir)
# → Writes DATBOX/BODIES.DAT directly (bypassing pylmgc90)

GranuloStateIntegrator.integrate(result, controller)
# → Adds the avatars in a single batch into controller.state
```

6.5 convert.py — Script Converter

Converts an existing **pre.py** script into a **.lmgc90** file by **executing the script** with a **_MockPre** module that intercepts all **pre.*** calls:

```
class _MockPre: # Replaces pylmgc90.pre during exec
class _AvatarObj # Proxy of a rigid avatar
class _MeshAvatarObj # Proxy of a deformable body
class _EmptyAvatarObj # Proxy of an empty avatar
class _BrickObj # Proxy of a masonry brick
class _WallObj # Proxy of a masonry wall

class Converter:
run() # Executes the script with the mock
to_lmgc90_dict() # Builds the project JSON

# CLI:
# python convert.py my_script.py -o output.lmgc90
```

7. Data Flow

7.1 Creating an Avatar (complete example)

```
User fills in AvatarTab → clicks "■ Create Avatar"
↓
AvatarTab._on_create()
→ _build_avatar_from_form() # Values → Avatar dataclass
→ controller.add_avatar/avatar)
↓
ProjectController.add_avatar()
→ AvatarValidator.validate_or_raise/avatar, model) # Validation
→ mat_obj = self._pylmgc_materials[avatar.material_name]
→ mod_obj = self._pylmgc_models[avatar.model_name]
→ body_obj = LMG90Bridge.create_avatar/avatar, mod_obj, mat_obj)
```

```

→ self._bodies_container.addAvatar(body_obj)
→ self._pylmgc_bodies.append(body_obj)
→ self.state.avatars.append(avatar)
→ self.state_changed.emit() # if not in batch_mode
↓
AvatarTab.avatar_created.emit()
↓
MainWindow._refresh_all()
→ tree_view.refresh() # Tree updated
→ [all tabs].refresh() # Combos updated

```

7.2 Saving and Loading

```

Saving:
ProjectController.save_project()
→ ProjectSerializer.save(state, filepath)
→ state.to_dict() # Only MANUAL avatars
→ json.dump(data, file) # .lmgc90 file

Loading:
ProjectController.load_project(filepath)
→ ProjectSerializer.load(filepath)
→ ProjectState.from_dict(data) # Dataclasses rebuilt
→ _rebuild_pylmgc_objects() # Strict-order reconstruction
1. Materials + Models # pylmgc90 objects recreated
2. MANUAL Avatars # bridge.create_avatar()
3. Loops → generate_loop() # LOOP avatars recreated
4. Granulo → generate_granulo() # GRANULO avatars recreated
5. For Loops → generate_for_loop()
6. Laws + Visibility + DOF

```

8. Key Systems Explained

8.1 Avatar Indexing

The index of an avatar in `state.avatars` is its identifier everywhere:

- `state.avatar_groups["mur"] = [0, 1, 2, 3]` → indices in `state.avatars`
- `state.operations[0].target_value = 5` → avatar #5
- `self._pylmgc_bodies[5]` → pylmgc90 object of avatar #5

■ **Caution when deleting:** `remove_avatar(index)` does a `pop(index)` which shifts all subsequent indices. Groups and operations referencing avatars with a higher index become invalid. This is a known limitation that will require a future redesign (stable IDs).

8.2 Dynamic Variables

Dynamic variables (`state.dynamic_vars`) are **Python expressions**:

```
{"thickness": "0.5", "radius": "thickness * 2 + 0.1", "x_wall": "avatar[0].x"}
```

They are evaluated in definition order and injected into `SafeEvaluator`'s context. Thus, in a form, the user can write `radius` and get the computed value.

8.3 Preferences System

`ProjectPreferences` is stored in `state.preferences` and saved with the project. Important preferences:

Preference	Impact
<code>show_granulo_individually</code>	Hides GRANULO avatars in the tree and tabs
<code>create_pylmgc_on_generate</code>	Disables pylmgc creation during massive generation
<code>script_use_loop</code>	Generates compact loops in the pre.py script
<code>auto_refresh_viewer</code>	(reserved) Automatic refresh of the 3D view

8.4 Particle Factory

Factories are persisted as `List[dict]` in `state.factories`. On loading and when generating the script, `ParticleFactory.from_list_of_dicts()` rebuilds the engine, recomputes the body indices, and generates the code blocks.

9. Project Life Cycle

```
1. NEW PROJECT
MainWindow._on_new_project()
→ controller.new_project(name)
→ _reset_containers() [empty pylmgc containers]
→ state = ProjectState(name)

2. CONFIGURATION
Materials Tab → add_material()
Models Tab → add_model()
Avatars Tab → add_avatar()
... (laws, visibility, DOF, loops, granulo)

3. SAVING
Ctrl+S → save_project()
→ state.to_dict() → JSON

4. DATBOX GENERATION
Tools → DATBOX → controller.generate_datbox(path)
→ pre.writeDatbox(dim, mats, mods, bodies, tacts, sees, post)

5. PRE.PY SCRIPT GENERATION
Tools → Python Script → ScriptGenerator.generate(path)

6. CHIPY SCRIPT GENERATION (command.py)
Computation → Generate Script → ComputeScriptGenerator.generate(path, params)

7. COMPUTATION
Computation → Run → compute_tab.run_computation()
→ Executes command.py in a subprocess
→ Displays the LMGC90 logs

8. LOADING
File → Open → controller.load_project(path)
→ ProjectState.from_dict() + _rebuild_pylmgc_objects()
```

10. Conventions and Patterns

Naming Conventions

Element	Convention	Example
Classes	PascalCase	AvatarTab, LMGC90Bridge
Methods	snake_case	_on_create(), load_for_edit()
Qt Slots	_on_ prefix	_on_type_changed()
Qt Signals	Descriptive suffix	avatar_created, state_changed
Private methods	_ prefix	_build_avatar_from_form()
pylmgc containers	_pylmgc_ prefix	_pylmgc_materials

Signal/Slot Pattern

```
# In a tab:
class AvatarTab(BaseTab):
    avatar_created = pyqtSignal() # Declaration at class level

    def _on_create(self):
        ...
    self.avatar_created.emit() # Emission after successful action

# In MainWindow:
self.avatar_tab.avatar_created.connect(self._refresh_all)
```

Error Handling

```
try:
    # Business logic
    avatar = self._build_avatar_from_form()
    self.controller.add_avatar(avatar)
    # Success
    QMessageBox.information(self, "Success", "■ Avatar created")
except ValidationError as e:
    QMessageBox.warning(self, "Validation", str(e))
except ValueError as e:
    QMessageBox.critical(self, "Error", f"Invalid values:\n{e}")
except Exception as e:
    QMessageBox.critical(self, "Error", f"Creation failed:\n{e}")
```

11. Contribution Guide

Adding a New Avatar Type

1. `models.py`: Add the value to `AvatarType`
2. `validators.py`: Add the validation in `AvatarValidator.validate()`
3. `pylmgc_bridge.py`: Add the case in `LMGC90Bridge.create_avatar()`
4. `avatar_tab.py`:
 - Add the type to `AVATAR_TYPES_2D` or `AVATAR_TYPES_3D`
 - Handle field display in `_on_type_changed()`
 - Build the avatar in `_build_avatar_from_form()`
5. `viewer_3d.py`: Add the mesh builder to `_MESH_BUILDERS`
6. `script_generator.py`: Handle the generation in `_write_single_avatar()`

Adding a New Tab

1. Create `src/views/tabs/my_tab.py` inheriting from `BaseTab`
2. Implement the complete CRUD pattern
3. Declare the necessary signals
4. Import it in `src/views/tabs/__init__.py`
5. Instantiate it in `MainWindow._create_tabs()`
6. Add it to `MainWindow.all_tabs`
7. Connect the signals in `MainWindow._connect_signals()`

Adding a New Contact Law

1. `models.py`: Add it to `ContactLawType` and the appropriate category in `CONTACT_LAW_CATEGORIES`
2. `validators.py`: Add the required properties to `ContactLawValidator._REQUIRED_PROPS`
3. `pymgc_bridge.py`: Add the case in `create_contact_law()`
4. `contact_tab.py`: Add the UI fields in `_on_type_changed()` and `_build_law_from_form()`

Adding a Preference Parameter

1. `models.py`: Add the field to `ProjectPreferences` with a default value
2. Add it to `to_dict()` and `from_dict()`
3. `dialogs.py`: Add the widget in `PreferencesDialog._build_perf_tab()` (or another tab)
4. Add it to `_load_preferences()` and `get_preferences()`
5. Use the preference via `getattr(self.controller.state.preferences, 'my_pref', default)`

Testing and Debugging

- **Application logger**: `from src.core.app_logger import get_logger; _log = get_logger('my_module')`
- **Journal**: Computation → Application Journal (F7)
- **LMGC90 Logs**: Computation → View LMGC90 Logs (F6)
- **Dynamic Variables**: Tools → Dynamic Variables (Ctrl+V)

Appendix — Quick Reference Files

Need	File
Add/modify a data type	<code>src/core/models.py</code>
Modify validation	<code>src/core/validators.py</code>
Modify the pymgc90 call	<code>src/core/pymgc_bridge.py</code>
Add business logic	<code>src/controllers/project_controller.py</code>
Modify a tab's UI	<code>src/views/tabs/<name>_tab.py</code>
Modify the 3D view	<code>src/gui/dialogs/viewer_3d.py</code>
Modify the generated pre.py script	<code>src/utils/script_generator.py</code>
Modify the generated chipy script	<code>src/utils/compute_script_generator.py</code>
Modify expression evaluation	<code>src/utils/safe_eval.py</code>
Modify JSON serialization	<code>src/core/serializers.py</code> + <code>models.py</code>